

Second-order generalised algebraic theories, by examples (under construction)

Ambrus Kaposi

September 11, 2026

In these notes, we study non-substructural programming languages at a high level of abstraction: a language is a second-order generalised algebraic theory (SOGAT). The syntax of such a language is its initial model, in which there are only well-typed (intrinsic) terms quotiented by conversion, every operation is automatically a congruence with respect to conversion and every operation is stable under substitution. These notes are pedagogical excerpts of the papers [KX24, BKS23]. A more categorical treatment can be found in the PhD thesis of Rafaël Bocquet [Boc25].

Related ideas are higher-order abstract syntax [Hof99], logical frameworks [HHP93], two-level type theories [ACKS23], synthetic Tait computability [Ste22]. We will rely on being able to work informally in type theory (informal Agda, Coq, Idris or Lean).

These notes can be formalised in the following metatheories: observational type theory, extensional type theory with quotient inductive-inductive types and propositional extensionality, homotopy type theory, constructive set theory.

These notes were originally written for the International School on Logical Frameworks and Proof Systems Interoperability (LFPSI) which was in Orsay between 8–11 September 2025.

Thanks to Andreas Avoukatos, Levente Fodor, Botond Mészáros and Vojtěch Štěpančík for fixing typos.

Sections compulsory for the type systems course at ELTE in Autumn 2025: 1, 2.3, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7.2, 3.8.

Contents

1	Levels of abstraction	2
2	Derivability and admissibility in GATs	6
2.1	Monoid	6
2.2	Pointed set with endofunction	8
2.3	An expression language	9
2.3.1	Type inference	13
2.4	Simply typed combinator calculus	14
2.5	Other GATs	17
3	Derivability in SOGATs	18
3.1	Simply typed lambda calculus	18
3.1.1	Bidirectional type checking	22
3.2	System T	24
3.3	Finite types	26
3.4	Generic programming	28
3.5	Inductive types	30
3.6	Coinductive types	33
3.7	Polymorphism	34
3.7.1	Hindley–Milner	35
3.7.2	System F	36
3.7.3	Kinds	38
3.8	Fixpoint operator for types and terms, untyped calculi	39
3.8.1	PCF	39
3.8.2	Recursive types	41

3.8.3	Untyped lambda calculus	42
3.9	Martin-Löf type theory	42
3.10	Primitive recursive arithmetic	43
3.11	First-order logic	44
3.12	Theories of signatures for (SO)(G)ATs	44
4	Converting SOGATs into GATs	46
4.1	STLC	46
4.2	The general translation	48
5	Admissibility in SOGATs	52
5.1	Proofs about closed syntactic terms	52
5.2	Internal languages of presheaf models of MLTT	53
5.3	Proofs about open syntactic terms	54

1 Levels of abstraction

A language can be described in different ways ranging from concrete to abstract, see Figure 1. In this section, we explain the different levels briefly via a simple expression language (Razor, see Subsection 2.3). The following example program can be written in Razor.

```
if isZero (num 0 + num 1) then false else isZero (num 0)
```

A Razor program is either a numeric or a boolean expression. Numbers can be formed using `num i` where `i` is a natural number. Booleans are `true` or `false`. We have the usual if-then-else operator, addition and an `isZero` operator which says whether a number is 0. The above program evaluates (runs) in the following steps (each new line is a step).

```
if isZero (num 0 + num 1) then false else isZero (num 0)
if isZero (num 1) then false else isZero (num 0)
if false then false else isZero (num 0)
isZero (num 0)
true
```

Figure 2 gives complete descriptions of Razor at levels of abstraction (1)–(5):

- (1) As a first approximation, a Razor program is a string, that is, a sequence of (ASCII) characters. This is how we write programs on a computer. Any string is a program. Many strings do not correspond to meaningful programs in our language such as `num 3 - num 2` as we don't have subtraction. Also, there are different strings which represent the same program. For example, `isZero (num 1)` and `isZero (num 1)` are different as strings but should be the same programs as the extra spaces after `isZero` shouldn't matter. Instead of describing which strings are meaningful programs and defining an equivalence relation for identifying strings that represent the same program, we will describe programs using a more abstract structure.
- (2) The more abstract structure is list of lexical elements. The language then given by the set of possible lexical elements $\{ (,), \text{true}, \text{false}, \text{if}, \text{then}, \text{else}, \text{num}, \text{isZero}, +, 0, 1, 2, 3, \dots \}$. Now we have much fewer programs and `num 3 - num 2` is not a program anymore because there is no lexical element for `-`. Any two programs given as strings which differ only in the number of spaces will end up as the same program at this level: `isZero (num 1)` and `isZero (num 1)` are both given by the sequence $[\text{isZero}, (, \text{num}, 1,)]$. However, we still have meaningless programs, e.g. $[(, \text{true}]$ (there is no closing parenthesis) or $[\text{num}, 1, +]$ (+ needs two arguments), and so on. Also, there are programs which could be identified, e.g. $[(, \text{true},)]$ and $[\text{true}]$ (the parentheses are redundant in the former). Again, to solve these issues, we move to a higher-level representation of programs. Note that there are standard ways to navigate between levels (1) and (2): (2) to (1) is printing. (1) to (2) is performed by a lexical analyser (lexer) which turns a string into a sequence of lexical elements or returns an error.
- (3) At this level, a program is an abstract syntax tree (AST). The programming language is given by the list of operators and their arities (arity of an operator: number of its parameters). One way to specify the operators and their arities is by a BNF-like notation:

$$t ::= \text{true} \mid \text{false} \mid \text{ite } t_1 t_2 \mid \text{num } n \mid \text{isZero } t \mid t + t \qquad n ::= 0 \mid 1 \mid 2 \mid \dots$$

Another way is simply listing how many operators are their for each arity:

arity (number parameters)	0	1	2	3	...
number of operators	$2 + \infty$	1	1	1	0

A third way is writing down the (meta)type of each operation as in Figure 2 part (3).

At this level, we only have well-parenthesised expressions and each operator receives the correct number of arguments. Programs are now trees which have **true**, **false** or **num i** at their leaves and they can have ternary branching with **ite** at the branching node, unary branching with **isZero** at the node or binary branching with **+** at the node.

- (4) In well-typed (intrinsic) syntax trees, the types of the arguments are restricted to the correct ones. The description of the language is a set of derivation rules (Notation 25), or simply the (meta)types of each operation, see Figure 2 part (4). A program is a derivation tree.
- (5) In well-typed quotiented syntax, two programs which have the same result are identified. Again, the language is described by a set of derivation rules, but these can also derive equations. A program is a derivation tree (two programs are identified if there we can derive an equation between them).

Abstract binding trees are not relevant for Razor as there are no variables.

The *extrinsic* approach moves to more abstract levels by defining relations which select the well-formed (well-scoped, well-typed, etc) expressions in the concrete representation. In contrast, we use the *intrinsic* approach where in the more abstract representation the non-well-formed expressions are not even expressible. At level (5), there is no need to prove that conversion preserves typing (“preservation” from “progress and preservation”) because the equations are already expressed in a typed way. Certain properties of the language cannot be expressed at the abstract levels. For example it is not possible to count the number of brackets in a program at level (3). At level (5), it is not possible to reason about program efficiency because all convertible programs are in the same equivalence class: we cannot write a function which distinguishes convertible programs. We view the lower levels as important parts of a programming language, but we see them as intermediate technical steps to reach the most abstract level which describes the essence of the language.

In these notes, we will use the most abstract level to describe languages.

There are several features of programming languages that we do not know how to define intrinsically, and these notes only cover a very small fragment of the theory of programming languages. We direct the curious reader to standard textbooks [Pie02, PdAC⁺23, Har16, WKS22, Csö12, DL11].

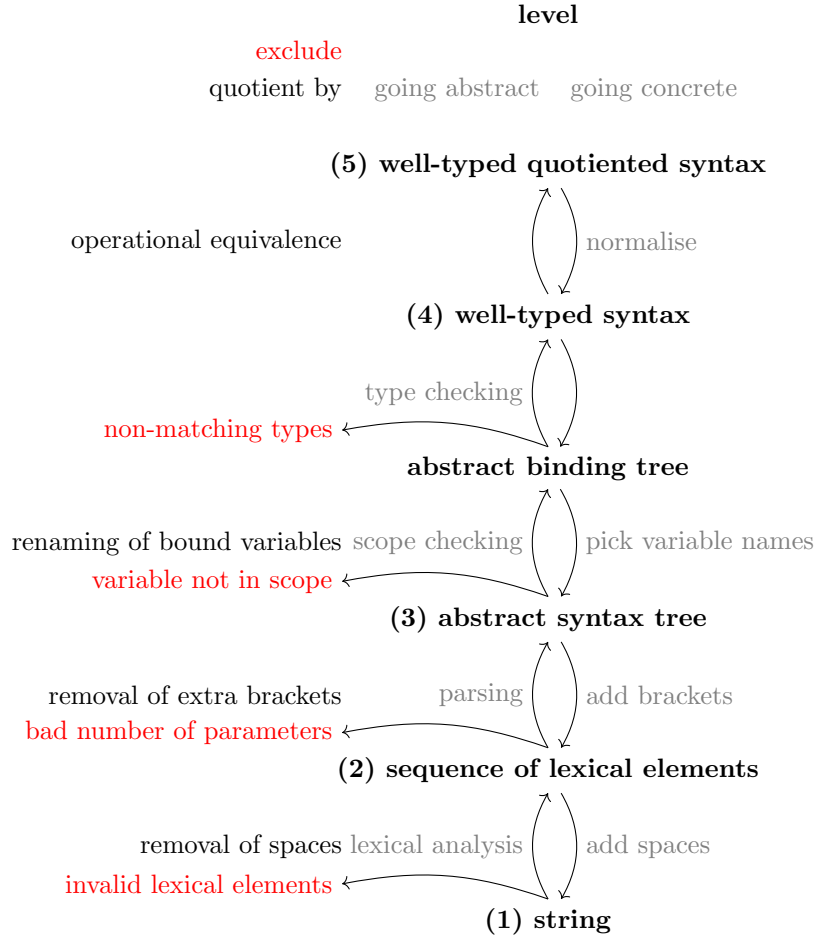
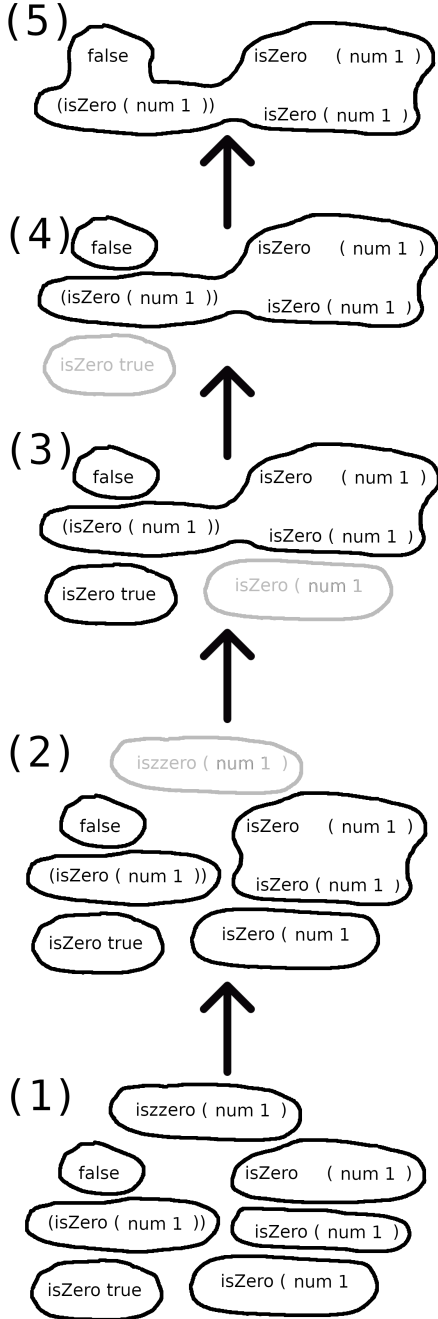


Figure 1: Different levels of abstraction when defining a programming language and transformations between levels. At more abstract levels, certain programs are excluded and others identified. Abstract binding trees are sometimes called well-scoped syntax trees.



(5) well-typed quotiented syntax

```

Ty      : Set
Tm      : Ty → Set
Bool    : Ty
Nat     : Ty
true    : Tm Bool
false   : Tm Bool
ite     : Tm Bool → Tm A → Tm A → Tm A
num     : ℕ → Tm Nat
isZero  : Tm Nat → Tm Bool
_+_     : Tm Nat → Tm Nat → Tm Nat
iteβ1  : ite true u v = u
iteβ2  : ite false u v = v
isZeroβ1 : isZero (num 0) = true
isZeroβ2 : isZero (num (1+n)) = false
+β      : num m + num n = num (m + n)

```

(4) well-typed syntax

```

Ty      : Set
Tm      : Ty → Set
Bool    : Ty
Nat     : Ty
true    : Tm Bool
false   : Tm Bool
ite     : Tm Bool → Tm A → Tm A → Tm A
num     : ℕ → Tm Nat
isZero  : Tm Nat → Tm Bool
_+_     : Tm Nat → Tm Nat → Tm Nat

```

(3) abstract syntax tree

```

Tm      : Set
true    : Tm
false   : Tm
ite     : Tm → Tm → Tm → Tm
num     : ℕ → Tm
isZero  : Tm → Tm
_+_     : Tm → Tm → Tm

```

(2) list of the following lexical elements:

(,), true, false, if, then, else, num, isZero, +,
0, 1, 2, 3, ...

(1) any string

Figure 2: Left: example Razor programs at different levels of abstraction. Each bubble represents a separate program. Right: description of the Razor expression language at levels (1)–(5).

2 Derivability and admissibility in GATs

As a warmup, we review how languages without binders can be seen as generalised algebraic theories (GATs). The goal of this section is to familiarise the reader with the following concepts: model, derivability, morphism, dependent model, dependent morphism, syntax, induction, iteration, admissibility, logical consistency, equational consistency, normal forms, normalisation. These concepts are specific to the particular GAT, and we show what they are for different example GATs. By the end of this section, the reader should be able to formulate them for any GAT (except normal forms which only exist for certain languages).

2.1 Monoid

We start with a well-known algebraic theory (AT): monoids. A model of the theory of monoid is also called a monoid algebra or simply monoid.

Definition 1 (Monoid). *A monoid model comprises the following components:*

$$\begin{aligned} C & : \text{Set} \\ - \cdot - & : C \rightarrow C \rightarrow C \\ \text{ass} & : x \cdot (y \cdot z) = (x \cdot y) \cdot z \\ u & : C \\ \text{idl} & : u \cdot x = x \\ \text{idr} & : x \cdot u = x \end{aligned}$$

A model contains one carrier set (sort), two operations (one binary and one nullary) which satisfy three equations.

Example models (the equations also hold, but we don't write their proofs):

$C := \mathbb{N}$	$C := \mathbb{N}$	$C := \{*\}$	$C := \{\text{tt}, \text{ff}\}$
$x \cdot y := x + y$	$x \cdot y := x * y$	$x \cdot y := *$	$x \cdot y := x \wedge y$
$u := 0$	$u := 1$	$u := *$	$u := \text{tt}$

Exercise 2. *Prove the equations for the above four monoids. Define all the monoids with sort $\{\text{tt}, \text{ff}\}$. Define the following monoids: strings with concatenation, square matrices with multiplication, $A \rightarrow A$ functions with composition, subsets of A with intersection/union.*

Non-examples: C is the empty set, $C = \mathbb{N}$ with exponentiation, $C = \mathbb{Z}$ with subtraction. We give names to the models and refer to their components via subscript. E.g. if we call the above first model M , then $C_M = \mathbb{N}$, $x \cdot_M y = x + y$ and $u_M = 0$.

A *derivable operation* is one that is defined for any model, for example $\text{dup}(x : C) : C := x \cdot x$. A *derivable equation* is one that holds in any model, for example $(u \cdot u) \cdot u = u$ which is derived by

$$(u \cdot u) \cdot u \stackrel{\text{idl}}{=} u \cdot u \stackrel{\text{idl}}{=} u.$$

In a proof assistant, we can define derivable operations and equations by assuming a model as a module parameter / postulate / axiom / variable, and only using this when defining the operation or proving the equation. Then we can specialise the derivable things to particular models, for the above M , we have $\text{dup}_M 3 = 3 \cdot_M 3 = 3 + 3 = 6$ and $(u_M \cdot_M u_M) \cdot_M u_M = (0 + 0) + 0 = 0 + 0 = 0 = u_M$.

A *morphism* (or homomorphism) between models is a function between the carriers that preserves the operations, precisely a morphism from M to N comprises the following components:

$$\begin{aligned} C & : C_M \rightarrow C_N \\ - \cdot - & : (x y : C_M) \rightarrow C (x \cdot_M y) = C x \cdot_N C y \\ u & : C u_M = u_N \end{aligned}$$

Note that there are no components corresponding to the equations (this only changes when the GAT has sort equations).

Exercise 3. *Define all the morphisms between any pair of models from the above four examples.*

A *dependent model* (displayed model, motive and methods of the induction principle) over a model M has the same number of components as a model, and is dependent over them, that is:

$$\begin{aligned} \mathbf{C} & : \mathbf{C}_M \rightarrow \mathbf{Set} \\ - \cdot - & : \mathbf{C} x_M \rightarrow \mathbf{C} y_M \rightarrow \mathbf{C} (x_M \cdot_M y_M) \\ \mathbf{ass} & : x \cdot (y \cdot z) = (x \cdot y) \cdot z \\ \mathbf{u} & : \mathbf{C} u_M \\ \mathbf{idl} & : u \cdot x = x \\ \mathbf{idr} & : x \cdot u = x \end{aligned}$$

Here we used (the somewhat extreme) notation where metavariables have subscripts. $- \cdot -$ has two implicit arguments x_M and y_M , both in $\mathbf{C}_M$ (the equations $\mathbf{ass}$, $\mathbf{idl}$, $\mathbf{idr}$ in the notion of model also had implicit arguments). Note that $\mathbf{ass}$ also depends on $\mathbf{ass}_M$: we have $x : \mathbf{C} x_M$, $y : \mathbf{C} y_M$, $z : \mathbf{C} z_M$ and the left hand side $x \cdot (y \cdot z)$ is in $\mathbf{C} (x_M \cdot_M (y_M \cdot_M z_M))$, the right hand side is in $\mathbf{C} ((x_M \cdot_M y_M) \cdot_M z_M)$. These sets are equal by $\mathbf{ass}_M$. The situation is similar for $\mathbf{idl}$, $\mathbf{idr}$. Examples where $M = (\mathbb{N}, +, 0)$:

$$\begin{aligned} \mathbf{C} n & := \mathbf{Vec} \mathbb{N} n & \mathbf{C} _ & := \{*\} \\ x \cdot y & := x \mathbin{+} y & x \cdot y & := * \\ \mathbf{u} & := [] & \mathbf{u} & := * \end{aligned}$$

Exercise 4. Any model can be turned into a dependent model where we ignore the dependency.

Exercise 5. Any dependent model D over M can be turned into a model together with a morphism into M . The carrier will be $(x_M : \mathbf{C}_M) \times \mathbf{C}_D x_M$ (a dependent Descartes-product, or Σ -type in the metatheory).

A *dependent morphism* (section) from a model M to a dependent model D over M is like a homomorphism, but the function is dependent:

$$\begin{aligned} \mathbf{C} & : (x : \mathbf{C}_M) \rightarrow \mathbf{C}_D x \\ - \cdot - & : (x y : \mathbf{C}_M) \rightarrow \mathbf{C} (x \cdot_M y) = \mathbf{C} x \cdot_D \mathbf{C} y \\ \mathbf{u} & : \mathbf{C} u_M = u_D \end{aligned}$$

The *syntax* is a model from which there is a dependent morphism into any dependent model (the dependent model has to be over the syntax for this to make sense). We denote the syntax by $\mathbf{l}$ (for initial model). The function which takes a dependent model over the syntax and returns the dependent morphism is called *induction* (also called (dependent) eliminator, universal property).

Exercise 6. Show that there is a syntax for monoids (hint: the carrier is a particularly simple set).

Exercise 7. Show that for a given model M , the following two are equivalent:

- there is a dependent morphism into any model over M ,
- there is a unique homomorphism from M into any model (initiality).

Dependent models and morphisms were introduced in order to specify syntax and induction. Special cases of induction are iteration (fold, catamorphism, non-dependent eliminator, interpreter) and recursion (sometimes also called non-dependent eliminator). The syntax has iteration, which means that for any model M , there is a morphism from $\mathbf{l}$ to M . Recursion is the special case of induction where the $\mathbf{C}_\mathbf{l} \rightarrow \mathbf{Set}$ component in the dependent model is a constant function.

Exercise 8. There is an identity morphism from any model to itself. Morphisms can be composed. An isomorphism between models M and N (denoted $M \cong N$) comprises morphisms $M \rightarrow N$ and $N \rightarrow M$ such that there composites are the identity morphisms. Show that any two syntaxes are isomorphic.

An *admissible* operation / equation is one that can be defined / proven for the syntax via induction. For the syntax of monoids, we prove that for any $x : \mathbf{C}_\mathbf{l}$ (note that $\mathbf{l}$ is the syntax), $x = u_\mathbf{l}$ by defining the following dependent model over $\mathbf{l}$:

$$\begin{aligned} \mathbf{C} x & & & := (x = u_\mathbf{l}) \\ (e : x = u_\mathbf{l}) \cdot (e' : x' = u_\mathbf{l}) : x \cdot_\mathbf{l} x' & \stackrel{e}{=} u_\mathbf{l} \cdot_\mathbf{l} x' & \stackrel{e'}{=} u_\mathbf{l} \cdot_\mathbf{l} u_\mathbf{l} & \stackrel{\mathbf{idl}_\mathbf{l}}{=} u_\mathbf{l} \\ \mathbf{u} & & & : u_\mathbf{l} = u_\mathbf{l} \end{aligned}$$

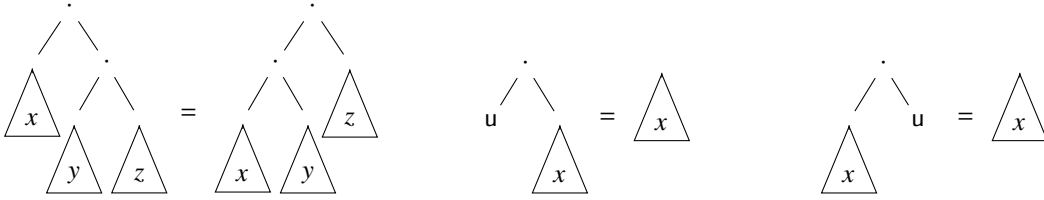
Via induction, we obtain a function $(x : \mathbf{C}_\mathbf{l}) \rightarrow \mathbf{C} x$ which is the same as $(x : \mathbf{C}_\mathbf{l}) \rightarrow x = u_\mathbf{l}$. We say that $x = u$ is an admissible equation, but it is not derivable.

Exercise 9 (From [Moe22]). *Show that the inverse operation is admissible for monoids by defining a dependent model where $\mathbf{C}x = (x^{-1} : \mathbf{C}) \times (x \cdot x^{-1} = \mathbf{u}) \times (x^{-1} \cdot x = \mathbf{u})$.*

For monoids we were able to define the syntax in an ad-hoc way (Exercise 6), but there is a generic way to construct the syntax which works for any algebraic theory. For monoids, we first try to define $\mathbf{C}_1$ as the set of

binary trees where the binary branching nodes denote $\cdot$ and the leaves denote $\mathbf{u}$. The trees  and  denote $(\mathbf{u} \cdot \mathbf{u}) \cdot \mathbf{u}$ and $\mathbf{u} \cdot (\mathbf{u} \cdot \mathbf{u})$, respectively. Actually, we like to draw them as  and  to make the connection between the operations and the nodes / leaves of the tree explicit. This definition of $\mathbf{C}_1$ however

does not suffice. We cannot prove e.g. that the tree  and the tree which only contains the leaf $\mathbf{u}$ are equal, so we cannot provide the component idl_1 . We solve this by *quotienting* the set of binary trees by the three equations ass , idl and idr , that is, the trees which have the following shapes will be identified (where a triangle denotes any tree):



We call these quotiented trees *syntax trees* (we could call them quotiented syntax trees – however we can consider the language of monoids without equations, and the syntax for that language provides the unquotiented monoid syntax trees). One can prove that the model defined like this is a syntax, that is, it has induction. However, we don't do this, we simply assume that there is a syntax (which just means a model with induction). For a generic construction of syntaxes, see e.g. [KKA19]. In a type theory with support for quotient inductive sets (quotient inductive types), one can define the syntax of monoids as the quotient inductive set with two point-constructors $(-\cdot-, \mathbf{u})$ and three equality (path) constructors (ass , idl and idr). The dependent eliminator for this quotient inductive set exactly says that this model has induction.

2.2 Pointed set with endofunction

This language is particularly simple, it does not have equations.

Definition 10 (PSE). *A model comprises the following components:*

$$\begin{aligned} \mathbf{N} &: \mathbf{Set} \\ z &: \mathbf{N} \\ s &: \mathbf{N} \rightarrow \mathbf{N} \end{aligned}$$

A derivable function is e.g. $\text{sss}(n : \mathbf{N}) : \mathbf{N} := s(s(s n))$. There are no interesting derivable equations.

The syntax is $(\mathbf{N}, 0, +1)$, a dependent model over this contains $\mathbf{N} : \mathbf{N} \rightarrow \mathbf{Set}$, $z : \mathbf{N} 0$ and $s : \mathbf{N} n \rightarrow \mathbf{N} (1 + n)$. Induction for $(\mathbf{N}, 0, +1)$ thus says the usual notion of induction: given a (proof-relevant) predicate which holds for 0 and which preserves successor, the predicate holds for all natural numbers. Hence $\mathbf{l}$ is the natural numbers. The fact that natural numbers form an exponential semiring is admissible for the language PSE:

Exercise 11. *Define the admissible operations of addition, multiplication, exponentiation. Prove the admissible equations associativity of addition, left and right identity, commutativity, etc.*

Exercise 12. *Show via induction that $0 \neq 1 + n$ and that $+1$ is injective.*

When we draw syntax trees, they are just unary branching, like $s(s(s z))$:

s
|
s
|
s
|
z

Exercise 13. *Show that induction is equivalent to initiality.*

2.3 An expression language

The following is a simple expression language which is not algebraic, but *generalised* algebraic. There are two sorts, and the second one is indexed over the first one.¹ We call the language Razor following [Hut23].

Definition 14 (Razor). *A model comprises the following components:*

Ty	: Set
Tm	: $\text{Ty} \rightarrow \text{Set}$
Bool	: Ty
Nat	: Ty
true	: Tm Bool
false	: Tm Bool
ite	: $\text{Tm Bool} \rightarrow \text{Tm } A \rightarrow \text{Tm } A \rightarrow \text{Tm } A$
num	: $\mathbb{N} \rightarrow \text{Tm Nat}$
$- + -$	: $\text{Tm Nat} \rightarrow \text{Tm Nat} \rightarrow \text{Tm Nat}$
isZero	: $\text{Tm Nat} \rightarrow \text{Tm Bool}$
$\text{ite}\beta_1$	: $\text{ite true } u \ v = u$
$\text{ite}\beta_2$	: $\text{ite false } u \ v = v$
$+ \beta$	: $\text{num } m + \text{num } n = \text{num } (m + n)$
$\text{isZero}\beta_1$	: $\text{isZero } (\text{num } 0) = \text{true}$
$\text{isZero}\beta_2$	: $\text{isZero } (\text{num } (1 + n)) = \text{false}$

We call elements of Ty types and elements of $\text{Tm } A$ terms of type A . There is no single set of terms, instead for each type, there is a separate set of terms of that type. Each operation is *typed*: their input types and output type is restricted. The operation ite has one implicit type argument A and its first explicit argument needs to have type Bool , while the second and third explicit arguments need to have (the same) type A .

The derivable operations can be seen as programs that can be defined in this language, e.g. $\text{not } (b : \text{Tm Bool}) : \text{Tm Bool} := \text{ite } b \ \text{false} \ \text{true}$. The equations explain how to run the programs, e.g. $\text{not true} = \text{ite true false true} \stackrel{\text{ite}\beta_1}{=} \text{false}$. Another example:

$$\begin{aligned}
& (\text{num } 1 + \text{num } 2) + (\text{num } 3 + \text{num } 4) = (+\beta) \\
& (\text{num } (1 + 2)) + (\text{num } 3 + \text{num } 4) = \\
& \text{num } 3 + (\text{num } 3 + \text{num } 4) = (+\beta) \\
& \text{num } 3 + \text{num } (3 + 4) = \\
& \text{num } 3 + \text{num } 7 = (+\beta) \\
& \text{num } (3 + 7) = \\
& \text{num } 10
\end{aligned}$$

Note that in the expression $\text{num } (1 + 2)$, the $+$ refers to the metatheoretic addition, while in $\text{num } 1 + \text{num } 2$ the $+$ refers to object theoretic addition.

The standard (metacircular, set) model of Razor is where types are sets (metatheoretic types) and terms

¹Certain such GATs can be reduced to ATs by replacing the Ty -indexing with a function from Tm to Ty . For Razor, this is not the case because after such a reduction, the ite operation becomes partial. The resulting theory is called an essentially algebraic theory (EAT). EATs are another extension of algebraic theories. GATs and EATs are equivalent in the sense that for each GAT, there is an EAT with an equivalent category of models, and for each EAT, there is a GAT with an equivalent category of models.

are elements of the sets (metatheoretic terms):

$$\begin{aligned}
\text{Ty} &:= \text{Set} \\
\text{Tm } A &:= A \\
\text{Bool} &:= \{\text{tt}, \text{ff}\} \\
\text{Nat} &: \mathbb{N} \\
\text{true} &:= \text{tt} \\
\text{false} &:= \text{ff} \\
\text{ite } b \ t \ f &:= \text{match } b \ \{\text{tt} \mapsto t; \text{ff} \mapsto f\} \\
\text{num } n &:= n \\
u + v &:= u + v \\
\text{isZero } u &:= \text{match } u \ \{0 \mapsto \text{tt}; (1 + n) \mapsto \text{ff}\}
\end{aligned}$$

We do not write metatheoretic universe levels, hence informally $\text{Set} : \text{Set}$, but formally types in the standard model are a large metatheoretic type (they are in Set_1). In the definition of ite and isZero we used pattern matching on elements of metatheoretic booleans $\{\text{tt}, \text{ff}\}$. In the standard model, all equations hold by reflexivity, as they hold definitionally in the metatheory.

Exercise 15. *If in any model $\text{true} = \text{false}$ then for any A and $u, v : \text{Tm } A$, we have $u = v$.*

Exercise 16. *If in any model $\text{num } 0 = \text{num } 1$ then for any A and $u, v : \text{Tm } A$, we have $u = v$.*

Exercise 17. *There is a model where $\text{num } 1 = \text{num } 2$, but $\text{true} \neq \text{false}$.*

Exercise 18. *There is a model where Tm Bool has three elements (nonstandard model).*

Exercise 19. *Prove or disprove the following statements.*

- *There is a model where $\text{true} \neq \text{false}$ and for every t and u , $\text{ite } t \ u \ u = u$.*
- *In every model for every t and u , $\text{ite } t \ u \ u = u$.*
- *There is a model in which $\text{isZero } (\text{num } 0) = \text{false}$.*
- *In every model $\text{isZero } (\text{num } 3) = \text{isZero } (\text{num } 5)$.*
- *There is no model in which $\text{Tm Bool} = \mathbb{N}$.*

A morphism $M \rightarrow N$ contains two functions, one for types and one for terms. The latter refers to the former:

$$\begin{aligned}
\text{Ty} &: \text{Ty}_M \rightarrow \text{Ty}_N \\
\text{Tm} &: \text{Tm}_M \ A_M \rightarrow \text{Tm}_N \ (\text{Ty } A_M) \\
\text{Bool} &: \text{Ty Bool}_M = \text{Bool}_N \\
\text{Nat} &: \text{Ty Nat}_M = \text{Nat}_N \\
\text{true} &: \text{Tm true}_M = \text{true}_N \\
\text{false} &: \text{Tm false}_M = \text{false}_N \\
\text{ite} &: (b_M : \text{Tm}_M \ \text{Bool}_M) (t_M \ f_M : \text{Tm}_M \ A) \rightarrow \text{Tm} (\text{ite}_M \ b_M \ t_M \ f_M) = \text{ite}_N \ (\text{Tm } b_M) \ (\text{Tm } t_M) \ (\text{Tm } f_M) \\
\text{num} &: (n : \mathbb{N}) \rightarrow \text{Tm} (\text{num}_M \ n) = \text{num}_N \ n \\
- + - &: (u_M \ v_M : \text{Tm}_M \ \text{Nat}_M) \rightarrow \text{Tm} (u_M +_M v_M) = \text{Tm } u_M +_N \text{Tm } v_M \\
\text{isZero} &: (u_M : \text{Tm}_M \ \text{Nat}_M) \rightarrow \text{Tm} (\text{isZero}_M \ u_M) = \text{isZero}_N \ (\text{Tm } u_M)
\end{aligned}$$

The equality true depends on the equality Bool as its left hand side is in $\text{Tm}_N \ (\text{Ty Bool}_M)$, while the right hand side is in $\text{Tm}_N \ \text{Bool}_N$. The situation is similar for other term equations.

Theorem 20. *Razor is logically inconsistent, that is, every sort in the syntax has an element.*

Proof. Ty_1 has an element Bool_1 , $\text{Tm}_1 \ \text{Bool}_1$ has an element true_1 , $\text{Tm}_1 \ \text{Nat}_1$ has an element $\text{num}_1 \ 0$. $\square$

Theorem 21. *Razor is equationally consistent, that is, not all syntactic terms are equal, that is, there is a type A_1 and terms $a, a' : \text{Tm } A_1$ such that $a \neq a'$.*

Proof. We choose $A_1 := \text{Bool}_1$ and $a := \text{true}_1$ and $a' := \text{false}_1$, then assuming $a = a'$, their iterations into the standard model are also equal, hence $\text{tt} = \text{ff}$. $\square$

Iteration into the standard model can be called normalisation (recall that this is a morphism from $\mathbf{I}$ to the standard model). Normal forms are given by the $\mathbf{T}_y$ component of iteration which we rename to $\mathbf{N}_f$, and we omit the names of the equations; this looks like a pattern-matching definition:

$$\begin{aligned}\mathbf{N}_f &: \mathbf{T}_y \mathbf{I} \rightarrow \mathbf{Set} \\ \mathbf{N}_f \mathbf{Bool}_\mathbf{I} &= \{\mathbf{tt}, \mathbf{ff}\} \\ \mathbf{N}_f \mathbf{Nat}_\mathbf{I} &= \mathbb{N}\end{aligned}$$

The $\mathbf{T}_m$ component of the iteration morphism gives the normalisation function, we rename it to $\mathbf{norm}$, its computation rules are the components of the morphism for the term operators:

$$\begin{aligned}\mathbf{norm} &: \mathbf{T}_m \mathbf{I} A_\mathbf{I} \rightarrow \mathbf{N}_f A_\mathbf{I} \\ \mathbf{norm} \mathbf{true}_\mathbf{I} &= \mathbf{tt} \\ \mathbf{norm} \mathbf{false}_\mathbf{I} &= \mathbf{ff} \\ \mathbf{norm} (\mathbf{ite}_\mathbf{I} b_\mathbf{I} t_\mathbf{I} f_\mathbf{I}) &= \mathbf{match} (\mathbf{norm} b_\mathbf{I}) \{\mathbf{tt} \mapsto \mathbf{norm} t_\mathbf{I}; \mathbf{ff} \mapsto \mathbf{norm} f_\mathbf{I}\} \\ \mathbf{norm} (\mathbf{num}_\mathbf{I} n) &= n \\ \mathbf{norm} (u_\mathbf{I} +_\mathbf{I} v_\mathbf{I}) &= \mathbf{norm} u_\mathbf{I} + \mathbf{norm} v_\mathbf{I} \\ \mathbf{norm} (\mathbf{isZero}_\mathbf{I} u_\mathbf{I}) &= \mathbf{match} (\mathbf{norm} u_\mathbf{I}) \{0 \mapsto \mathbf{tt}; (1 + n) \mapsto \mathbf{ff}\}\end{aligned}$$

A dependent model over a model M comprises the following components:

$$\begin{aligned}\mathbf{T}_y &: \mathbf{T}_y M \rightarrow \mathbf{Set} \\ \mathbf{T}_m &: \{A_M : \mathbf{T}_y M\} \rightarrow \mathbf{T}_y A_M \rightarrow \mathbf{T}_m M A_M \rightarrow \mathbf{Set} \\ \mathbf{Bool} &: \mathbf{T}_y \mathbf{Bool}_M \\ \mathbf{Nat} &: \mathbf{T}_y \mathbf{Nat}_M \\ \mathbf{true} &: \mathbf{T}_m \{\mathbf{Bool}_M\} \mathbf{Bool} \mathbf{true}_M \\ \mathbf{false} &: \mathbf{T}_m \{\mathbf{Bool}_M\} \mathbf{Bool} \mathbf{false}_M \\ \mathbf{ite} &: \mathbf{T}_m \mathbf{Bool} b_M \rightarrow \mathbf{T}_m A t_M \rightarrow \mathbf{T}_m A f_M \rightarrow \mathbf{T}_m A (\mathbf{ite}_M b_M t_M f_M) \\ \mathbf{num} &: (n : \mathbb{N}) \rightarrow \mathbf{T}_m \mathbf{Nat} (\mathbf{num}_M n) \\ - + - &: \mathbf{T}_m \mathbf{Nat} u_M \rightarrow \mathbf{T}_m \mathbf{Nat} v_M \rightarrow \mathbf{T}_m \mathbf{Nat} (u_M +_M v_M) \\ \mathbf{isZero} &: \mathbf{T}_m \mathbf{Nat} u_M \rightarrow \mathbf{T}_m \mathbf{Bool} (\mathbf{isZero}_M u_M) \\ \mathbf{ite}\beta_1 &: \mathbf{ite} \mathbf{true} u v = u \\ \mathbf{ite}\beta_2 &: \mathbf{ite} \mathbf{false} u v = v \\ +\beta &: \mathbf{num} m + \mathbf{num} n = \mathbf{num} (m + n) \\ \mathbf{isZero}\beta_1 &: \mathbf{isZero} (\mathbf{num} 0) = \mathbf{true} \\ \mathbf{isZero}\beta_2 &: \mathbf{isZero} (\mathbf{num} (1 + n)) = \mathbf{false}\end{aligned}$$

Note that the equations in the dependent model depend on the corresponding equations in M . For example, the left hand side of $\mathbf{ite}\beta_1$ is in $\mathbf{T}_m \{A_M\} A (\mathbf{ite}_M \mathbf{true}_M u_M v_M)$, while the right hand side is in $\mathbf{T}_m \{A_M\} A u_M$.

The syntax supports induction, which means that there is the following dependent morphism from the syntax to a dependent model D over it:

$$\begin{aligned}\mathbf{T}_y &: (A_\mathbf{I} : \mathbf{T}_y \mathbf{I}) \rightarrow \mathbf{T}_y D A_\mathbf{I} \\ \mathbf{T}_m &: (a_\mathbf{I} : \mathbf{T}_m \mathbf{I} A_\mathbf{I}) \rightarrow \mathbf{T}_m D (\mathbf{T}_y A_\mathbf{I}) a_\mathbf{I} \\ \mathbf{Bool} &: \mathbf{T}_y \mathbf{Bool}_\mathbf{I} = \mathbf{Bool}_D \\ \mathbf{Nat} &: \mathbf{T}_y \mathbf{Nat}_\mathbf{I} = \mathbf{Nat}_D \\ \mathbf{true} &: \mathbf{T}_m \mathbf{true}_\mathbf{I} = \mathbf{true}_D \\ \mathbf{false} &: \mathbf{T}_m \mathbf{false}_\mathbf{I} = \mathbf{false}_D \\ \mathbf{ite} &: (b_\mathbf{I} : \mathbf{T}_m \mathbf{Bool}_\mathbf{I})(t_\mathbf{I} f_\mathbf{I} : \mathbf{T}_m \mathbf{I} A_\mathbf{I}) \rightarrow \mathbf{T}_m (\mathbf{ite}_\mathbf{I} b_\mathbf{I} t_\mathbf{I} f_\mathbf{I}) = \mathbf{ite}_D (\mathbf{T}_m b_\mathbf{I}) (\mathbf{T}_m t_\mathbf{I}) (\mathbf{T}_m f_\mathbf{I}) \\ \mathbf{num} &: (n : \mathbb{N}) \rightarrow \mathbf{T}_m (\mathbf{num}_\mathbf{I} n) = \mathbf{num}_D n \\ - + - &: (u_\mathbf{I} v_\mathbf{I} : \mathbf{T}_m \mathbf{Nat}_\mathbf{I}) \rightarrow \mathbf{T}_m (u_\mathbf{I} +_\mathbf{I} v_\mathbf{I}) = \mathbf{T}_m u_\mathbf{I} +_D \mathbf{T}_m v_\mathbf{I} \\ \mathbf{isZero} &: (u_\mathbf{I} : \mathbf{T}_m \mathbf{Nat}_\mathbf{I}) \rightarrow \mathbf{T}_m (\mathbf{isZero}_\mathbf{I} u_\mathbf{I}) = \mathbf{isZero}_D (\mathbf{T}_m u_\mathbf{I})\end{aligned}$$

We define a dependent model where the term components are trivial (we don't list them):

$$\begin{aligned}
\text{Ty } A_I &:= \text{Nf } A_I \rightarrow \text{Tm}_I A_I \\
\text{Tm } A a_I &:= \mathbb{1} \\
\text{Bool} &: \underbrace{\text{Nf Bool}_I}_{=\{\text{tt}, \text{ff}\}} \rightarrow \text{Tm}_I \text{Bool}_I \\
\text{Bool } b &:= \text{match } b \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \} \\
\text{Nat} &: \underbrace{\text{Nf Nat}_I}_{=\mathbb{N}} \rightarrow \text{Tm}_I \text{Nat}_I \\
\text{Nat } n &:= \text{num}_I n
\end{aligned}$$

Induction into this dependent model provides us with a quote function which maps normal forms back into syntactic terms:

$$\begin{aligned}
\text{quote} &: (A_I : \text{Ty}_I) \rightarrow \text{Nf } A_I \rightarrow \text{Tm}_I A_I \\
\text{quote Bool}_I b &= \text{match } b \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \} \\
\text{quote Nat}_I n &= \text{num}_I n
\end{aligned}$$

Completeness of normalisation says that for any $a_I : \text{Tm}_I A_I$, $\text{quote } A_I (\text{norm } a_I) = a_I$. We prove this by constructing another dependent model over I where the Ty component is trivial:

$$\begin{aligned}
\text{Ty } A_I &:= \mathbb{1} \\
\text{Tm } \{A_I\} * a_I &:= (\text{quote } A_I (\text{norm } a_I) = a_I) \\
\text{true} &: \text{quote Bool}_I (\text{norm true}_I) = \text{quote Bool}_I \text{tt} = \text{match tt} \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \} = \text{true}_I \\
\text{false} &: \text{quote Bool}_I (\text{norm false}_I) = \text{quote Bool}_I \text{ff} = \text{match ff} \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \} = \text{false}_I \\
\text{ite } (e_b : \text{quote Bool}_I (\text{norm } b_I) = b_I) (e_t : \text{quote } A_I (\text{norm } t_I) = t_I) (e_f : \text{quote } A_I (\text{norm } f_I) = f_I) & \\
&: \text{quote } A_I (\text{norm } (\text{ite}_I b_I t_I f_I)) & \\
&\quad \text{quote } A_I (\text{match } (\text{norm } b_I) \{ \text{tt} \mapsto \text{norm } t_I; \text{ff} \mapsto \text{norm } f_I \}) &= (\text{norm } b_I = \text{tt}) \\
&\quad \text{quote } A_I (\text{match tt} \{ \text{tt} \mapsto \text{norm } t_I; \text{ff} \mapsto \text{norm } f_I \}) &= \\
&\quad \text{quote } A_I (\text{norm } t_I) &= (\text{ite } \beta_{1I}) \\
&\quad \text{ite}_I \text{true}_I (\text{quote } A_I (\text{norm } t_I)) (\text{quote } A_I (\text{norm } f_I)) &= \\
&\quad \text{ite}_I (\text{match tt} \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \}) (\text{quote } A_I (\text{norm } t_I)) (\text{quote } A_I (\text{norm } f_I)) &= (\text{norm } b_I = \text{tt}) \\
&\quad \text{ite}_I (\text{match } (\text{norm } b_I) \{ \text{tt} \mapsto \text{true}_I; \text{ff} \mapsto \text{false}_I \}) (\text{quote } A_I (\text{norm } t_I)) (\text{quote } A_I (\text{norm } f_I)) = & \\
&\quad \text{ite}_I (\text{quote Bool}_I (\text{norm } b_I)) (\text{quote } A_I (\text{norm } t_I)) (\text{quote } A_I (\text{norm } f_I)) &= (e_b, e_t, e_f) \\
&\quad \text{ite}_I b_I t_I f_I &
\end{aligned}$$

As terms in this dependent model are equations between elements of $\text{Tm}_I A_I$ for some A_I , we don't have to provide the equation components ($\text{Tm}_I A_I$ is a set in the sense of homotopy type theory, it has uniqueness of identity proofs).

Exercise 22. *Finish defining the dependent model: write the case $\text{norm } b_I = \text{ff}$ for ite and define the components num , $- + -$, isZero .*

Exercise 23. *Merge quote and completeness , so that they both are parts of induction into a single dependent model.*

Exercise 24. *Prove stability: that is, given a normal form, if we quote it and then normalise it, we get back the same normal form. In summary, normalisation with completeness and stability says that there is an isomorphism between the sets $\text{Tm}_I A_I$ and $\text{Nf } A_I$.*

In general, *normal forms* are a complete representation of the syntax where equality is easily decidable (for example, they are given by an inductive definition, that is, the syntax of a GAT without equations). Decidable equality of normal forms implies decidable equality of terms by completeness: given $a_I, a_I' : \text{Tm}_I A_I$, if $e : \text{norm } a_I = \text{norm } a_I'$ then

$$a_I \stackrel{\text{completeness}}{=} \text{quote } (\text{norm } a_I) \stackrel{e}{=} \text{quote } (\text{norm } a_I') \stackrel{\text{completeness}}{=} a_I',$$

and given $\text{norm } a_I \neq \text{norm } a_I'$, from $a_I = a_I'$, by congruence we get $\text{norm } a_I = \text{norm } a_I'$. Stability is not needed for decidability of equality of the syntax, but it is useful to obtain a new induction principle for the syntax: one can prove things about the syntax by induction on normal forms.

Notation 25 (Derivation rules). *Elements of the syntax can be depicted by syntax trees, but then the well-*

typedness of the tree is not immediately visible. Drawing the tree

$$\begin{array}{c} \text{ite} \\ \swarrow \quad \downarrow \quad \searrow \\ \text{true} \quad \text{num } 1 \quad \text{false} \end{array}$$
does not reveal that this term does not exist. Instead, we first describe the notion of model using derivation rule notation which means that we uncurry all the operators, give names to the arguments and in each operator, we replace the remaining $\rightarrow$ with a horizontal line:

$$\begin{array}{c} \overline{\text{Ty} : \text{Set}} \quad \overline{A : \text{Ty}} \quad \overline{\text{Tm } A : \text{Set}} \quad \overline{\text{Bool} : \text{Ty}} \quad \overline{\text{Nat} : \text{Ty}} \quad \overline{\text{true} : \text{Tm Bool}} \quad \overline{\text{false} : \text{Tm Bool}} \\ \hline \overline{b : \text{Tm Bool} \quad t : \text{Tm } A \quad f : \text{Tm } A} \quad \overline{n : \mathbb{N}} \quad \overline{u : \text{Tm Nat} \quad v : \text{Tm Nat}} \quad \overline{u : \text{Tm Nat}} \\ \hline \text{ite } b \ t \ f : \text{Tm } A \quad \text{num } n : \text{Tm Nat} \quad u + v : \text{Tm Nat} \quad \text{isZero } u : \text{Tm Bool} \\ \hline \text{ite}\beta_1 : \text{ite true } u \ v = u \quad \text{ite}\beta_2 : \text{ite false } u \ v = v \quad +\beta : \text{num } m + \text{num } n = \text{num } (m + n) \\ \hline \text{isZero}\beta_1 : \text{isZero (num 0)} = \text{true} \quad \text{isZero}\beta_2 : \text{isZero (num (1 + n))} = \text{false} \end{array}$$

Just as in the algebraic notation we have implicit arguments, for example $A : \text{Ty}$ is not written above the line for ite, but we assume it is (implicitly) there.

Notation 26 (Derivation trees). *Derivation trees are like upside-down syntax trees where at the nodes the full term and its set are both written; the children are subterms just as before. Example:*

$$\begin{array}{c} \overline{\text{num } 1 : \text{Tm Nat}} \\ \hline \overline{\text{isZero (num 1)} : \text{Tm Bool}} \quad \overline{\text{true} : \text{Tm Bool}} \quad \overline{\text{false} : \text{Tm Bool}} \\ \hline \text{ite (isZero (num 1)) true false} : \text{Tm Bool} \end{array}$$

Each horizontal line in the derivation tree is a special case of one of the derivation rules.

It is clear that the following tree cannot be finished:

$$\begin{array}{c} \overline{\text{true} : \text{Tm Bool}} \quad \overline{\text{num } 1 : \text{Tm Nat}} \quad \overline{\text{false} : \text{Tm Nat}} \\ \hline \text{ite true (num 1) false} : \text{Tm Nat} \end{array}$$

Exercise 27. *Show that induction is equivalent to initiality.*

2.3.1 Type inference

The AST-level definition of Razor is the following.

Definition 28 (Untyped Razor). *A model comprises the following components:*

$$\begin{array}{ll} \text{Tm} & : \text{Set} \\ \text{true} & : \text{Tm} \\ \text{false} & : \text{Tm} \\ \text{ite} & : \text{Tm} \rightarrow \text{Tm} \rightarrow \text{Tm} \rightarrow \text{Tm} \\ \text{num} & : \mathbb{N} \rightarrow \text{Tm} \\ - + - & : \text{Tm} \rightarrow \text{Tm} \rightarrow \text{Tm} \\ \text{isZero} & : \text{Tm} \rightarrow \text{Tm} \end{array}$$

We define the following model of untyped Razor referring to the syntax of (typed) Razor. (Actually any model with decidability of equality for Ty suffices.)

Definition 29 (Type inference model).

$$\begin{array}{ll} \text{Tm} & := \text{Maybe } ((A : \text{Ty}) \times \text{Tm } A) \\ \text{true} & := \text{just (Bool, true)} \\ \text{false} & := \text{just (Bool, false)} \\ \text{ite } t \ u \ v & := \text{match } (t, u, v) \{ (\text{just (Bool, } t'), \text{just (Bool, } u'), \text{just (Bool, } v')) \mapsto \text{just (Bool, ite } t' \ u' \ v'); \\ & \quad (\text{just (Bool, } t'), \text{just (Nat, } u'), \text{just (Nat, } v')) \mapsto \text{just (Nat, ite } t' \ u' \ v'); \\ & \quad _ \mapsto \text{nothing} \} \\ \text{num } n & := \text{just (Nat, num } n) \\ u + v & := \text{match } (u, v) \{ (\text{just (Nat, } u'), \text{just (Nat, } v')) \mapsto \text{just (Nat, } u' + v'); _ \mapsto \text{nothing} \} \\ \text{isZero } t & := \text{match } t \{ \text{just (Nat, } t') \mapsto \text{just (Bool, isZero } t'); _ \mapsto \text{nothing} \} \end{array}$$

Interpretation into this model is type inference. We apologise for the extreme overloading. It is clear from the absence of indices in the input of the function that we refer to the syntax of untyped Razor:

$$\text{infer} : \text{Tm}_I \rightarrow \text{Maybe}((A : \text{Ty}) \times \text{Tm } A)$$

For example, we compute

```
infer (ite (num 1) true false) =
match (infer (num 1), infer true, infer false) { (just (Bool, t'), just (Bool, u'), just (Bool, v')) ↦ just (Bool, ite t' u' v');
                                                  (just (Bool, t'), just (Nat, u'), just (Nat, v')) ↦ just (Nat, ite t' u' v');
                                                  _ ↦ nothing } =
match (just (Nat, num 1), just (Bool, true), just (Bool, false)) {
  (just (Bool, t'), just (Bool, u'), just (Bool, v')) ↦ just (Bool, ite t' u' v');
  (just (Bool, t'), just (Nat, u'), just (Nat, v')) ↦ just (Nat, ite t' u' v');
  _ ↦ nothing } =
nothing.
```

Exercise 30. Show that $\text{infer} (\text{num } 1 + \text{num } 2) = \text{just} (\text{Nat}, (\text{num } 1 + \text{num } 2))$. Show that there is an A and $t : \text{Tm } A$ such that $\text{infer} (\text{num } 0 + (\text{ite} (\text{isZero} (\text{num } 1)) (\text{num } 1) (\text{num } 2))) = \text{just} (A, t)$.

Exercise 31. Define type erasure, a map from well-typed syntactic terms (see Figure 2 level (4) for the definition of well-typed Razor) to untyped terms $\text{erase} : \text{Tm } A \rightarrow \text{Tm}$. Give examples of $t : \text{Tm}$ such that there is no A and $t' : \text{Tm } A$ with $\text{erase } t' = t$. Show soundness of inference, that is, $\text{infer } t = \text{just} (A, t')$ implies $\text{erase } t' = t$. Show completeness of inference, that is, for any $t : \text{Tm } A$, we have $\text{infer} (\text{erase } t) = \text{just} (A, t)$. Using soundness and/or completeness, give generic ways to prove statements such as “for a given t , show that (doesn't) exist A and $t' : \text{Tm } A$ such that $\text{erase } t' = t$ ”.

2.4 Simply typed combinator calculus

Using Moses Schönfinkel's K and S combinators [Sch24], higher order functions can be described without variables.

Definition 32 (STCC). A model comprises the following components:

$$\begin{aligned} \text{Ty} & : \text{Set} \\ \text{Tm} & : \text{Ty} \rightarrow \text{Set} \\ \iota & : \text{Ty} \\ - \Rightarrow - & : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\ - \cdot - & : \text{Tm } (A \Rightarrow B) \rightarrow \text{Tm } A \rightarrow \text{Tm } B \\ \text{K} & : \text{Tm } (A \Rightarrow B \Rightarrow A) \\ \text{S} & : \text{Tm } ((A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \text{K}\beta & : \text{K} \cdot a \cdot b = a \\ \text{S}\beta & : \text{S} \cdot f \cdot g \cdot a = f \cdot a \cdot (g \cdot a) \end{aligned}$$

We added one base type so that the syntax is not empty. Note that $- \Rightarrow -$ and K have two, S has three implicit arguments. $- \Rightarrow -$ is right-associative, application $- \cdot -$ is left-associative.

Exercise 33. Derive the following combinators, their expected definitional behaviour is specified on the right:

$$\begin{array}{ll} \text{I} : \text{Tm } (A \Rightarrow A) & \text{I} \cdot a = a \\ \text{B} : \text{Tm } ((B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) & \text{B} \cdot f \cdot g \cdot a = f \cdot (g \cdot a) \\ \text{C} : \text{Tm } ((A \Rightarrow B \Rightarrow C) \Rightarrow B \Rightarrow A \Rightarrow C) & \text{C} \cdot f \cdot b \cdot a = f \cdot a \cdot b \end{array}$$

Exercise 34. Define the standard model where $\text{Ty} = \text{Set}$, $\text{Tm } A = A$. The interpretation of ι is a parameter of the standard model.

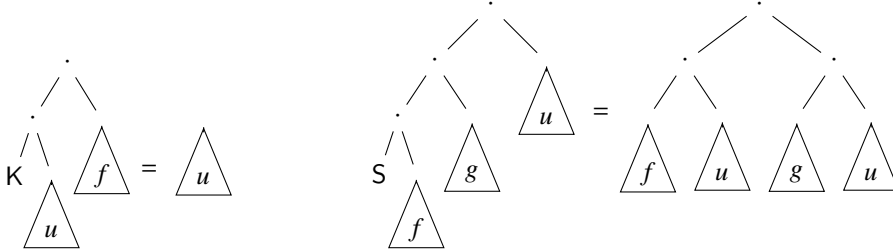
Theorem 35. STCC is logically consistent, that is, there is a type A_I such that $\text{Tm}_I A_I$ is empty.

Proof. Interpreting $\text{Tm}_I \iota_I$ into the standard model with $\iota = \emptyset$ gives an element of $\emptyset$. □

Theorem 36. *STCC is equationally consistent, meaning not all terms are equal. That is, there is a type A_1 and terms $a_1, a'_1 : \text{Tm } A_1$ such that $a_1 \neq a'_1$.*

Proof. We choose $A_1 := (\iota \Rightarrow_1 \iota \Rightarrow_1 \iota)$ and $a_1 := \text{lam}_1 \lambda x_1. \text{lam } \lambda y_1. x_1$ and $a'_1 := \text{lam}_1 \lambda x_1. \text{lam } \lambda y_1. y_1$. We apply the iterator into the standard model with $\iota = 2$ both to a_1 and a'_1 . Assuming $a_1 = a'_1$, their interpretations are also equal, so we get $(\lambda x y. x) = (\lambda x y. y)$, and from this we get $\text{tt} = (\lambda x y. x) \text{tt ff} = (\lambda x y. y) \text{tt ff} = \text{ff}$. $\square$

In the syntax of STCC, types are binary trees (nodes are $\Rightarrow$, leaves are ι). If we ignore types, terms are boolean-labelled binary trees quotiented by the following equations:



If we don't ignore types, then we can only build a node if the left hand subtree is in $\text{Tm } (A \Rightarrow B)$ for some A, B , and the right hand subtree is in $\text{Tm } A$ for the same A . Again, this is enforced by derivation trees, for example:

$$\frac{\frac{S : \text{Tm } ((\iota \Rightarrow (\iota \Rightarrow \iota) \Rightarrow \iota) \Rightarrow ((\iota \Rightarrow \iota \Rightarrow \iota) \Rightarrow \iota \Rightarrow \iota))}{S \cdot K : \text{Tm } ((\iota \Rightarrow \iota \Rightarrow \iota) \Rightarrow \iota \Rightarrow \iota)} \quad \frac{K : \text{Tm } (\iota \Rightarrow (\iota \Rightarrow \iota) \Rightarrow \iota)}{K : \text{Tm } (\iota \Rightarrow \iota \Rightarrow \iota)}}{S \cdot K \cdot K : \text{Tm } (\iota \Rightarrow \iota)}$$

Note that we didn't write implicit arguments. The two K s on the right hand side of the tree are different: there is $K \{ \iota \} \{ \iota \} : \text{Tm } (\iota \Rightarrow \iota \Rightarrow \iota)$ and $K \{ \iota \} \{ \iota \Rightarrow \iota \} : \text{Tm } (\iota \Rightarrow (\iota \Rightarrow \iota) \Rightarrow \iota)$.

Exercise 37. *What is a nice minimal presyntax (untyped AST description) for STCC where types can be always inferred? That is, which implicit arguments have to be written explicitly?*

What are the normal forms for the syntax of STCC? We give an inductive description of the expressions where $K\beta$ and $S\beta$ cannot be applied because K and S does not have enough arguments:

Definition 38 (STCC normal forms). *A normal form model comprises the following:*

$$\begin{aligned} \text{Nf} &: (A_1 : \text{Ty}_1) \rightarrow \text{Tm}_1 A_1 \rightarrow \text{Set} \\ K_0 &: \text{Nf } (A_1 \Rightarrow_1 B_1 \Rightarrow_1 A_1) K_1 \\ K_1 &: \text{Nf } A_1 a_1 \rightarrow \text{Nf } (B_1 \Rightarrow_1 A_1) (K_1 \cdot_1 a_1) \\ S_0 &: \text{Nf } ((A_1 \Rightarrow_1 B_1 \Rightarrow_1 C_1) \Rightarrow_1 (A_1 \Rightarrow_1 B_1) \Rightarrow_1 A_1 \Rightarrow_1 C_1) S_1 \\ S_1 &: \text{Nf } (A_1 \Rightarrow_1 B_1 \Rightarrow_1 C_1) f_1 \rightarrow \text{Nf } ((A_1 \Rightarrow_1 B_1) \Rightarrow_1 A_1 \Rightarrow_1 C_1) (S_1 \cdot_1 f_1) \\ S_2 &: \text{Nf } (A_1 \Rightarrow_1 B_1 \Rightarrow_1 C_1) f_1 \rightarrow \text{Nf } (A_1 \Rightarrow_1 B_1) g_1 \rightarrow \text{Nf } (A_1 \Rightarrow_1 C_1) (S_1 \cdot_1 f_1 \cdot_1 g_1) \end{aligned}$$

These normal forms are indexed not only by their syntactic type, but also by the term they correspond to (the result of quote).

Exercise 39. *Show that equality is decidable in the syntax of normal forms (this relies on equality of syntactic types). The nicest way is to do double-induction on normal forms to prove decidability of equality of the total space of normal forms, that is, the set $(A_1 : \text{Ty}_1) \times (a_1 : \text{Tm}_1 A_1) \times \text{Nf}_1 A_1 a_1$.*

A dependent model over I comprises the following components:

$$\begin{aligned} \text{Ty} &: \text{Ty}_1 \rightarrow \text{Set} \\ \text{Tm} &: \text{Ty } A_1 \rightarrow \text{Tm}_1 A_1 \rightarrow \text{Set} \\ \iota &: \text{Ty } \iota \\ - \Rightarrow - &: \text{Ty } A_1 \rightarrow \text{Ty } B_1 \rightarrow \text{Ty } (A_1 \Rightarrow_1 B_1) \\ - \cdot - &: \text{Tm } (A \Rightarrow B) f_1 \rightarrow \text{Tm } A a_1 \rightarrow \text{Tm } B (f_1 \cdot_1 a_1) \\ K &: \text{Tm } (A \Rightarrow B \Rightarrow A) K_1 \\ S &: \text{Tm } ((A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) S_1 \\ K\beta &: K \cdot a \cdot b = a \\ S\beta &: S \cdot f \cdot g \cdot a = f \cdot a \cdot (g \cdot a) \end{aligned}$$

Given a dependent model D over I , a section comprises the following:

$$\begin{aligned}
\text{Ty} & : (A_I : \text{Ty}_I) \rightarrow \text{Ty}_D A_I \\
\text{Tm} & : (a_I : \text{Tm}_A A_I) \rightarrow \text{Tm}_D (\text{Ty } A_I) a_I \\
\iota & : \text{Ty } \iota_I = \iota_D \\
- \Rightarrow - & : (A B : \text{Ty}_I) \rightarrow \text{Ty } (A_I \Rightarrow_I B_I) = \text{Ty } A_I \Rightarrow_D \text{Ty } B_I \\
- \cdot - & : (f_I : \text{Tm}_I (A_I \Rightarrow_I B_I))(a_I : \text{Tm}_I A_I) \Rightarrow \text{Tm } (f_I \cdot_I a_I) = \text{Tm } f_I \cdot_D \text{Tm } a_I \\
K & : \text{Tm } K_I = K_D \\
S & : \text{Tm } S_I = S_D
\end{aligned}$$

We use Tait's method [Tai67] (also called logical predicate or reducibility method) to prove normalisation for STCC. We define a dependent model over I where types are a proof-relevant predicate over terms together with a reify function, and terms are witnesses of the predicate. The predicate for ι is constant false. The predicate for a function says that if the predicate holds for an input, it also holds for the output, and the function is in normal form.

$$\begin{aligned}
\text{Ty } A_I & := (P_A : \text{Tm}_I A_I \rightarrow \text{Set}) \times (\{a_I : \text{Tm}_I A_I\} \rightarrow P_A a_I \rightarrow \text{Nf } A_I a_I) \\
\text{Tm } (P_A, r_A) a_I & := P_A a_I \\
\iota & := (\lambda _ . \mathbf{0}, \lambda b . \text{match } b \{ \}) \\
(P_A, r_A) \Rightarrow (P_B, r_B) & := \left(\lambda f_I . (\{a_I : \text{Tm}_I A_I\} \rightarrow P_A a_I \rightarrow P_B (f_I \cdot_I a_I)) \times \text{Nf } (A_I \Rightarrow B_I) f_I, \lambda (p_f, n_f) . n_f \right) \\
(p_f, n_f) \cdot p_a & := p_f p_a \\
K \{P_A, r_A\} \{P_B, r_B\} & := \left(\lambda p_a . (\lambda p_b . p_a, K_1 (r_A a_I)), K_0 \right) \\
S \{P_A, r_A\} \{P_B, r_B\} \{P_C, r_C\} & := \left(\lambda (p_f, n_f) . (\lambda (p_g, n_g) . (\lambda p_a . (p_f p_a) \cdot_1 (p_g p_a), S_2 n_f n_g), S_1 n_f), S_0 \right)
\end{aligned}$$

Equations $K\beta$ and $S\beta$ hold by definition. When defining K and S , we implicitly made use of $K\beta_I$ and $S\beta_I$, respectively. By induction into this model we obtain the predicate for each type, the reify function for each type and the witness of the predicate for each term:

$$\begin{aligned}
P & : (A_I : \text{Ty}_I) \rightarrow \text{Tm}_I A_I \rightarrow \text{Set} \\
r & : (A_I : \text{Ty}_I) \{a_I : \text{Tm}_I A_I\} \rightarrow P A_I a_I \rightarrow \text{Nf } A_I a_I \\
p & : (a_I : \text{Tm}_I A_I) \rightarrow P A_I a_I
\end{aligned}$$

We put these together to obtain normalisation:

$$\text{norm } (a_I : \text{Tm}_I A_I) := r A_I a_I (p a_I)$$

Note that in the normalisation dependent model, we only assumed an STCC normal form model, we did not rely on it being syntax. However, if it is the syntax of STCC normal forms, we can do the following:

Exercise 40. *Via normalisation, show decidability of equality for syntactic STCC terms.*

We were quite clever when coming up with the notion of normal forms. We don't need to be clever:

Exercise 41. *Show that the syntax of STCC without equations also suffices for normalisation and decidability of equality of (quotiented) STCC syntax.*

Food for thought 42. *Can we define the quotiented syntax of STCC without using quotients at all (just via normal forms)? Hint: normalisation has to be defined as part of the syntax.*

Food for thought 43. *Can we define a strict syntax of STCC in Agda? A strict syntax is a syntax where both equations hold by definition. Hint: restrict the standard model (Exercise 34) to the definable elements, and this restriction should be in $S\text{Prop}$.*

Exercise 44. *Intuitionistic logic with only implication as a connective is the same as STCC where we don't*

care about equations, a model is the following:

$$\begin{aligned}
\text{For} & : \text{Set} \\
\text{Pf} & : \text{For} \rightarrow \text{Set} \\
\text{irr} & : (a \ a' : \text{Pf } A) \rightarrow a = a' \\
\iota & : \text{For} \\
- \supset - & : \text{For} \rightarrow \text{For} \rightarrow \text{For} \\
- \cdot - & : \text{Pf } (A \supset B) \rightarrow \text{Pf } A \rightarrow \text{Pf } B \\
\text{K} & : \text{Pf } (A \supset B \supset A) \\
\text{S} & : \text{Pf } ((A \supset B \supset C) \supset (A \supset B) \supset A \supset C)
\end{aligned}$$

Define normal forms as a proof-irrelevant predicate on Pf_1 and show normalisation. Analysing the shape of normal forms, show that Peirce's law is not provable.

Food for thought 45. Extend this logic with true, false, conjunction, disjunction, and show via normalisation that the law of excluded middle is not derivable.

2.5 Other GATs

See [KKA19] for an algorithm which derives the notion of model, morphism, dependent model, dependent morphism from any GAT signature. Models and morphisms organise into a category (morphisms can be composed, composition is associative, etc), dependent models and dependent morphisms form a family structure over this category, together they produce a category with family, see Example 228.

Classes of inductive sets (meta-types) and classes of algebraic theories roughly correspond to each other. Inductive sets are initial models (syntaxes) for some algebraic theory, algebraic theories determine classes of algebras for some inductive types. The correspondence:

single sorted algebraic theory without equations	simple inductive type (W-type)
multi sorted algebraic theory without equations	mutually defined inductive types (indexed W-type)
single sorted algebraic theory	quotient inductive type (QIT, QW-type)
generalised algebraic theory without equations	inductive inductive type (IIT)
generalised algebraic theory (GAT)	quotient inductive inductive type (QIIT)
higher generalised algebraic theory	higher inductive inductive type (HIIT)

However algebraic theories are more fine-grained: for example, as inductive types $\text{List } A$ and the free monoid over A are equivalent (Exercise 51), but as algebraic theories, the latter has a larger category of models. The situation is similar between the lambda calculus and combinatory calculus [AKSV23]. Mutual inductive types can be reduced to indexed inductive types [KvR20] which can again be reduced to simple inductive types [Kap19], but this is not the case for the corresponding classes of algebraic theories.

Definition 46 (Zero-suc-pred). *A model of zero-suc-pred comprises the following:*

$$\begin{array}{llllll}
\mathbb{I} : \text{Set} & \text{zero} : \mathbb{I} & \text{suc} : \mathbb{I} \rightarrow \mathbb{I} & \text{pred} : \mathbb{I} \rightarrow \mathbb{I} & \text{suc } (\text{pred } i) = i & \text{pred } (\text{suc } i) = i
\end{array}$$

Exercise 47. Show that the syntax of zero-suc-pred can be defined via integers (in a proof assistant, you can define integers as the set $\mathbb{N} + \mathbb{1} + \mathbb{N}$ as a coproduct set expressing that an integer is either a negative natural, zero or a positive natural number).

Food for thought 48. Can we define a strict syntax of zero-suc-pred in Agda? A strict syntax is a syntax where both equations hold by definition.

Definition 49 (Monoid over $A : \text{Set}$). *A model is a model of a monoid (Definition 1) extended with an operation $\eta : A \rightarrow C$.*

The syntax of monoid over A is also called the free monoid over A .

Definition 50 (A -nil-cons). *For an $A : \text{Set}$, an A -nil-cons model comprises the following:*

$$\begin{array}{lll}
L : \text{Set} & \text{nil} : L & \text{cons} : A \rightarrow L \rightarrow L
\end{array}$$

The syntax of A -nil-cons is also called $\text{List } A$.

Exercise 51. Show that the syntax of A -nil-cons gives rise to a syntax of monoid over A . Show normalisation for monoids over A where normal forms are $\text{List } A$.

Exercise 52. Define the syntax of monoid over A without quotients (partly Exercise 51). Refine this syntax to be strict, which means that all three monoid equations hold by definition (hint: use Haskell’s difference list trick, or Yoneda).

Definition 53 (Graph). A model comprises the following:

$$\begin{aligned}\text{Ob} &: \text{Set} \\ \text{Mor} &: \text{Ob} \rightarrow \text{Ob} \rightarrow \text{Set}\end{aligned}$$

Exercise 54 (From [Moe22]). Show that reflexivity (an operation $(I : \text{Ob}) \rightarrow \text{Mor } I I$) for graphs is admissible.

Definition 55 (Category). A model is a graph with the following additional components:

$$\begin{aligned}- \circ - &: \text{Mor } J I \rightarrow \text{Mor } K J \rightarrow \text{Mor } K I \\ \text{id} &: \text{Mor } I I \\ \text{ass} &: (f \circ g) \circ h = f \circ (g \circ h) \\ \text{idl} &: \text{id} \circ f = f \\ \text{idr} &: f \circ \text{id} = f\end{aligned}$$

Definition 56 (Monoid’). A monoid is a category with an additional operation $\text{ob} : \text{Ob}$ and equation $(I J : \text{Ob}) \rightarrow I = J$.

Exercise 57. What is the relationship between monoid and monoid’?

Definition 58 (Preorder). A model is a category with the additional equation $(f g : \text{Mor } J I) \rightarrow f = g$.

Exercise 59. What is the simplest definition of syntax for category? What about cartesian closed category?

Remark 60 (GATs which are not ATs). Let’s fix a GAT. A relation over a model of this GAT is a dependent model over the product of the model with itself (it does not need to be an equivalence relation). The model can be quotiented by the relation, producing the quotiented model and a morphism from the original model to the quotiented model. If our GAT was an AT, then this morphism is an epimorphism (meaning roughly that it is surjective). This is not the case for GATs. For example, transitive graphs don’t have this property: the transitive graph with four vertices and two edges $a \rightarrow b$ $c \rightarrow d$ can be quotiented such that $b = c$, then the quotiented graph has three edges, so the morphism from the original transitive graph cannot be surjective. The phenomenon that as the result of quotienting certain operations become possible (here composing the two edges) does not happen in ATs.

Remark 61 (Normalisation for general GATs). Normal forms and normalisation cannot be defined for arbitrary GATs. For example, in the syntax of untyped combinator calculus (Definition 194), equality is undecidable, hence it does not have normalisation.

3 Derivability in SOGATs

Second-order GATs (SOGATs) allow second-order operations. Second-order operations are also called binders. In this section, we define several languages with binders as SOGATs, and show how to define derivable operations and prove derivable equations in second-order models. The phrase “second-order” is a negative qualifier: second-order algebraic theories are not really algebraic (this is like people’s democracy, which is not really democracy). For example, there is no good notion of morphism between second-order models. However, for derivability, second-order models are good enough. We will treat this problem in Section 4.

3.1 Simply typed lambda calculus

Definition 62 (Simply typed lambda calculus, STLC). A second-order model of STLC comprises the following components:

$$\begin{aligned}\text{Ty} &: \text{Set} \\ \text{Tm} &: \text{Ty} \rightarrow \text{Set} \\ \iota &: \text{Ty} \\ - \Rightarrow - &: \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\ \text{lam} &: (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (A \Rightarrow B) \\ - \cdot - &: \text{Tm } (A \Rightarrow B) \rightarrow \text{Tm } A \rightarrow \text{Tm } B \\ \beta &: \text{lam } b \cdot a = b a \\ \eta &: f = \text{lam } \lambda x. f \cdot x\end{aligned}$$

The operators `lam` and `- · -` take *implicit arguments*, A and B . E.g. the explicit type of `lam` is $\{A : \text{Ty}\}\{B : \text{Ty}\} \rightarrow (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (A \Rightarrow B)$. All the arguments of the equations β and η are implicit. E.g. the explicit type of β is $\{A : \text{Ty}\}\{B : \text{Ty}\}\{b : \text{Tm } A \rightarrow \text{Tm } B\}\{a : \text{Tm } A\} \rightarrow (- \cdot -) \{A\}\{B\} (\text{lam } \{A\}\{B\} b) a = b a$. Note that in the equation η , f has to be in $\text{Tm } (A \Rightarrow B)$ for some A and B for the equation to make sense.

The operation `lam` is second-order (also called a *binder*): its third input (after the two implicit inputs A, B) is a function.

Notation 63 (Isomorphism). *The last four lines can be written more consisely:*

$$\text{lam} : (\text{Tm } A \rightarrow \text{Tm } B) \cong \text{Tm } (A \Rightarrow B) : - \cdot -$$

We say that the sets $(\text{Tm } A \rightarrow \text{Tm } B)$ and $\text{Tm } (A \Rightarrow B)$ are isomorphic. We write the maps on the two sides of $\cong$, the notation in general is

$$(f : X \cong Y : g) := (f : X \rightarrow Y) \times (g : Y \rightarrow X) \times ((x : X) \rightarrow g(f x) = x) \times ((y : Y) \rightarrow f(g y) = y).$$

Derivability is programming: we can program in a SOGAT by postulating a second-order model in Agda and combining the components to build new programs. We can run the programs by proving (deriving) equalities between them.

Example 64. *In any second-order model of STLC, we define the identity function on ι by `lam` $\{\iota\}\{\iota\} (\lambda x. x)$, which is in $\text{Tm } (\iota \Rightarrow \iota)$. Note the difference between the object theory `lam` and meta λ . Actually, we can define the identity function on any type using meta quantification:*

$$\lambda A. \text{lam } \{A\}\{A\} (\lambda x. x) : (A : \text{Ty}) \rightarrow \text{Tm } (A \Rightarrow A).$$

We compute (run the programs) in a second-order model deriving equalities. E.g. given $u : \text{Tm } \iota$ we argue

$$(\text{lam } (\lambda x. x)) \cdot u \stackrel{\beta}{=} (\lambda x. x) u \stackrel{\text{meta function} = \text{application}}{=} u.$$

This seems like cheating: we avoided writing substitution by pushing it into the metatheory. But this is OK, we do not want to fully bootstrap STLC, we just want to define it. In the metatheory we of course assume (higher-order) functions, function application, its β rule, and so on.

Notation 65 (Derivation rules for SOGATs). *Similarly to GATs, the derivation rule notation uses uncurried function space, named parameters and horizontal lines instead of the arrow $\rightarrow$ symbol. For arrows in second-order positions, we use the turnstile $\vdash$, and we use named function application for the $\vdash$ function space. Second-order models of STLC are described by the following derivation rules:*

$$\begin{array}{c} \frac{}{\text{Ty} : \text{Set}} \quad \frac{A : \text{Ty}}{\text{Tm } A : \text{Set}} \quad \frac{}{\iota : \text{Ty}} \quad \frac{A : \text{Ty} \quad B : \text{Ty}}{A \Rightarrow B : \text{Ty}} \quad \frac{x : \text{Tm } A \vdash t : \text{Tm } B}{\text{lam } x. t : \text{Tm } (A \Rightarrow B)} \\[10pt] \frac{t : \text{Tm } (A \Rightarrow B) \quad a : \text{Tm } A}{t \cdot a : \text{Tm } B} \quad \frac{}{(\text{lam } x. b) \cdot a = b[x \mapsto a]} \beta \quad \frac{}{f = \text{lam } x. f \cdot x} \eta \end{array}$$

Some arguments of the horizontal line function space can be implicit. Sometimes the name of the rule is written on the right hand side of the horizontal line.

We can omit the $: \text{Ty}$ and Tm parts and more concisely write the following without sacrificing precision (this is always the case when there are two sorts $\text{Ty} : \text{Set}, \text{Tm} : \text{Ty} \rightarrow \text{Set}$).

$$\begin{array}{c} \frac{A \quad B}{A \Rightarrow B} \quad \frac{x : A \vdash t : B}{\text{lam } x. t : A \Rightarrow B} \quad \frac{t : A \Rightarrow B \quad a : A}{t \cdot a : B} \quad \frac{}{(\text{lam } x. b) \cdot a = b[x \mapsto a]} \quad \frac{}{f = \text{lam } x. f \cdot x} \end{array}$$

Example 66 (Second-order operations in mathematics). *For the integral operation, the derivation rule notation is on the left and the second-order algebraic notation is on the right:*

$$\frac{a : \mathbb{R} \quad b : \mathbb{R} \quad x : \mathbb{R} \vdash t : \mathbb{R}}{\int_a^b t \, dx} \quad f : \mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R},$$

the expression $\int_0^1 \frac{1}{x^2} \, dx$ corresponds to $\int 01 (\lambda x. \frac{1}{x^2})$.

Notation 67 (Derivation trees for SOGATs). *When we write derivation trees, the second-order arguments are collected on the left hand side of the $\vdash$. For example, given $A : \text{Ty}$ and $B : \text{Ty}$, we derive the constant function*

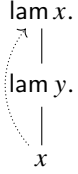
as follows: the verbose notation is on the left, the concise is on the right. In the concise notation we also omit the λ s.

$$\frac{\frac{x : \text{Tm } A, y : \text{Tm } B \vdash x : \text{Tm } A}{x : \text{Tm } A \vdash \text{lam } \lambda y. x : \text{Tm } (B \Rightarrow A)}}{\text{lam } \lambda x. \text{lam } \lambda y. x : \text{Tm } (A \Rightarrow B \Rightarrow A)}$$

$$\frac{\frac{x : A, y : B \vdash x : A}{x : A \vdash \text{lam } y. x : B \Rightarrow A}}{\text{lam } x. \text{lam } y. x : A \Rightarrow B \Rightarrow A}$$

Now the leaves of the tree can be assumptions from the left hand side of the turnstile in addition to derivation rules without arguments. The algebraic (or Agda) version of this is simply the conclusion $\text{lam } \lambda x. \text{lam } \lambda y. x$, or $\text{lam } (\lambda x. \text{lam } (\lambda y. x))$ with more brackets.

Notation 68 (Abstract binding trees (ABTs)). Another notation for SOGAT-derivation trees is abstract binding trees where variables are pointers to the binders which are reachable through the path to the root. The constant function is drawn as follows.

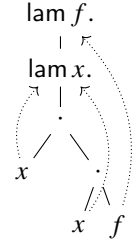


Abstract binding tree notation has the same issue as abstract syntax trees: the restriction that trees can only be put together in well-typed ways is not apparent.

Example 69. We define the “double” function using short derivation rule and abstract binding tree notation.

$$\frac{\frac{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A}{f : A \Rightarrow A, x : A \vdash f \cdot (f \cdot x) : A}}{\frac{f : A \Rightarrow A, x : A \vdash \text{lam } x. f \cdot (f \cdot x) : A \Rightarrow A}{\text{lam } f. \text{lam } x. f \cdot (f \cdot x) : (A \Rightarrow A) \Rightarrow A \Rightarrow A}}$$

$$\frac{\frac{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A}{f : A \Rightarrow A, x : A \vdash f \cdot x : A}}{\frac{f : A \Rightarrow A, x : A \vdash f \cdot x : A}{f : A \Rightarrow A, x : A \vdash f \cdot x : A}}$$



Example 70. Function composition:

$$\frac{\frac{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f : B \Rightarrow A}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f \cdot (g \cdot x) : A}}{\frac{f : B \Rightarrow A, g : C \Rightarrow B \vdash \text{lam } x. f \cdot (g \cdot x) : C \Rightarrow A}{f : B \Rightarrow A \vdash \text{lam } g. \text{lam } x. f \cdot (g \cdot x) : (C \Rightarrow B) \Rightarrow C \Rightarrow A}}{\text{lam } f. \text{lam } g. \text{lam } x. f \cdot (g \cdot x) : (B \Rightarrow A) \Rightarrow (C \Rightarrow B) \Rightarrow C \Rightarrow A}$$

$$\frac{\frac{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g : C \Rightarrow B}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g \cdot x : B}}{\frac{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g \cdot x : B}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g \cdot x : B}}$$

Example 71. We define a model of STCC (Definition 32) assuming a second-order model of STLC.

$$\begin{aligned} \text{Ty} &:= \text{Ty} \\ \text{Tm } A &:= \text{Tm } A \\ \iota &:= \iota \\ A \Rightarrow B &:= A \Rightarrow B \\ K &:= \text{lam } (\lambda u. \text{lam } (\lambda f. u)) \\ S &:= \text{lam } (\lambda f. \text{lam } (\lambda g. \text{lam } (\lambda u. f \cdot u \cdot (g \cdot u)))) \end{aligned}$$

The equations hold, e.g.:

$$\begin{aligned} K\beta : K \cdot u \cdot f &= (\text{definition of } K) \\ &= ((\text{lam } (\lambda u. \text{lam } (\lambda f. u))) \cdot u) \cdot f = (\beta) \\ &= ((\lambda u. \text{lam } (\lambda f. u)) u) \cdot f = (\text{meta function application}) \\ &= (\text{lam } (\lambda f. u)) \cdot f = (\beta) \\ &= (\lambda f. u) f = (\text{meta function application}) \\ &= u \end{aligned}$$

Exercise 72. Prove that $S\beta$ holds.

Food for thought 73. Can we derive a second-order model of STLC from a model of STCC? What if we start with a second-order model having some extra equations [AKSV23]?

Example 74. We can use derivation tree building to do informal type inference. For example, we don't know whether the expression

$$\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z$$

has a type (makes sense in a second-order model of STLC), so we try to build its derivation tree from the bottom. Let's say it has an arbitrary type A .

$$\frac{?}{\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : A}$$

As the expression starts with lam , we use its derivation rule and we learn that $A = B \Rightarrow C$ for some B and C :

$$\frac{\frac{?}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : C}}{\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow C}$$

We use lam again and we learn $C = D \Rightarrow E$, so we replace all occurrences of C with $D \Rightarrow E$:

$$\frac{\frac{\frac{?}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : E}}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow E}}{\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow E)}$$

We use lam again and we learn $E = F \Rightarrow G$:

$$\frac{\frac{\frac{?}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)} \\ \text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))$$

Now we can only use the derivation rule of application ($\cdot$), and we introduce a new metavariable H because we cannot read off the domain of the function from its conclusion. Recall that $y \cdot x \cdot z = (y \cdot x) \cdot z$.

$$\frac{\frac{\frac{?}{x : B, y : D, z : F \vdash y \cdot x : H \Rightarrow G}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G} \\ \frac{\frac{\frac{?}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash z : F}}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)} \\ \text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))$$

We first reach a leaf on the right hand side: z is to the left of the turnstile, so we learn $H = F$:

$$\frac{\frac{\frac{?}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G} \\ \frac{\frac{\frac{?}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash z : F}}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)} \\ \text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))$$

On the left hand side, there is an application $\cdot$, we introduce a new metavariable I :

$$\frac{\frac{\frac{?}{x : B, y : D, z : F \vdash y : I \Rightarrow F \Rightarrow G}}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G}}{\frac{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G}} \\ \frac{\frac{\frac{?}{x : B, y : D, z : F \vdash x : I}}{x : B, y : D, z : F \vdash x : I}}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)} \\ \text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))$$

The solution to the right hand side ? forces $I = B$, because this is the type of x :

$$\frac{\frac{\frac{?}{x : B, y : D, z : F \vdash y : B \Rightarrow F \Rightarrow G} \quad \frac{x : B, y : D, z : F \vdash x : B}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G} \quad \frac{x : B, y : D, z : F \vdash z : F}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G} \quad \frac{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}{\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

On the left hand side we learn $D = B \Rightarrow F \Rightarrow G$:

$$\frac{\frac{\frac{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y : B \Rightarrow F \Rightarrow G}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x : F \Rightarrow G} \quad \frac{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash x : B}{\dots, z : F \vdash z : F}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x \cdot z : G} \quad \frac{x : B, y : B \Rightarrow F \Rightarrow G \vdash \text{lam } \lambda z. y \cdot x \cdot z : F \Rightarrow G}{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : (B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G)} \quad \frac{x : B \vdash \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : (B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G)}{\text{lam } \lambda x. \text{lam } \lambda y. \text{lam } \lambda z. y \cdot x \cdot z : B \Rightarrow ((B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G))}$$

So the most general type of the expression is $B \Rightarrow ((B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G))$ for some types B, F, G .

Exercise 75. Using the above method, try to infer the type of the following expressions (we assume Razor extended with functions merging Definitions 14 and 62):

$\text{lam } \lambda x. x \cdot x$
 $\text{lam } \lambda f. f \cdot (\text{num } 1 + f \cdot \text{true})$
 $\text{lam } \lambda f. f \cdot (\text{num } 1 + f \cdot \text{num } 2)$

Exercise 76. Show that function composition (Example 70) is associative. That is, abbreviating $\text{comp} := \text{lam } f. \text{lam } g. \text{lam } x. f \cdot (g \cdot x)$, for every u, v, w we have $\text{comp} \cdot (\text{comp} \cdot u \cdot v) \cdot w = \text{comp} \cdot u \cdot (\text{comp} \cdot v \cdot w)$.

Exercise 77. Show that for any $A : \text{Ty}$, $\text{Tm}(A \Rightarrow A)$ is a monoid with function composition and identity.

Exercise 78. Show that we can define a category where objects are Ty , a morphism from A to B is $\text{Tm}(A \Rightarrow B)$, composition is function composition (a refined version of the previous exercise).

We cannot prove logical consistency just assuming a second-order model, because it might be the trivial model (where all sorts are 1). We also cannot prove equational consistency saying that there are terms a, a' such that $a = a' \rightarrow \mathbb{0}$, but we can get something similar: instead of obtaining a meta $\mathbb{0}$, we obtain that the second-order model is trivial:

Exercise 79. Prove that for any $B : \text{Ty}$ we have $a : \text{Ty}$ and $a, a' : \text{Tm } A$ where $a = a'$ implies that any $b, b' : \text{Tm } B$, $b = b'$.

Exercise 80. There is a second-order model of STLC where there is an $A : \text{Ty}$ and $a, a' : \text{Tm } A$ such that $a \neq a'$.

3.1.1 Bidirectional type checking

The ABT-level (well-scoped) definition of STLC is the following.

Definition 81 (Well-scoped STLC). A first-order model of well-scoped STLC with De Bruijn indices comprises

the following components:

Con	: Set
$\diamond$	: Con
$-\triangleright$	: Con $\rightarrow$ Con
Var	: Con $\rightarrow$ Set
vz	: Var ($\Gamma \triangleright$)
vs	: Var $\Gamma \rightarrow$ Var ($\Gamma \triangleright$)
Tm	: Con $\rightarrow$ Set
var	: Var $\Gamma \rightarrow$ Tm Γ
lam	: Tm ($\Gamma \triangleright$) $\rightarrow$ Tm Γ
$-\cdot-$	: Tm $\Gamma \rightarrow$ Tm $\Gamma \rightarrow$ Tm Γ
Ty	: Set
ι	: Ty
$-\Rightarrow-$	: Ty $\rightarrow$ Ty $\rightarrow$ Ty
ann	: Tm $\Gamma \rightarrow$ Ty $\rightarrow$ Tm Γ

Contexts (Con) give the information about the number of variables: $\diamond$ is the empty context, $\diamond \triangleright$ is the context containing one variable, $\diamond \triangleright \triangleright$ contains two variables, and so on. In the syntax of well-scoped STLC, Con is isomorphic to natural numbers. Var contains well-scoped De-Brujin indices. In the syntax, Var Γ is the finite set containing Γ many elements. Terms are indexed by the context: in the syntax, Tm Γ is a term that can refer to Γ many variables. In particular, Tm $\diamond$ is a closed term. Terms include annotated terms (ann) where a type for the term is specified.

Some examples for terms in the above well-scoped language (left) and the corresponding terms in a second-order model of STLC:

$\text{lam} (\text{var vz})$	$\text{lam } \lambda x. x$
$\text{lam} (\text{lam} (\text{var} (\text{vs vz})))$	$\text{lam } \lambda x. \text{lam } \lambda y. x$
$\text{lam} (\text{lam} (\text{var vz}))$	$\text{lam } \lambda x. \text{lam } \lambda y. y$
$\text{lam} (\text{lam} (\text{var} (\text{vs vz}) \cdot (\text{var} (\text{vs vz}) \cdot \text{var vz})))$	$\text{lam } \lambda f. \text{lam } \lambda x. f \cdot (f \cdot x)$

Type checking should map the left hand side untyped terms to the right hand side typed terms. However there is not enough information in these untyped terms to guess the type in the right hand side (which are implicit arguments). This is why we have annotated terms. The type inference algorithm will map the following untyped terms (left) to the typed terms (right). We added some implicit arguments.

$\text{ann} (\text{lam} (\text{var vz})) (\iota \Rightarrow \iota)$	$\text{lam } \{\iota\} \lambda x. x$
$\text{ann} (\text{lam} (\text{lam} (\text{var} (\text{vs vz})))) (\iota \Rightarrow \iota \Rightarrow \iota)$	$\text{lam } \{\iota\} \lambda x. \text{lam } \{\iota\} \lambda y. x$
$\text{ann} (\text{lam} (\text{lam} (\text{var vz}))) (\iota \Rightarrow \iota \Rightarrow \iota)$	$\text{lam } \{\iota\} \lambda x. \text{lam } \{\iota\} \lambda y. y$
$\text{ann} (\text{lam} (\text{lam} (\text{var} (\text{vs vz}) \cdot (\text{var} (\text{vs vz}) \cdot \text{var vz})))) ((\iota \Rightarrow \iota) \Rightarrow \iota \Rightarrow \iota)$	$\text{lam } \{\iota \Rightarrow \iota\} \lambda f. \text{lam } \{\iota\} \lambda x. f \cdot (f \cdot x)$

Definition 82 (Typechecking and inference model). *The following is a model of well-scoped STLC, referring to a second-order model of STLC. We rely on decidability of equality of Ty denoted by $\stackrel{?}{=}$ and by matching on*

its elements by λ , and we use the monadic operations of *Maybe* denoted $\gg$, $\gg$, guard .

$$\begin{aligned}
\text{Con} &:= (U : \text{Set}) \times (El : U \rightarrow \text{Set}) \\
\Diamond &:= (1, \lambda _ . 1) \\
(U, El) \triangleright &:= (U \times \text{Ty}, \lambda(\Gamma, A). El \Gamma \times \text{Tm } A) \\
\text{Var } (U, El) &:= (\Gamma : U) \rightarrow (A : \text{Ty}) \times (El \Gamma \rightarrow \text{Tm } A) \\
\text{vz} &:= \lambda(\Gamma, A). (A, \lambda(\gamma, a). a) \\
\text{vs } x &:= \lambda(\Gamma, B). \text{match } (x \Gamma) \{ (A, a) \mapsto (A, \lambda(\gamma, b). a \gamma) \} \\
\text{Tm } (U, El) &:= (\Gamma : U) \rightarrow ((A : \text{Ty}) \rightarrow \text{Maybe } (El \Gamma \rightarrow \text{Tm } A)) \times \text{Maybe } ((A : \text{Ty}) \times (El \Gamma \rightarrow \text{Tm } A)) \\
\text{var } x \Gamma &:= \text{match } (x \Gamma) \{ (A, a) \mapsto ((\lambda A'. \text{guard } (A \stackrel{?}{=} A') \gg \text{just } (A, a)), \text{just } (A, a)) \} \\
\text{lam } t \Gamma &:= ((\lambda(A \Rightarrow B). (t(\Gamma, A))._1 B \gg \lambda f. \text{just } \lambda \gamma. \text{lam } \lambda a. f(\gamma, a)), \text{nothing}) \\
(t \cdot u) \Gamma &:= \text{match } (t \Gamma, u \Gamma) \{ ((_, \text{just } (A \Rightarrow B, t')), (chk_u, _)) \mapsto \\
&\quad ((\lambda B'. chk_u A \gg \lambda u'. \text{guard } (B \stackrel{?}{=} B') \gg \text{just } \lambda \gamma. t' \gamma \cdot u' \gamma), chk_u A \gg \lambda u'. \text{just } \lambda \gamma. t' \gamma \cdot u' \gamma) \} \\
\text{Ty} &:= \text{Ty} \\
\iota &:= \iota \\
A \Rightarrow B &:= A \Rightarrow B \\
\text{ann } t A \Gamma &:= (\lambda A'. \text{guard } (A \stackrel{?}{=} A') \gg (t \Gamma)_1 A, (t \Gamma)_1 A \gg \lambda a. \text{just } (A, a))
\end{aligned}$$

Interpretation into this model are the following functions:

$$\begin{aligned}
\text{Tys} &: \text{Con}_1 \rightarrow \text{Set} \\
\text{Tms} &: (n : \text{Con}_1) \rightarrow \text{Tys } n \rightarrow \text{Set} \\
\text{check} &: \text{Tm}_1 n \rightarrow (\Gamma : \text{Tys } n) \rightarrow (A : \text{Ty}) \rightarrow \text{Maybe } (\text{Tms } \Gamma \rightarrow \text{Tm } A) \\
\text{infer} &: \text{Tm}_1 n \rightarrow (\Gamma : \text{Tys } n) \rightarrow \text{Maybe } ((A : \text{Ty}) \times (\text{Tms } \Gamma \rightarrow \text{Tm } A))
\end{aligned}$$

For a closed preterm $t : \text{Tm}_1 \Diamond$, the last two specialise as follows:

$$\begin{aligned}
\text{check } t &: 1 \rightarrow (A : \text{Ty}) \rightarrow \text{Maybe } (1 \rightarrow \text{Tm } A) \\
\text{infer } t &: 1 \rightarrow \text{Maybe } ((A : \text{Ty}) \times (1 \rightarrow \text{Tm } A))
\end{aligned}$$

Exercise 83. Do type checking and inference on some example preterms.

3.2 System T

The following language has a proper type of natural numbers, it does not simply inherit $\mathbb{N}$ from the metatheory, like *Razor* did.

Definition 84 (System T). A second-order model of *System T* comprises the following components:

$$\begin{aligned}
\text{Ty} &: \text{Set} \\
\text{Tm} &: \text{Ty} \rightarrow \text{Set} \\
-\Rightarrow - &: \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\
\text{lam} &: (\text{Tm } A \rightarrow \text{Tm } B) \cong \text{Tm } (A \Rightarrow B) : - \cdot - \\
\text{Nat} &: \text{Ty} \\
\text{zero} &: \text{Tm } \text{Nat} \\
\text{suc} &: \text{Tm } \text{Nat} \rightarrow \text{Tm } \text{Nat} \\
\text{ite} &: \text{Tm } A \rightarrow (\text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } \text{Nat} \rightarrow \text{Tm } A \\
\text{Nat}\beta_1 &: \text{ite } z \ s \ \text{zero} = z \\
\text{Nat}\beta_2 &: \text{ite } z \ s \ (\text{suc } t) = s \ (\text{ite } z \ s \ t)
\end{aligned}$$

Natural numbers are now given by `zero` and successor `suc`, for example, three is `suc (suc (suc zero))`. Functions out of natural numbers can be defined via `ite` (iterator, also called `recursor`, `fold`).

Definition 85 (Constructors, destructors, computation and uniqueness rules). Operators for each type can be grouped into *constructors* (introduction rules) and *destructors* (elimination rules, eliminators). Equations can be grouped into β (computation) and η (uniqueness) rules. β rules explain how to compute if a destructor is applied to a constructor, η rules express what happens if a constructor is applied to a destructor.

We group our rules for System T and Razor for comparison.

language	type	constructor	destructor	computation	uniqueness
System T (Def. 84)	$- \Rightarrow -$	lam	$- \cdot -$	$\Rightarrow\beta$	$\Rightarrow\eta$
	Nat	zero, suc	ite	$\text{Nat}\beta_1, \text{Nat}\beta_2$	
Razor (Def. 14)	Bool	true, false	ite	$\text{ite}\beta_1, \text{ite}\beta_2$	
	Nat	num	$- + -, \text{isZero}$	$+\beta, \text{isZero}\beta_1, \text{isZero}\beta_2$	

Nat does not come with an η rule, but this will be fixed in Section 3.5.

Example 86. *The double function:*

double : Tm (Nat $\Rightarrow$ Nat)
double := lam λx .ite zero (λy .suc (suc y)) x

Computing the double of 2:

double · suc (suc zero) =(abbreviation)
(lam λx .ite zero (λy .suc (suc y)) x) · suc (suc zero) =(β)
(λx .ite zero (λy .suc (suc y)) x) (suc (suc zero)) =(meta application)
ite zero (λy .suc (suc y)) (suc (suc zero)) =(Nat β_2)
(λy .suc (suc y)) (ite zero (λy .suc (suc y)) (suc zero)) =(meta application)
suc (suc (ite zero (λy .suc (suc y)) (suc zero))) =(Nat β_2)
suc (suc ((λy .suc (suc y)) (ite zero (λy .suc (suc y)) zero))) =(meta application)
suc (suc (suc (suc (ite zero (λy .suc (suc y)) zero)))) =(Nat β_1)
suc (suc (suc (suc zero)))

Definition 87 (Embedding $\mathbb{N}$ into Tm Nat). *The meta function $\ulcorner - \urcorner : \mathbb{N} \rightarrow \text{Tm Nat}$ is defined as $\ulcorner n \urcorner := \text{suc}^n \text{zero}$, e.g. $\ulcorner 3 \urcorner = \text{suc} (\text{suc} (\text{suc zero}))$.*

Definition 88 (Definability). *An $f : \mathbb{N} \rightarrow \mathbb{N}$ is definable in a second-order model of System T if there is a $t : \text{Tm} (\text{Nat} \Rightarrow \text{Nat})$ such that for all n , $\ulcorner f n \urcorner = t \cdot \ulcorner n \urcorner$. An $f : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ is definable if there is a $t : \text{Tm} (\text{Nat} \Rightarrow \text{Nat} \Rightarrow \text{Nat})$ such that for all m, n , $\ulcorner f m n \urcorner = t \cdot \ulcorner m \urcorner \cdot \ulcorner n \urcorner$.*

Exercise 89. *Addition, multiplication, exponentiation, the Ackermann function are definable in any second-order model of System T.*

Exercise 90. *Which of the following terms define identity?*

lam λx .x
lam λx .ite zero suc x
lam λx .ite x (λy .y) x
lam λx .ite zero (λy .y) x
lam λx .ite (suc zero) suc x
lam λx .ite (suc zero) (λy .y) x

Exercise 91. *Which of the following terms define the function $x \mapsto 2 * x + 1$?*

lam λx .suc x
lam λx .suc (ite x suc x)
lam λx .ite (suc x) suc x
lam λx .ite (suc zero) (λy .suc (suc y)) x
lam λx .suc (ite zero (λy .suc (suc y)) x)

Exercise 92. *Show that if $\text{zero} = \text{suc } t$, then for any $u, v : \text{Tm } A$, we have $u = v$.*

Exercise 93. *Show that the predecessor function is definable (difficult, but it will become easier once we have product types).*

It is not possible to define a term $\text{pred} : \text{Tm } (\text{Nat} \Rightarrow \text{Nat})$ such that $\text{pred} \cdot \text{suc } t = t$ for any $t : \text{Tm } \text{Nat}$. [Par89]. For this, we would need the recursor for Nat which in its method for successor receives the natural number, not only the result of the recursive call:

$$\begin{aligned} \text{rec} &: \text{Tm } A \rightarrow (\text{Tm } \text{Nat} \rightarrow \text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } \text{Nat} \rightarrow \text{Tm } A \\ \text{rec } z \text{ } s \text{ zero} &= z \\ \text{rec } z \text{ } s \text{ (suc } t) &= s \text{ } t \text{ (rec } z \text{ } s \text{ } t) \end{aligned}$$

Now we can define $\text{pred} := \text{lam } \lambda x. \text{rec zero } (\lambda n _ . n) x$ and it satisfies

$$\begin{aligned} \text{pred} \cdot \text{suc } t &= (\text{abbreviation}) \\ (\text{lam } \lambda x. \text{rec zero } (\lambda y _ . y) x) \cdot \text{suc } t &= (\Rightarrow \beta) \\ (\lambda x. \text{rec zero } (\lambda y _ . y) x) (\text{suc } t) &= (\text{meta application}) \\ \text{rec zero } (\lambda y _ . y) (\text{suc } t) &= (\text{Nat} \beta_2) \\ (\lambda y _ . y) t (\text{rec zero } (\lambda y _ . y) t) &= (\text{meta application}) \\ t. & \end{aligned}$$

Exercise 94. Show that the iterator can be derived from the recursor.

Exercise 95. Show that the factorial is definable in System T with recursor.

Exercise 96. Define a model of Razor (Definition 14) assuming a second-order model of System T with recursor.

Theorem 97. There is an $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ function, and if this function is definable in System T , then there is a $v : \text{Tm } \text{Nat}$, such that $v = \text{suc } v$.

Proof. First we fix a Gödel coding which takes an arbitrary term of type $\text{Nat} \Rightarrow \text{Nat}$ and turns it into a natural number: $\text{code} : \text{Tm } (\text{Nat} \Rightarrow \text{Nat}) \rightarrow \mathbb{N}$.

An evaluator for System T is an $\text{eval} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ function with the following property: for every u and n , we have $\ulcorner \text{eval } (\text{code } u) \text{ } n \urcorner = u \cdot \ulcorner n \urcorner$. Now eval being definable means that there is a t with $\ulcorner \text{eval } m \text{ } n \urcorner = t \cdot \ulcorner m \urcorner \cdot \ulcorner n \urcorner$ for every m and n . Now we define u as follows: $u := \text{lam } \lambda x. \text{suc } (t \cdot x \cdot x)$. And we reason:

$$\begin{aligned} t \cdot \ulcorner \text{code } u \urcorner \cdot \ulcorner \text{code } u \urcorner &= (\text{eval definable}) \\ \ulcorner \text{eval } (\text{code } u) \text{ (code } u) \urcorner &= (\text{property of eval}) \\ u \cdot \ulcorner \text{code } u \urcorner &= (\text{definition of } u) \\ (\text{lam } \lambda x. \text{suc } (t \cdot x \cdot x)) \cdot \ulcorner \text{code } u \urcorner &= (\Rightarrow \beta) \\ \text{suc } (t \cdot \ulcorner \text{code } u \urcorner \cdot \ulcorner \text{code } u \urcorner) & \end{aligned}$$

So we choose $v := t \cdot \ulcorner \text{code } u \urcorner \cdot \ulcorner \text{code } u \urcorner$ and then $v = \text{suc } v$. □

Remark 98. Unlike $\text{zero} = \text{suc zero}$, the existence of a v with $v = \text{suc } v$ in a second-order model does not imply that any two terms are equal. However, the above result implies that an evaluator for System T cannot be defined in the syntax of System T . For the notion of syntax for second-order languages (like System T), see Section 4. If we have $\text{Nat}\eta$ in a second-order model, then the existence of v with $v = \text{suc } v$ implies that any two terms are equal. However, in the syntax of System T with $\text{Nat}\eta$, equality of terms is undecidable, and thus a Gödel-coding function is not possible.

3.3 Finite types

We extend STLC with nullary and binary sums and products.

Definition 99 (Language with finite types). A second-order model of Fin is STLC (Definition 62) extended

with the following components:

$$\begin{aligned}
\perp & : \text{Ty} \\
\text{exfalse} & : \text{Tm } \perp \rightarrow \text{Tm } A \\
\perp\eta & : (t : \text{Tm } \perp \rightarrow \text{Tm } A) \rightarrow t = \text{exfalse} \\
- + - & : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\
\text{inl} & : \text{Tm } A \rightarrow \text{Tm } (A + B) \\
\text{inr} & : \text{Tm } B \rightarrow \text{Tm } (A + B) \\
\text{case} & : (\text{Tm } A \rightarrow \text{Tm } C) \rightarrow (\text{Tm } B \rightarrow \text{Tm } C) \rightarrow \text{Tm } (A + B) \rightarrow \text{Tm } C \\
+\beta_1 & : \text{case } f \ g \ (\text{inl } a) = f \ a \\
+\beta_2 & : \text{case } f \ g \ (\text{inr } b) = g \ b \\
+\eta & : (t : \text{Tm } (A + B) \rightarrow \text{Tm } C) \rightarrow t = \text{case } (\lambda a. t \ (\text{inl } a)) \ (\lambda b. t \ (\text{inr } b)) \\
\top & : \text{Ty} \\
\text{tt} & : \text{Tm } \top \\
\top\eta & : t = \text{tt} \\
- \times - & : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\
-, - & : \text{Tm } A \rightarrow \text{Tm } B \rightarrow \text{Tm } (A \times B) \\
\text{fst} & : \text{Tm } (A \times B) \rightarrow \text{Tm } A \\
\text{snd} & : \text{Tm } (A \times B) \rightarrow \text{Tm } B \\
\times\beta_1 & : \text{fst } (a, b) = a \\
\times\beta_2 & : \text{snd } (a, b) = b \\
\times\eta & : t = (\text{fst } t, \text{snd } t)
\end{aligned}$$

Exercise 100. Group the above operators into constructors and destructors. Check that the β and η rules adhere to their specification.

Exercise 101. Define **Bool** by $\top + \top$ and show that the following operations are definable and equations are provable:

$$\begin{aligned}
\text{true} & : \text{Tm Bool} \\
\text{false} & : \text{Tm Bool} \\
\text{ite} & : \text{Tm } A \rightarrow \text{Tm } A \rightarrow \text{Tm Bool} \rightarrow \text{Tm } A \\
\text{Bool}\beta_1 & : \text{ite } a \ a' \ \text{true} = a \\
\text{Bool}\beta_2 & : \text{ite } a \ a' \ \text{false} = a' \\
\text{Bool}\eta & : (t : \text{Tm Bool} \rightarrow \text{Tm } A) \rightarrow t = \text{ite } (t \ \text{true}) \ (t \ \text{false})
\end{aligned}$$

Exercise 102. Show that the terms $\text{lam } \lambda x. \text{ite } x \ \text{false} \ \text{true}$ and $\text{lam } \lambda x. (\text{ite } x \ \text{false} \ (\text{ite } x \ \text{true} \ \text{true}))$ are equal (you will need **Bool** η).

Exercise 103. Define four terms of type **Bool** $\Rightarrow$ **Bool** such that if any two are equal, then **true** = **false**!

Exercise 104. Show that every term of type **Bool** $\Rightarrow$ **Bool** is equal to one of the four terms defined in the previous exercise.

Exercise 105. Define the three element type with a case operator and use it to define modulo 3 addition.

Exercise 106. Using **Bool** and product types, define homogeneous binary sums (that is, $A + A$ for any A). Define its constructors, destructor and derive its β and η rules.

Exercise 107. Define the option (maybe) type former which adds an extra element to an existing type.

Exercise 108. For every $n : \mathbb{N}$, define the **Fin** n type which contains exactly n elements. Define its constructors, destructor.

Exercise 109. For every n , define n -ary product and sum types.

Exercise 110. Derive $h \ (\text{case } f \ g \ w) = \text{case } (\lambda a. h \ (f \ a)) \ (\lambda b. h \ (g \ b)) \ w$ (this is called naturality of case in C).

Exercise 111. Show that there is exactly one term of the following types: $A \Rightarrow \top$ and $\perp \Rightarrow A$ (for arbitrary A).

Definition 112 (Type isomorphism). A and B types are isomorphic, if there are $t : \text{Tm } A \rightarrow \text{Tm } B$ and $t' : \text{Tm } B \rightarrow \text{Tm } A$, such that $t'(ta) = a$ $t(t'b) = b$ for all a, b . We denote this by $A \cong B$.

Exercise 113. Show $(A \times \top) \cong A$ for every A .

Exercise 114. Show that for any A, B we have $A + B \cong B + A$.

The next exercise is the type theory version of the algebraic equation $(a + b)^2 = a^2 + 2ab + b^2$.

Exercise 115. Show that for arbitrary A, B types, $\text{Bool} \Rightarrow (A + B) \cong \text{Bool} \Rightarrow A + \text{Bool} \times A \times B + \text{Bool} \Rightarrow B$.

Exercise 116. Show that the $\cong$ relation is reflexive, symmetric and transitive (equivalence relation), and it is a congruence for $\times$ and $+$. (For example $A \cong A'$ implies $A \times B \cong A' \times B$.)

Definition 117 (Commutative exponential semiring). A commutative exponential semiring (commutative exponential ring without negation, commutative exponential rig) is a $C : \text{Set}$ with the following operations and equations. As $+$ and $*$ are commutative, we only list one identity and distributivity law. We don't give names to the equations.

$$\begin{aligned}
- + - &: C \rightarrow C \rightarrow C \\
0 &: C \\
- * - &: C \rightarrow C \rightarrow C \\
1 &: C \\
- ^ - &: C \rightarrow C \rightarrow C \\
&: (a + b) + c = a + (b + c) \\
&: a + b = b + a \\
&: 0 + a = a \\
&: (a * b) * c = a * (b * c) \\
&: a * b = b * a \\
&: 1 * a = a \\
&: a * (b + c) = a * b + a * c \\
&: a^{b+c} = a^b * a^c \\
&: a^0 = 1 \\
&: (a * b)^c = a^c * b^c \\
&: 1^c = 1 \\
&: a^{b*c} = (a^b)^c \\
&: a^1 = a
\end{aligned}$$

Exercise 118. (Meta) natural numbers form a commutative exponential semiring with addition, multiplication and exponentiation. They also form a commutative semiring with maximum and addition. Natural numbers extended with infinity form a commutative semiring with minimum and addition.

Exercise 119. In the language *Fin*, types form a commutative semiring where equations are isomorphisms.

Remark 120. Commutative exponential semirings are what Tarski calls high school algebra [BL93] (although they omit the operation 0). A related interesting completeness problem was settled by Wilkie [Wil01]. The dependently typed analogue is still open [Alt08].

3.4 Generic programming

In this section we follow the book of Robert Harper [Har16].

Assume we have a $\text{Bool} \Rightarrow \text{Nat}$ function, one which maps true to 1 and false to 0 (e.g. $\text{lam } (\text{ite } (\text{suc } \text{zero}) \text{ zero})$). We can apply this to a term of type Bool . But we can also apply it to a term in $\text{Bool} \times \text{Nat}$ in a way that the Nat part is not touched, i.e. (t, t') is mapped to $(\text{ite } (\text{suc } \text{zero}) \text{ zero } t, t')$. Or we can apply it to a term in $(\text{Nat} + \text{Bool}) \times \text{Bool}$ where we apply it to the two Bools and we leave the Nat .

In general: we want to extend an $A \Rightarrow B$ function to an $F : \text{Ty} \rightarrow \text{Ty}$ type operator (a type depending on a type) in such a way that we obtain an $FA \Rightarrow FB$ function. This is what generic programming allows us for certain F type operators. An $F = \lambda X. A$, where A can only contain X , nullary and binary (co)products is called a polynomial type operator. If A contains function types, then the positive type operators are those for which the function can be extended to.

The type operator corresponding to the polynomial $\lambda x. 2x^3 + x^2 + 3x + 1$ is $\lambda X. (\top + \top) \times (X \times X \times X) + X \times X + (\top + \top + \top) \times X + \top$.

Definition 121 (Map for polynomial type operators). *For a second-order model of Fin (Definition 99), a model of map for polynomial type operators has one sort Poly, and some unnamed operators:*

$$\begin{aligned}
\text{Poly} &: (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Set} \\
&: \text{Poly} (\lambda X. X) \\
&: \text{Poly} (\lambda X. \top) \\
&: \text{Poly } F \rightarrow \text{Poly } G \rightarrow \text{Poly} (\lambda X. F X \times G X) \\
&: \text{Poly} (\lambda X. \perp) \\
&: \text{Poly } F \rightarrow \text{Poly } G \rightarrow \text{Poly} (\lambda X. F X + G X)
\end{aligned}$$

There is also an operator **map** with some unnamed equations:

$$\begin{aligned}
\text{map} &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{Poly } F\} \rightarrow (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (F A) \rightarrow \text{Tm } (F B) \\
\text{map } (\lambda X. X) & \quad f a = f a \\
\text{map } (\lambda X. \top) & \quad f t = \text{tt} \\
\text{map } (\lambda X. F X \times G X) & f w = (\text{map } F f (\text{fst } w), \text{map } G f (\text{snd } w)) \\
\text{map } (\lambda X. \perp) & \quad f t = \text{exfalse } t \\
\text{map } (\lambda X. F X + G X) & f w = \text{case } w \ (\lambda u. \text{inl } (\text{map } F f u)) \ (\lambda v. \text{inr } (\text{map } G f v))
\end{aligned}$$

Exercise 122. Show that the following type operator is polynomial.

$$\lambda X. \perp + (\top \times X + X \times X)$$

Exercise 123. Derive $\text{map } (\lambda X. \top + (\text{Bool} \times X)) f (\text{inr } (\text{true}, t)) = \text{inr } (\text{true}, f t)$.

Exercise 124. Assuming Poly is the initial Poly model (that is, a meta inductive type), define **map** as iteration into a Poly model.

Exercise 125. Assuming **map** is defined as in Exercise 131, show that if $A : \text{Ty}$, then $\text{map } (\lambda X. A) f w = w$.

Exercise 126. Assuming **map** is defined as in Exercise 131, show that it is a functor: $\text{map } F (\lambda x. f (g x)) w = \text{map } F f (\text{map } F g w)$ and $\text{map } F (\lambda x. x) u = u$.

If we also allow functions in the type operator, then sometimes we cannot extend our type operator. For example there is no **map** function for the type operators

- $F X = X \Rightarrow \perp$,
- $F X = X \Rightarrow X$,

however there is a **map** function for

- $F X = \text{Bool} \Rightarrow X$,
- $F X = (X + X) \Rightarrow \text{Bool} \Rightarrow X$.

In the following language we introduce the notions of positive and negative type operators. We have a (covariant) **map** for the positive type operators while we have a contravariant **map** for the negative ones.

Definition 127 (Map for positive type operators). *For a second-order model of Fin (Definition 99), a model of map for positive type operators has two sorts Pos and Neg and some unnamed operators:*

$$\begin{aligned}
\text{Pos} &: (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Set} \\
\text{Neg} &: (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Set} \\
&: \text{Pos } (\lambda X. X) \\
&: (C : \text{Ty}) \rightarrow \text{Pos } (\lambda _ . C) \\
&: (C : \text{Ty}) \rightarrow \text{Neg } (\lambda _ . C) \\
&: \text{Pos } F \rightarrow \text{Pos } G \rightarrow \text{Pos } (\lambda X. F X + G X) \\
&: \text{Neg } F \rightarrow \text{Neg } G \rightarrow \text{Neg } (\lambda X. F X + G X) \\
&: \text{Pos } F \rightarrow \text{Pos } G \rightarrow \text{Pos } (\lambda X. F X \times G X) \\
&: \text{Neg } F \rightarrow \text{Neg } G \rightarrow \text{Neg } (\lambda X. F X \times G X) \\
&: \text{Neg } F \rightarrow \text{Pos } G \rightarrow \text{Pos } (\lambda X. F X \Rightarrow G X) \\
&: \text{Pos } F \rightarrow \text{Neg } G \rightarrow \text{Neg } (\lambda X. F X \Rightarrow G X)
\end{aligned}$$

There are also two operators satisfying the equations below:

$$\begin{aligned}
\text{map}^P &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{Pos } F\} \rightarrow (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (F A) \rightarrow \text{Tm } (F B) \\
\text{map}^N &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{Neg } F\} \rightarrow (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (F B) \rightarrow \text{Tm } (F A) \\
\text{map}^P (\lambda X. X) & \quad f a = f a \\
\text{map}^P (\lambda X. C) & \quad f c = c \\
\text{map}^N (\lambda X. C) & \quad f c = c \\
\text{map}^P (\lambda X. F X + G X) & \quad f w = \text{case } w \text{ (}\lambda u. \text{inl (map}^P F f u)\text{) (}\lambda v. \text{inr (map}^P G f v)\text{)} \\
\text{map}^N (\lambda X. F X + G X) & \quad f w = \text{case } w \text{ (}\lambda u. \text{inl (map}^N F f u)\text{) (}\lambda v. \text{inr (map}^N G f v)\text{)} \\
\text{map}^P (\lambda X. F X \times G X) & \quad f w = (\text{map}^P F f (\text{fst } w), \text{map}^P G f (\text{snd } w)) \\
\text{map}^N (\lambda X. F X \times G X) & \quad f w = (\text{map}^N F f (\text{fst } w), \text{map}^N G f (\text{snd } w)) \\
\text{map}^P (\lambda X. F X \Rightarrow G X) & \quad f g = \text{lam } \lambda x. \text{map}^P G f (g \cdot (\text{map}^N F f x)) \\
\text{map}^N (\lambda X. F X \Rightarrow G X) & \quad f g = \text{lam } \lambda x. \text{map}^N G f (g \cdot (\text{map}^P F f x))
\end{aligned}$$

We illustrate the map^P -equation for $\Rightarrow$ by the following diagram.

$$\begin{array}{ccccc}
A & & F A & \xrightarrow{g} & G A \\
\downarrow f & & \uparrow \text{map}^N F f & & \downarrow \text{map}^P F f \\
B & & F B & \xrightarrow{\text{map}^P (\lambda X. F X \Rightarrow G X) f g} & G B
\end{array}$$

Exercise 128. Show that the following type operators are positive, negative or both.

- $FX = X \Rightarrow \text{Bool} \Rightarrow \text{Bool}$
- $FX = (X \Rightarrow \text{Bool}) \Rightarrow X$
- $FX = ((X \Rightarrow \text{Bool}) \Rightarrow \text{Bool}) \Rightarrow \text{Bool}$
- $FX = (\text{Bool} \Rightarrow \text{Bool}) \Rightarrow X + X$
- $FX = (X \times (X + \top)) \Rightarrow \text{Bool}$
- $FX = (\text{Bool} \Rightarrow \text{Bool}) \Rightarrow \top$

Exercise 129. Show that if the type operator $FX = X \Rightarrow \perp$ comes with a map^P function, then $\text{Tm } \perp$ has an element.

Food for thought 130. The following type operators are not positive, nor negative. What sort of contradiction do we get if they come equipped with map^P or map^N functions?

- $FX = X \Rightarrow X$
- $FX = (\text{Bool} \Rightarrow X) \Rightarrow X$

Exercise 131. Assuming Pos and Neg are the initial Pos–Neg model (that is, two mutually defined meta inductive types), define map^P and map^N by iteration into a Pos–Neg model.

Exercise 132. Prove that map^P is a covariant functor, while map^N is contravariant:

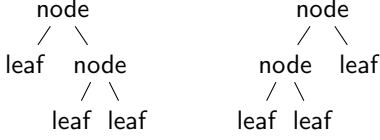
$$\begin{aligned}
\text{map}^P F (\lambda x. f (g x)) w &= \text{map}^P F f (\text{map}^P F g w) & \text{map}^P F (\lambda x. x) u &= u \\
\text{map}^N F (\lambda x. f (g x)) w &= \text{map}^N F g (\text{map}^N F f w) & \text{map}^N F (\lambda x. x) u &= u
\end{aligned}$$

3.5 Inductive types

Bool in Razor (Definition 14) and Nat in System T (Definition 84) were *inductive types*. Another example is the type of binary trees Tree which has two constructors:

$$\begin{aligned}
\text{Tree} &: \text{Ty} \\
\text{leaf} &: \text{Tm Tree} \\
\text{node} &: \text{Tm Tree} \rightarrow \text{Tm Tree} \rightarrow \text{Tm Tree}
\end{aligned}$$

The trees in **Tree** have binary branching and there is no information at the branching nodes or at the leaves. For example, the trees **node leaf** (**node leaf leaf**) and **node (node leaf leaf) leaf** are depicted as follows.



A **Tree** algebra (also called a **Tree** model inside our object language) is given by the following components:

$$A : \text{Ty} \quad u : \text{Tm } A \quad v : \text{Tm } A \rightarrow \text{Tm } A \rightarrow \text{Tm } A,$$

A homomorphism from the algebra (**Tree**, **leaf**, **node**) to the algebra (A, u, v) is given by a function which respects the operations as follows:

$$\begin{aligned} t &: \text{Tm Tree} \rightarrow \text{Tm } A \\ t \text{ leaf} &= u \\ t (\text{node } l \ r) &= v (t \ l) (t \ r) \end{aligned}$$

The iterator of **Tree** says that for any algebra (A, u, v) , there is such a homomorphism. The two equations that the homomorphism satisfies are called computation (β) rules. The uniqueness (η) rule says that every other homomorphism from (**Tree**, **leaf**, **node**) to (A, u, v) is equal to the iterator.

The two constructors of **Tree** can be merged into a single constructor along the following isomorphisms:

$$\begin{aligned} \text{Tm Tree} \times (\text{Tm Tree} \rightarrow \text{Tm Tree} \rightarrow \text{Tm Tree}) &\cong \\ (\text{Tm } \tau \rightarrow \text{Tm Tree}) \times (\text{Tm Tree} \times \text{Tm Tree} \rightarrow \text{Tm Tree}) &\cong \\ \text{Tm } (\tau \Rightarrow \text{Tree}) \times \text{Tm } ((\text{Tree} \times \text{Tree}) \Rightarrow \text{Tree}) &\cong \\ \text{Tm } ((\tau \Rightarrow \text{Tree}) \times ((\text{Tree} \times \text{Tree}) \Rightarrow \text{Tree})) &\cong \\ \text{Tm } ((\tau + \text{Tree} \times \text{Tree}) \Rightarrow \text{Tree}) &\cong \\ \text{Tm } (\tau + \text{Tree} \times \text{Tree}) \rightarrow \text{Tm Tree} &= \\ \text{Tm } ((\lambda X. \tau + X \times X) \text{ Tree}) \rightarrow \text{Tm Tree} & \end{aligned}$$

Definition 133 (Algebra of a type operator). *Assume a type operator $F : \text{Ty} \rightarrow \text{Ty}$. An F -algebra consists of a type $A : \text{Ty}$ and a function $\text{Tm } (F A) \rightarrow \text{Tm } A$.*

An inductive type consists of an F -algebra and an iterator. The type in the F -algebra is what we call the inductive type itself. The constructors of the inductive type are given by the function $\text{Tm } (F A) \rightarrow \text{Tm } A$ function in the F -algebra. The iterator encodes that there is a unique F -homomorphism from the inductive type into any other F -algebra. This is why the inductive type is also called the initial F -algebra. (It could be also called the syntax for F , but note that this syntax is internal to a model of our object theory.)

In order to obtain a well-defined notion of F -homomorphism, we need to restrict the type operator F to be positive. We will be even more restrictive, we will only allow strictly positive type operators. There are more strictly positive type operators than polynomial type operators, but less than positive type operators.

Food for thought 134. *Assuming that positive type operators have initial algebras is inconsistent with excluded middle.*

The **Tree** type is determined by the $\lambda X. \tau + X \times X$ type operator, its constructor is

$$\text{con} : \text{Tm } (\tau + \text{Tree} \times \text{Tree}) \rightarrow \text{Tm Tree},$$

its iterator and its computation rule are the following:

$$\begin{aligned} \text{ite} &: (\text{Tm } (\tau + A \times A) \rightarrow \text{Tm } A) \rightarrow \text{Tm Tree} \rightarrow \text{Tm } A \\ \text{ite } f (\text{con } w) &= \text{case } w (\lambda _. f (\text{inl } \text{tt})) \left(\lambda u. f \left(\text{inr } (\text{ite } f (\text{fst } u), \text{ite } f (\text{snd } u)) \right) \right) \end{aligned}$$

The following language contains an inductive type for all F strictly positive type operators. We express the equation for the homomorphism using **map**.

Definition 135 (Language with inductive types). *A second-order model extends Fin (Definition 99) with the following components:*

$$\begin{aligned}
\text{SPos} &: (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Set} \\
&: \text{SPos}(\lambda X.X) \\
&: (A : \text{Ty}) \rightarrow \text{SPos}(\lambda _.A) \\
&: \text{SPos } F \rightarrow \text{SPos } G \rightarrow \text{SPos}(\lambda X.F X + G X) \\
&: \text{SPos } F \rightarrow \text{SPos } G \rightarrow \text{SPos}(\lambda X.F X \times G X) \\
&: (C : \text{Ty}) \rightarrow \text{SPos } F \rightarrow \text{SPos}(\lambda X.C \Rightarrow F X)
\end{aligned}$$

We also have a map operator:

$$\begin{aligned}
\text{map} &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow (f : \text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (F A) \rightarrow \text{Tm } (F B) \\
\text{map } (\lambda X.X) & \quad f a = f a \\
\text{map } (\lambda _.C) & \quad f c = c \\
\text{map } (\lambda X.F X + G X) f w &= \text{case } w \text{ (}\lambda u.\text{inl } (\text{map } F f u) \text{) } (\lambda v.\text{inr } (\text{map } G f v) \text{)} \\
\text{map } (\lambda X.F X \times G X) f w &= (\text{map } F f (\text{fst } w), \text{map } G f (\text{snd } w)) \\
\text{map } (\lambda X.C \Rightarrow F X) f g &= \text{lam } \lambda a.\text{map } F f (g \cdot a)
\end{aligned}$$

And inductive types with their constructor, iterator, computation and uniqueness rules:

$$\begin{aligned}
\text{Ind} &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow \text{Ty} \\
\text{con} &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow \text{Tm } (F (\text{Ind } F)) \rightarrow \text{Tm } (\text{Ind } F) \\
\text{ite} &: (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow (\text{Tm } (F A) \rightarrow \text{Tm } A) \rightarrow \text{Tm } (\text{Ind } F) \rightarrow \text{Tm } A \\
\text{Ind}\beta &: \text{ite } F f (\text{con } F x) = f (\text{map } F (\text{ite } F f) x) \\
\text{Ind}\eta &: (z : \text{Tm } (\text{Ind } F) \rightarrow \text{Tm } A) \rightarrow \left((\lambda x.z (\text{con } F x)) = (\lambda x.f(\text{map } F z x)) \right) \rightarrow z = \text{ite } F f
\end{aligned}$$

Remark 136. Usually η rules are omitted because (i) they are not needed for running programs; (ii) equality checking in the syntax becomes undecidable in the presence of η for natural numbers. This is not the case for η for finite types [Sch17].

Inductive types are trees with finite depth. This means that they might have infinitely many branches, but if we follow a branch, then in finite steps we reach a leaf. Note that this does not mean that we can put an element of an inductive type into a hole of finite depth: there are infinitely branching trees where every branch is finite but it has branches of arbitrary length.

Exercise 137. The inductive type $\text{Ind } (\lambda X.\top + (\text{Nat} \Rightarrow X))$ contains infinitely branching trees. Define a tree in which the i^{th} branch has height i .

Exercise 138. Natural numbers are given by $\text{Nat} := \text{Ind } (\lambda X.\top + X)$. Define `zero` and `suc` using `con`, define the iterator and prove the computation rules.

Exercise 139. Define the following inductive types: lists of natural numbers, binary trees with natural numbers at the leaves, binary trees with natural numbers at the nodes, ternary trees, untyped combinator calculus (without equations), Razor (without equations or types).

Exercise 140. $\text{Ind } F$ is the fixpoint of the F function: $F (\text{Ind } F) \cong \text{Ind } F$.

Exercise 141. Show that for every A , we have $\text{Ind } (\lambda X.A) \cong A$.

The recursor of `Nat` can be defined via the iterator: we use the iterator in a way that it not only returns A , but also the concrete natural number on which it was called. For reference, we also list the rules for the iterator:

$$\begin{aligned}
\text{ite} &: \{A : \text{Ty}\} \rightarrow \text{Tm } A \rightarrow (\text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } \text{Nat} \rightarrow \text{Tm } A \\
\text{ite}\beta_1 &: \text{ite } z s \text{ zero} = z \\
\text{ite}\beta_2 &: \text{ite } z s (\text{suc } n) = s (\text{ite } z s n) \\
\text{ite}\eta &: (z : \text{Tm } \text{Nat} \rightarrow \text{Tm } A) \rightarrow (z \text{ zero} = z) \rightarrow ((n : \text{Tm } \text{Nat}) \rightarrow z (\text{suc } n) = s (z n)) \rightarrow z = \text{ite } z s \\
\text{rec} &: \{A : \text{Ty}\} \rightarrow \text{Tm } A \rightarrow (\text{Tm } \text{Nat} \rightarrow \text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } \text{Nat} \rightarrow \text{Tm } A \\
\text{rec } \{A\} z s n &:= \text{snd} \left(\text{ite} \{ \text{Nat} \times A \} (\text{zero}, z) (\lambda w.(\text{suc } (\text{fst } w), s (\text{fst } w) (\text{snd } w))) n \right)
\end{aligned}$$

Exercise 142. Prove the β rules of the recursor:

$$\begin{aligned}\text{rec}\beta_1 &: \text{rec } z \text{ } s \text{ zero} = z \\ \text{rec}\beta_2 &: \text{rec } z \text{ } s \text{ (suc } n) = s \text{ } n \text{ (rec } z \text{ } s \text{ } n)\end{aligned}$$

For the latter we need the η law of the iterator (without this, $\text{rec}\beta_2$ is not derivable [Par89]).

Food for thought 143. Define the η rule of the recursor. Can it be derived from that of the iterator?

Food for thought 144. Define the recursor for an arbitrary inductive type and show that it can be derived from the iterator.

If we only have inductive types for polynomial type operators, we obtain finitely branching trees.

Food for thought 145. For every F polynomial type operator, $\text{Ind } F$ is either isomorphic to a finite type or isomorphic to Nat .

Exercise 146 (Cantor's diagonal argument). Show that if $(\text{Nat} \Rightarrow \text{Bool}) \cong \text{Nat}$, then $\text{true} = \text{false}$.

3.6 Coinductive types

Coinductive types are dual to inductive types.

- Inductive types are determined by their constructors, coinductive types by their destructors.
- The functions which we define out of inductive types using the iterator always *terminate*, the generators creating elements of coinductive types are always *productive*.
- Elements of inductive types are trees with finite depth, while coinductive types generate trees of potentially infinite depth.

Programs running forever (e.g. servers, operating systems, interactive programs), automata, Turing-machines and other machines are elements of coinductive types. They also model parts of object oriented programming: a class is a coinductive type, the fields/member functions are the destructors, an object is an element of the coinductive type. In the 1990s Turner suggested strong functional programming [?] as a style where we separate inductive and coinductive types and there is no general recursion. No one has ever tried programming in such a setting.

The most well-known example of a coinductive type is the type of streams (lists of infinite length).

$$\begin{aligned}\text{Stream} &: \text{Ty} \rightarrow \text{Ty} \\ \text{head} &: \text{Tm}(\text{Stream } A) \rightarrow \text{Tm } A \\ \text{tail} &: \text{Tm}(\text{Stream } A) \rightarrow \text{Tm}(\text{Stream } A) \\ \text{gen} &: (\text{Tm } C \rightarrow \text{Tm } A) \rightarrow (\text{Tm } C \rightarrow \text{Tm } C) \rightarrow \text{Tm } C \rightarrow \text{Tm}(\text{Stream } A) \\ \text{Stream}\beta_1 &: \text{head}(\text{gen } h \text{ } t \text{ } c) = h \text{ } c \\ \text{Stream}\beta_2 &: \text{tail}(\text{gen } h \text{ } t \text{ } c) = \text{gen } h \text{ } t \text{ } (t \text{ } c)\end{aligned}$$

Streams have two destructors: head and tail. The tail can be applied any number of times to the stream, so indeed this gives us infinite number of $\text{Tm } A$ s from a $u : \text{Tm}(\text{Stream } A)$:

$$\text{head } u, \text{ head }(\text{tail } u), \text{ head }(\text{tail }(\text{tail } u)), \text{ head }(\text{tail }(\text{tail }(\text{tail } u))), \dots$$

Streams can be made using the generator (coiterator, constructor, corecursor): the input of the generator is a type of seeds (C), functions h and t and a seed (an element of $\text{Tm } C$) from which the stream grows out. The h tells us how to compute the current head element from the seed, while t says how to advance the seed (make a step): this is what the β rules express.

The stream containing the natural numbers $0, 1, 2, 3, \dots$ is given by the following term where $C := \text{Nat}$:

$$\begin{aligned}\text{nats} &: \text{Tm}(\text{Stream } \text{Nat}) \\ \text{nats} &:= \text{gen } (\lambda n. n) \text{ suc zero}\end{aligned}$$

Indeed,

$$\begin{aligned}
& \text{head nats} &= \\
& \text{head (gen (\lambda n.n) suc zero)} &= (\text{Stream} \beta_1) \\
& (\lambda n.n) \text{ zero} &= \\
& \text{zero}, \\
& \text{head (tail nats)} &= \\
& \text{head (tail (gen (\lambda n.n) suc zero))} &= (\text{Stream} \beta_2) \\
& \text{head (gen (\lambda n.n) suc (suc zero))} &= (\text{Stream} \beta_1) \\
& (\lambda n.n) (\text{suc zero}) &= \\
& \text{suc zero}, \\
& \text{head (tail (tail nats))} &= \\
& \text{head (tail (tail (gen (\lambda n.n) suc zero)))} &= (\text{Stream} \beta_2) \\
& \text{head (tail (gen (\lambda n.n) suc (suc zero)))} &= (\text{Stream} \beta_2) \\
& \text{head (gen (\lambda n.n) suc (suc (suc zero)))} &= (\text{Stream} \beta_1) \\
& (\lambda n.n) (\text{suc (suc zero)}) &= \\
& \text{suc (suc zero)},
\end{aligned}$$

and so on.

Exercise 147. Define pointwise addition on two `Stream Nats`, and define the `map` function on streams.

We cannot define the filter function on streams because it is not necessarily productive.

Every strictly positive type operator determines a coinductive type. In the following, we extend the language of inductive type with coinductive types (actually we don't need inductive types, only `SPos` and `map`).

Definition 148 (Language with inductive and coinductive types). A second-order model extends Definition 135 with the following components.

$$\begin{aligned}
& \text{Coind} : \text{SPos } F \rightarrow \text{Ty} \\
& \text{des} : (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow \text{Tm } (\text{Coind } F) \rightarrow \text{Tm } (F (\text{Coind } F)) \\
& \text{gen} : (F : \text{Ty} \rightarrow \text{Ty}) \rightarrow \{\text{SPos } F\} \rightarrow (\text{Tm } A \rightarrow \text{Tm } (F A)) \rightarrow \text{Tm } A \rightarrow \text{Tm } (\text{Coind } F) \\
& \text{Coind} \beta : \text{des } F (\text{gen } F f a) = \text{map } F (\text{gen } F f) (f a) \\
& \text{Coind} \eta : (z : \text{Tm } A \rightarrow \text{Tm } (\text{Coind } F)) \rightarrow \left((\lambda z. \text{des } F (z a)) = (\lambda z. \text{map } F z (f a)) \right) \rightarrow z = \text{gen } F f
\end{aligned}$$

Exercise 149. Show that the destructors and the generator of streams can be given using $\text{Stream } A := \text{Coind } (\lambda X. A \times X)$.

Exercise 150. Define addition and multiplication on conatural numbers $(\text{Coind } (\lambda X. \text{Idotp } \top + X))!$.

Exercise 151. Which inductive type is the dual of streams?

Exercise 152. What is the dual of lists, that is, $\text{Coind } (\lambda X. \top + A \times X)$?

Exercise 153. Define the coinductive type of machines which can receive a natural number, they can print out a number and there is a button on them. Using the generator, define an summation machine, which adds all the input numbers, returns the sum when asked, and the button resets the number to 0. Define another machine in the same type which prints the maximum of all the input numbers.

Exercise 154. A $\text{Coind } F$ típus az F operátor fixpontja: mutassuk meg, hogy $F (\text{Coind } F) \cong \text{Coind } F$.

Exercise 155. Show $\text{Coind } (\lambda X. A) \cong A$ for every A .

Food for thought 156. The dual of iterator is generator. What is the dual of the recursor?

3.7 Polymorphism

In the above languages we had to separately define function composition $(B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$ for each types A , B and C ; we had to separately define the filter $: \text{Tm } ((A \Rightarrow \text{Bool}) \Rightarrow \text{List } A \Rightarrow \text{List } A)$ function for each type A . We introduce polymorphic types in order to define these terms once and for all for every type. The type of general function composition becomes $\forall \lambda A. \forall \lambda B. \forall \lambda C. (B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$, the type of filter becomes $\forall \lambda A. (A \Rightarrow \text{Bool}) \Rightarrow \text{List } A \Rightarrow \text{List } A$.

3.7.1 Hindley–Milner

In the following type system we distinguish monomorphic (MTy) and polymorphic types (Ty), and the universal quantifier can only appear at the front (at the top of) a polymorphic type. Base types such as Nat are monomorphic.

Definition 157 (A Hindley–Milner style language). *A second-order model contains the following components.*

$$\begin{aligned}
\text{MTy} & : \text{Set} \\
\text{Ty} & : \text{Set} \\
\text{Tm} & : \text{Ty} \rightarrow \text{Set} \\
- \Rightarrow - & : \text{MTy} \rightarrow \text{MTy} \rightarrow \text{MTy} \\
\forall & : (\text{MTy} \rightarrow \text{Ty}) \rightarrow \text{Ty} \\
i & : \text{MTy} \rightarrow \text{Ty} \\
\text{lam} & : (\text{Tm } (i A) \rightarrow \text{Tm } (i B)) \cong \text{Tm } (i (A \Rightarrow B)) : - \cdot - \\
\text{Lam} & : ((A : \text{MTy}) \rightarrow \text{Tm } (B A)) \cong \text{Tm } (\forall B) : - \bullet -
\end{aligned}$$

In the above language we have not only term variables, but type variables as well: the operators $\forall$ and Lam bind type variables.

The polymorphic identity function is defined as follows.

$$\begin{aligned}
\text{id} & : \text{Tm } (\forall \lambda A. i (A \Rightarrow A)) \\
\text{id} & := \text{Lam } \lambda A. \text{lam } \lambda a. a
\end{aligned}$$

Although this is not immediately apparent, the above function uses the A input as well because the implicit parameters of lam refer to it. The explicit definition is the following.

$$\text{id} := \text{Lam } \{ \lambda A. i (A \Rightarrow A) \} (\lambda A. \text{lam } \{ A \} \{ A \} (\lambda a. a))$$

Assuming we extend the above language with $\text{Bool} : \text{Ty}$ and $\text{true} : \text{Tm Bool}$ operators, we can run the id program with those inputs as follows.

$$\begin{aligned}
& \text{id} \bullet \text{Bool} \cdot \text{true} & = \\
& (\text{Lam } \lambda A. \text{lam } \{ A \} \{ A \} \lambda a. a) \bullet \text{Bool} \cdot \text{true} & = (\forall \beta) \\
& ((\lambda A. \text{lam } \{ A \} \{ A \} \lambda a. a) \text{Bool}) \cdot \text{true} & = \\
& (\text{lam } \{ \text{Bool} \} \{ \text{Bool} \} \lambda a. a) \cdot \text{true} & = (\Rightarrow \beta) \\
& (\lambda a. a) \text{true} & = \\
& \text{true}
\end{aligned}$$

Exercise 158. *Define the following programs (derive them in any second-order model).*

$$\begin{aligned}
\text{const} & : \text{Tm } (\forall \lambda A. \forall \lambda B. i (A \Rightarrow B \Rightarrow A)) \\
\text{comp} & : \text{Tm } (\forall \lambda A. \forall \lambda B. \forall \lambda C. i ((B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C)) \\
\text{flip} & : \text{Tm } (\forall \lambda A. \forall \lambda B. \forall \lambda C. i ((A \Rightarrow B \Rightarrow C) \Rightarrow B \Rightarrow A \Rightarrow C))
\end{aligned}$$

This language does not support (co)inductive types, we have to put them in by hand. Haskell was originally designed as Hindley–Milner extended with a fixpoint operator and recursive types (see next section).

A Hindley–Milner type in general looks like this:

$$\forall \lambda A_1. \forall \lambda A_2. \dots \forall \lambda A_n. i (\underbrace{\dots}_{\text{there is no } \forall \text{ in here}})$$

The type $\text{Bool} \Rightarrow \forall \lambda A. A$ does not make sense in this language, the $\forall$ has to be in the front. The Hindley–Milner type system is very convenient for programming because types never have to be written out.

Food for thought 159. *Define a type inference algorithm for Hindley–Milner which takes preterms without typing information and produces well-typed terms which have the most general type (E.g. for const we obtain $\forall \lambda A. \forall \lambda B. A \Rightarrow B \Rightarrow A$ instead of $\forall \lambda A. A \Rightarrow A \Rightarrow A$). Preterms do not contain Lam or $- \bullet -$, these are inserted by the type inference algorithm. See [Hin69, Mil78, Csö12].*

3.7.2 System F

In the next language, $\forall$ can appear anywhere, not only at the front of a type. It was discovered independently by Jean-Yves Girard [Gir72] and John C. Reynolds [Rey74], the former called it System F, the latter called it polymorphic lambda calculus.

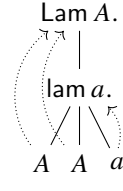
Definition 160 (System F). *A second-order model contains the following components:*

$$\begin{array}{ll} \text{Ty} & : \text{Set} \\ - \Rightarrow - & : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\ \forall & : (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ \text{Tm} & : \text{Ty} \rightarrow \text{Set} \\ \text{lam} & : (\text{Tm } A \rightarrow \text{Tm } B) \cong \text{Tm } (A \Rightarrow B) : - \cdot - \\ \text{Lam} & : ((X : \text{Ty}) \rightarrow \text{Tm } (A X)) \cong \text{Tm } (\forall A) : - \bullet - \end{array}$$

For the benefit of the reader, we also list the operations in concise derivation rule style (we don't write Ty and Tm anywhere):

$$\begin{array}{c} \frac{A \quad B}{A \Rightarrow B} \quad \frac{x : A \vdash b : B}{\text{lam } x.b : A \Rightarrow B} \quad \frac{f : A \Rightarrow B \quad a : A}{f \cdot a : B} \quad \frac{}{(\text{lam } x.b) \cdot a = b[x \mapsto a]} \quad \frac{f : A \Rightarrow B}{f = \text{lam } x.f \cdot x} \\ \frac{X \vdash A}{\forall X.A} \quad \frac{X \vdash a : A}{\text{Lam } X.a : \forall X.A} \quad \frac{f : \forall X.A \quad C}{f \bullet C : A[X \mapsto C]} \quad \frac{}{(\text{Lam } X.a) \bullet C = a[X \mapsto C]} \quad \frac{f : \forall X.A}{f = \text{Lam } X.f \bullet X} \end{array}$$

Derivation of the polymorphic identity function (we made the implicit arguments of lam explicit):

$$\frac{\frac{A, a : A \vdash a : A}{A \vdash \text{lam } \{A\} \{A\} (a.a) : A \Rightarrow A}}{\text{Lam } A. \text{lam } \{A\} \{A\} (a.a) : \forall A. A \Rightarrow A}$$


Exercise 161. *Define the following programs (derive them in any second-order model).*

$$\begin{array}{ll} \text{const} & : \text{Tm } (\forall \lambda A. \forall \lambda B. A \Rightarrow B \Rightarrow A) \\ \text{comp} & : \text{Tm } (\forall \lambda A. \forall \lambda B. \forall \lambda C. (B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \text{comp}' & : \text{Tm } (\forall \lambda B. \forall \lambda C. (B \Rightarrow C) \Rightarrow \forall \lambda A. (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ S & : \text{Tm } (\forall \lambda A. \forall \lambda B. \forall \lambda C. (A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \text{flip} & : \text{Tm } (\forall \lambda A. \forall \lambda B. \forall \lambda C. (A \Rightarrow B \Rightarrow C) \Rightarrow B \Rightarrow A \Rightarrow C) \end{array}$$

Exercise 162. *Show that any second-order model of System F gives rise to a second-order model of Hindley-Milner.*

Remark 163. *To rule out trivial solutions to the previous exercise, redo it inside the theory of signatures (see Section 3.12): define a morphism of from the signature of System F to the signature of Hindley-Milner; this also gives rise to a functor between the categories of models.*

All inductive and coinductive types described in Section 3.5, 3.6 are already supported by System F (with their β rules but without η rules). This method is called *Church encoding*, it is a generalisation of how Alonzo Church encoded natural numbers in the untyped lambda calculus (see the next section). The two-element type is defined as follows.

$$\begin{array}{ll} \text{Bool} & := \forall \lambda A. A \Rightarrow A \Rightarrow A \\ \text{true} & := \text{Lam } \lambda A. \text{lam } \lambda t. \text{lam } \lambda f. t \\ \text{false} & := \text{Lam } \lambda A. \text{lam } \lambda t. \text{lam } \lambda f. f \\ \text{ite} & : \text{Tm } \text{Bool} \rightarrow \text{Tm } (\forall \lambda A. A \Rightarrow A \Rightarrow A) \\ \text{ite} & := \lambda b. b \\ \text{Bool}\beta_1 & : \text{ite true} \bullet A \cdot t \cdot f = (\lambda b. b) \text{ true} \bullet A \cdot t \cdot f = \text{true} \bullet A \cdot t \cdot f = (\text{Lam } \lambda A. \text{lam } \lambda t. \text{lam } \lambda f. t) \bullet A \cdot t \cdot f = t \\ \text{Bool}\beta_2 & : \text{ite false} \bullet A \cdot t \cdot f = (\lambda b. b) \text{ false} \bullet A \cdot t \cdot f = \text{false} \bullet A \cdot t \cdot f = (\text{Lam } \lambda A. \text{lam } \lambda t. \text{lam } \lambda f. f) \bullet A \cdot t \cdot f = f \end{array}$$

The intuition is that a term in the above Bool type can only do two things: return its first or the second argument. In particular, it cannot look into the type A, it has to work in a uniform (parametric [Rey83, Wad89], natural [Awo10]) way for all As.

Note that Bool cannot be encoded in Hindley-Milner.

Exercise 164. We defined the type of iterator above so that it can be easily implemented. Show that its type is isomorphic to the type of the usual if-then-else operator:

$$(\text{Tm Bool} \rightarrow \text{Tm } (\forall \lambda A. A \Rightarrow A \Rightarrow A)) \cong (\{A : \text{Ty}\} \rightarrow \text{Tm Bool} \rightarrow \text{Tm } A \rightarrow \text{Tm } A \rightarrow \text{Tm } A).$$

Church encoding of a few more types:

empty	$\perp$	$\forall \lambda C. C$
unit	$\top$	$\forall \lambda C. C \Rightarrow C$
three-element		$\forall \lambda C. C \Rightarrow C \Rightarrow C \Rightarrow C$
sum	$A + B$	$\forall \lambda C. (A \Rightarrow C) \Rightarrow (B \Rightarrow C) \Rightarrow C$
product	$A \times B$	$\forall \lambda C. (A \Rightarrow B \Rightarrow C) \Rightarrow C$
natural numbers	Nat	$\forall \lambda C. C \Rightarrow (C \Rightarrow C) \Rightarrow C$
list of As	$\text{List } A$	$\forall \lambda C. C \Rightarrow (A \Rightarrow C \Rightarrow C) \Rightarrow C$
binary trees	Tree	$\forall \lambda C. C \Rightarrow (C \Rightarrow C \Rightarrow C) \Rightarrow C$

Exercise 165. Derive the constructors and iterators for the above types, show that their computation rules hold.

Food for thought 166. Show that any inductive type encoded by a strictly positive functor can be derived in System F (without η rule).

We can encode the other quantifier:

$$\begin{aligned} \exists : (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ \exists F := \forall \lambda C. (\forall \lambda X. F X \Rightarrow C) \Rightarrow C \end{aligned}$$

Exercise 167. Derive the constructor, destructor and β rule:

$$\begin{aligned} \text{mk}\exists : (X : \text{Ty}) \rightarrow \text{Tm } (F X) \rightarrow \text{Tm } (\exists F) \\ \text{case} : ((X : \text{Ty}) \rightarrow \text{Tm } (F X) \rightarrow \text{Tm } C) \rightarrow \text{Tm } (\exists F) \rightarrow \text{Tm } C \\ \exists\beta : \text{case } w \text{ (mk}\exists X f) = w X f \end{aligned}$$

We exemplify the encoding of coinductive types through streams. A stream is given by a type C , a coalgebra on C and an element of C called the seed from which the stream grows out.

$$\begin{aligned} \text{Stream} : \text{Ty} \rightarrow \text{Ty} \\ \text{Stream } A := \exists \lambda C. ((C \Rightarrow A) \times (C \Rightarrow C)) \times C \\ \text{head} : \text{Tm } (\text{Stream } A) \rightarrow \text{Tm } A \\ \text{head} := \text{case } (\lambda X w. \text{fst } (fst w) \cdot \text{snd } w) \\ \text{tail} : \text{Tm } (\text{Stream } A) \rightarrow \text{Tm } (\text{Stream } A) \\ \text{tail} := \text{case } (\lambda X w. \text{mk}\exists X (\text{fst } w, \text{snd } (fst w) \cdot \text{snd } w)) \\ \text{gen} : \{C : \text{Ty}\} \rightarrow (\text{Tm } C \rightarrow \text{Tm } A) \rightarrow (\text{Tm } C \rightarrow \text{Tm } C) \rightarrow \text{Tm } C \rightarrow \text{Tm } (\text{Stream } A) \\ \text{gen } \{C\} h t c := \text{mk}\exists C ((\text{lam } h, \text{lam } t), c) \end{aligned}$$

Exercise 168. Prove the computation rules of Church-encoded streams:

$$\begin{aligned} \text{Stream}\beta_1 : \text{head } (\text{gen } h t c) &= h c \\ \text{Stream}\beta_2 : \text{tail } (\text{gen } h t c) &= \text{gen } h t (t c) \end{aligned}$$

Food for thought 169. Show that any coinductive type encoded by a strictly positive functor can be derived in System F (without η rule).

Food for thought 170. If we have internal parametricity [ACKS24, Lor25], the η rules also hold [Wad90]. Can we extend System F with internal parametricity without adding dependent types?

Abstract types and interfaces can be encoded using existential types [MP88], and thus in System F. For example, if S is a type of Nat-stacks, then we want the following features:

empty	: $\text{Tm } S$	create an empty stack
push	: $\text{Tm } (\text{Nat} \Rightarrow S \Rightarrow S)$	put a Nat on the top of the stack
pop	: $\text{Tm } (S \Rightarrow S)$	remove an element from the top of the stack
top	: $\text{Tm } (S \Rightarrow \text{Nat})$	look at what is in the top of the stack
isEmpty	: $\text{Tm } (S \Rightarrow \text{Bool})$	let us know if the stack is empty

(Probably top should have type $\text{Tm}(S \Rightarrow (\text{Nat} + \top))$, but that is not the point here.) A program which computes a Nat using an abstract stack interface has the following type:

$$(\exists \lambda S. S \times (\text{Nat} \Rightarrow S \Rightarrow S) \times (S \Rightarrow S) \times (S \Rightarrow \text{Nat}) \times (S \Rightarrow \text{Bool})) \Rightarrow \text{Nat}.$$

Exercise 171. Define a term of the above type which creates a stack, then pushes 1, 2, 3 on it, pops one element, and returns what is on the top of the stack. Show that if the following equations hold for our stack implementation, the result will be 2:

$$\text{top} \cdot (\text{pop} \cdot (\text{push} \cdot n \cdot (\text{push} \cdot m \cdot s))) = m.$$

This program behaves uniformly for any stack implementation. Of course, there can be trivial stack implementations which don't behave well. If we want to statically restrict our stack implementation to correct ones, we need dependent types (Section 3.9). Otherwise, we can just assume that the stack implementation behaves well.

Exercise 172. How do you describe with equations such as the above that a stack implementation behaves correctly?

3.7.3 Kinds

TODO: translate.

System F-ben vannak polimorf függvények, de nincsenek típus szintű függvények. Például a listák nem adhatók meg, ami egy típusról típusra képező függvény. A következő rendszer ezt kijavítja. A típusokon és a termeken túl van egy újabb szortunk, a fajták (Kind-ok) szortja, a típusok pedig indexelve vannak az ő "típusukkal" (fajtájukkal). A $*$ fajtájú típusok a tulajdonképpeni típusok, nekik lehetnek termei. A $* \Rightarrow *$ fajtájú típusok a típus-függvények, melyek egy tulajdonképpeni típusból képeznek egy tulajdonképpeni típusba.

Definition 173 (System F_ω).

$$\begin{aligned} \text{Kind} & : \text{Set} \\ \text{Ty} & : \text{Kind} \rightarrow \text{Set} \\ - \Rightarrow - & : \text{Kind} \rightarrow \text{Kind} \rightarrow \text{Kind} \\ \text{LAM} & : (\text{Ty } K \rightarrow \text{Ty } L) \cong \text{Ty } (K \Rightarrow L) : - \bullet - \\ * & : \text{Kind} \\ \text{Tm} & : \text{Ty } * \rightarrow \text{Set} \\ \forall & : (\text{Ty } K \rightarrow \text{Ty } *) \rightarrow \text{Ty } * \\ \text{Lam} & : ((X : \text{Ty } K) \rightarrow \text{Tm } (A X)) \cong \text{Tm } (\forall A) : - \bullet - \\ - \Rightarrow - & : \text{Ty } * \rightarrow \text{Ty } * \rightarrow \text{Ty } * \\ \text{lam} & : (\text{Tm } A \rightarrow \text{Tm } B) \cong \text{Tm } (A \Rightarrow B) : - \cdot - \end{aligned}$$

A fajták és típusok a $\Rightarrow$ -val úgy viselkednek, mint egyszerű típuselméletben a típusok és a termek. A System F_ω érdekessége, hogy az egyik szort felhasznál egy operátort ($*$ -ot), tehát a szortok nem adhatók meg az operátoroktól elkülönülten.

A lista típus a következőképp Church-kódolható:

$$\begin{aligned} \text{List} & : \text{Ty } (* \Rightarrow *) \\ \text{List} & := \text{LAM } \lambda A. \forall \lambda B. B \Rightarrow (A \Rightarrow B \Rightarrow B) \Rightarrow B \end{aligned}$$

A Haskell bind operátorának a következő a típusa:

$$\forall \lambda M. \forall \lambda A. \forall \lambda B. (A \Rightarrow M \bullet B) \Rightarrow M \bullet A \Rightarrow M \bullet B$$

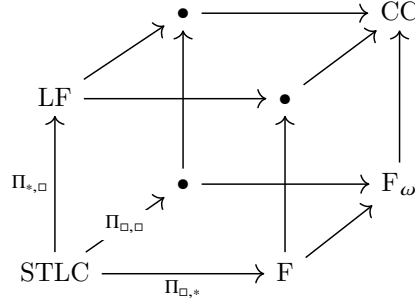
A λ -kat kihagyva és a $\forall$ által kötött típusok fajtaát odaírva a következőt kapjuk:

$$\forall (M : * \Rightarrow *). \forall (A : *). \forall (B : *). (A \Rightarrow M \bullet B) \Rightarrow M \bullet A \Rightarrow M \bullet B$$

Exercise 174. Definiáljuk a Church-kódolt lista típus konstruktorait és iterátorát (fold operátorát)!

Exercise 175. Adjuk meg a Church-kódolt lista típus return és bind operátorait és vizsgáljuk meg, hogy teljesítik-e a monád törvényeket!

The following definition shows that all the languages in the lambda cube [Bar91] can be given as SOGATs. The simply typed lambda calculus (STLC) only includes $\Pi_{*,*}$, and the edges in each dimension add one of the other three Π types, respectively. The calculus of constructions (CC) includes all four Π types.



We don't give names to the maps in the universal properties.

Definition 176 (CC).

$$\begin{aligned}
\Box & : \text{Set} \\
\text{Ty} & : \Box \rightarrow \text{Set} \\
* & : \Box \\
\text{Tm} & : \text{Ty} * \rightarrow \text{Set} \\
\Pi_{*,*} & : (A : \text{Ty} *) \rightarrow (\text{Tm } A \rightarrow \text{Ty} *) \rightarrow \text{Ty} * & \text{Tm } (\Pi_{*,*} A B) \cong (a : \text{Tm } A) \rightarrow \text{Tm } (B a) \\
\Pi_{*,\Box} & : (A : \text{Ty} *) \rightarrow (\text{Tm } A \rightarrow \Box) \rightarrow \Box & \text{Ty } (\Pi_{*,\Box} A L) \cong (a : \text{Tm } A) \rightarrow \text{Ty } (L a) \\
\Pi_{\Box,*} & : (K : \Box) \rightarrow (\text{Ty } K \rightarrow \text{Ty} *) \rightarrow \text{Ty} * & \text{Tm } (\Pi_{\Box,*} K B) \cong (A : \text{Ty } K) \rightarrow \text{Tm } (B A) \\
\Pi_{\Box,\Box} & : (K : \Box) \rightarrow (\text{Ty } K \rightarrow \Box) \rightarrow \Box & \text{Ty } (\Pi_{\Box,\Box} K L) \cong (A : \text{Ty } K) \rightarrow \text{Ty } (L A)
\end{aligned}$$

Food for thought 177. Show that all pure type systems are definable as SOGATs. See [CD07].

3.8 Fixpoint operator for types and terms, untyped calculi

If we have System F-style polymorphism, we don't need to add (co)inductive types by hand into our language. There is another very common way to add features to our language: adding noise, or in other words, making the language less safe. On the other hand, these languages are Turing-complete.

- S. 3.8.1 We can add a fixpoint-operator to our language and then we do not need iterators anymore, but we lose that every program terminates: we add meaningless recursive programs which simply loop. The language which has **Nat** without iterator but with a fixpoint operator is called PCF.
- S. 3.8.2 We can add recursive types by saying that every type operator has a fixpoint: we lose the distinction between inductive and coinductive types and now not only terms, but types contain rubbish.
- S. 3.8.3 We can remove all types, then we don't even need to add a fixpoint operator, we can just define it. This is the untyped lambda calculus. We have no guarantee that programs are meaningful, but we can implement anything.

3.8.1 PCF

A *fixpoint-operator* is given by $\text{fix} : (\text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } A$ which satisfies the equation $\text{fix } f = f (\text{fix } f)$.

Definition 178 (PCF). A second-order model of PCF has two sorts (types and type-indexed terms) and the following additional components using concise derivation rule notation:

$$\begin{array}{c}
\frac{A}{A \Rightarrow B} \quad \frac{B}{\text{lam } x.b : A \Rightarrow B} \quad \frac{x : A \vdash b : B}{f \cdot a : B} \quad \frac{f : A \Rightarrow B \quad a : A}{(\text{lam } x.b) \cdot a = b[x \mapsto a]} \quad \frac{x : A \vdash t : A}{\text{fix } x.t : A} \quad \frac{}{\text{fix } x.t = t[x \mapsto \text{fix } x.t]} \\
\\
\frac{}{\text{Nat}} \quad \frac{}{\text{zero} : \text{Nat}} \quad \frac{n : \text{Nat}}{\text{suc } n : \text{Nat}} \quad \frac{b : \text{Nat} \quad t : A \quad f : A}{\text{ifZero } b \, t \, f : A} \quad \frac{}{\text{ifZero zero } t \, f = t} \quad \frac{}{\text{ifZero } (\text{suc } n) \, t \, f = f} \\
\\
\frac{n : \text{Nat}}{\text{pred } n : \text{Nat}} \quad \frac{}{\text{pred } (\text{suc } n) = n} \quad \frac{}{\text{pred zero} = \text{zero}}
\end{array}$$

Example 179. We want to define the $x \mapsto x * 2 + 1$ function in a way that it satisfies the following equation:

$$f = \text{lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (f \cdot \text{pred } n)))}.$$

This cannot be seen as a definition because f appears on the right hand side of the equation. We fix this by abstracting over the recursive call:

$$\begin{aligned} g &: \text{Tm } (\text{Nat} \Rightarrow \text{Nat}) \rightarrow \text{Tm } (\text{Nat} \Rightarrow \text{Nat}) \\ g \ f &:= \text{lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (f \cdot \text{pred } n)))} \end{aligned}$$

The fixpoint of g satisfies the desired equation:

$$\begin{aligned} \text{fix } g &: \text{Tm } (\text{Nat} \Rightarrow \text{Nat}) \\ \text{fix } g & \quad \quad \quad = (\beta \text{ rule for fix}) \\ g \ (\text{fix } g) & \quad \quad \quad = (\text{definition of } g) \\ \text{lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (fix } g \cdot \text{pred } n)))} \end{aligned}$$

We also check that $\text{fix } g$ computes as expected:

$$\begin{aligned} \text{fix } g \cdot \text{suc (suc zero)} & \quad \quad \quad = (\beta \text{ rule for fix}) \\ (\text{lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (fix } g \cdot \text{pred } n)))}) \cdot \text{suc (suc zero)} & \quad \quad \quad = (\Rightarrow \beta) \\ \text{ifZero (suc (suc zero)) (suc zero) (suc (suc (fix } g \cdot \text{pred (suc (suc zero)))))} & \quad \quad \quad = (\text{ifZero's } \beta \text{ rule}) \\ \text{suc (suc (fix } g \cdot \text{pred (suc (suc zero)))))} & \quad \quad \quad = (\text{pred's } \beta \text{ rule}) \\ \text{suc (suc (fix } g \cdot \text{suc zero))} & \quad \quad \quad = (\beta \text{ rule for fix}) \\ \text{suc (suc (lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (fix } g \cdot \text{pred } n)))}) \cdot \text{suc zero)} & \quad \quad \quad = (\Rightarrow \beta) \\ \text{suc (suc (ifZero (suc zero) (suc zero) (suc (suc (fix } g \cdot \text{pred (suc zero)))))} & \quad \quad \quad = (\Rightarrow \beta) \\ \text{suc (suc (suc (suc (fix } g \cdot \text{pred (suc zero)))))} & \quad \quad \quad = (\text{pred's } \beta \text{ rule}) \\ \text{suc (suc (suc (suc (fix } g \cdot \text{zero))})} & \quad \quad \quad = (\beta \text{ rule for fix}) \\ \text{suc (suc (suc (suc (lam } \lambda n. \text{ifZero } n \text{ (suc zero) (suc (suc (fix } g \cdot \text{pred } n)))}) \cdot \text{zero}))} & \quad \quad \quad = (\Rightarrow \beta) \\ \text{suc (suc (suc (suc (ifZero zero (suc zero) (suc (suc (fix } g \cdot \text{pred zero)))))} & \quad \quad \quad = (\text{ifZero's } \beta \text{ rule}) \\ \text{suc (suc (suc (suc (suc zero))})} \end{aligned}$$

Exercise 180. Define the iterator of natural numbers and show its computation rules.

Exercise 181. Show weak equational consistency for any second-order model: that is, if $\text{zero} = \text{suc zero}$, then for all A and $u, v : \text{Tm } A$, $u = v$.

Exercise 182. Show that using a second-order model of PCF, a second-order model of System T without η can be given. See also Remark 163.

Exercise 183. Which functions do the following PCF terms define?

$$\begin{aligned} & \text{fix } (\lambda t. \text{lam } \lambda x. x) \\ & \text{fix } (\lambda t. \text{lam } \lambda x. \text{ifZero } x \ x) \\ & \text{fix } (\lambda t. \text{lam } \lambda x. \text{ifZero } x \ x \text{ (suc zero)}) \\ & \text{fix } (\lambda t. \text{lam } \lambda x. \text{ifZero } x \text{ (suc zero) zero}) \\ & \text{fix } \left(\lambda t. \text{lam } \lambda x. \text{ifZero } x \ x \text{ (suc (t \cdot pred x))} \right) \\ & \text{fix } \left(\lambda t. \text{lam } \lambda x. \text{ifZero } x \ \text{zero} \text{ (suc (t \cdot pred x))} \right) \\ & \text{fix } \left(\lambda t. \text{lam } \lambda x. \text{ifZero } x \text{ (suc zero) (suc (t \cdot pred x))} \right) \end{aligned}$$

The price we had to pay for having all recursive functions is that we have non-productive terms: $\text{fix } (\lambda x. x) : \text{Tm } A$ for any A , and we have $\text{fix } (\lambda x. x) = (\lambda x. x)(\text{fix } (\lambda x. x)) = \text{fix } (\lambda x. x)$, so we don't obtain any new information from computing. Similarly, $\text{fix suc} = \text{suc (fix suc)} = \text{suc (suc (fix suc))} = \dots$, which will be a very large term never terminating. Compare this with $\text{fix } (\lambda_. t) = (\lambda_. t) (\text{fix } (\lambda_. t)) = t$.

3.8.2 Recursive types

We have a fixpoint operator for $F : \text{Ty} \rightarrow \text{Ty}$ functions, that is, an operator $\text{Fix} : (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Ty}$. There are two ways to express that $\text{Fix } F$ is the fixpoint of F :

- *equirecursive types*, that is, we add the equation $\text{Fix } F = F (\text{Fix } F)$. This makes equality of types undecidable (and thus typechecking undecidable).
- *isorecursive types*: instead of adding an equation in types, we add an isomorphism $\text{Fix } F \cong F (\text{Fix } F)$.

We follow the latter route.

Definition 184 (A language with recursive types). *A second-order model extends Fin (Definition 99) with the following components:*

$$\begin{aligned} \text{fix} & : (\text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } A \\ \text{fix}\beta & : \text{fix } f = f (\text{fix } f) \\ \text{Fix} & : (\text{Ty} \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ \text{con} & : \text{Tm } (F (\text{Fix } F)) \cong \text{Tm } (\text{Fix } F) : \text{des} \end{aligned}$$

Example 185 (Nat). *We define natural numbers as a recursive type. We only write the implicit argument $\{\lambda X. \top + X\}$ of con once.*

$$\begin{aligned} \text{Nat} & := \text{Fix } \lambda X. \top + X \\ \text{zero} & := \text{con } \{\lambda X. \top + X\} (\text{inl tt}) \\ \text{suc} & : \text{Tm } \text{Nat} \rightarrow \text{Tm } \text{Nat} \\ \text{suc } n & := \text{con } (\text{inr } n) \\ \text{ite} & : \text{Tm } A \rightarrow (\text{Tm } A \rightarrow \text{Tm } A) \rightarrow \text{Tm } (\text{Nat} \Rightarrow A) \\ \text{ite } z \ s & := \text{fix } \lambda f. \text{lam } \lambda n. \text{case } (\lambda _. z) (\lambda m. s (f \cdot m)) (\text{des } n) \\ \text{Nat}\beta_1 : \text{ite } z \ s \cdot \text{zero} & \quad \quad \quad = (\text{definition}) \\ & (\text{fix } \lambda f. \text{lam } \lambda n. \text{case } (\lambda _. z) (\lambda m. s (f \cdot m)) (\text{des } n)) \cdot \text{zero} = (\text{fix}\beta) \\ & (\text{lam } \lambda n. \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des } n)) \cdot \text{zero} = (\Rightarrow\beta) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des zero}) = (\text{definition}) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des } (\text{con } (\text{inl tt}))) = (\text{Fix}\beta) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{inl tt}) = (+\beta_1) \\ & z \\ \text{Nat}\beta_2 : \text{ite } z \ s \cdot \text{suc } n & \quad \quad \quad = (\text{definition}) \\ & (\text{fix } \lambda f. \text{lam } \lambda n. \text{case } (\lambda _. z) (\lambda m. s (f \cdot m)) (\text{des } n)) \cdot \text{suc } n = (\text{fix}\beta) \\ & (\text{lam } \lambda n. \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des } n)) \cdot \text{suc } n = (\Rightarrow\beta) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des } (\text{suc } n)) = (\text{definition}) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{des } (\text{con } (\text{inr } n))) = (\text{Fix}\beta) \\ & \text{case } (\lambda _. z) (\lambda m. s (\text{ite } z \ s \cdot m)) (\text{inr } n) = (+\beta_2) \\ & s (\text{ite } z \ s \cdot n) \end{aligned}$$

Food for thought 186. *Does $\text{Nat}\eta$ hold?*

Example 187 (Stream). *We define streams as a recursive type.*

$$\begin{aligned} \text{Stream} & : \text{Ty} \rightarrow \text{Ty} \\ \text{Stream } A & := \text{Fix } \lambda A. A \times X \\ \text{head} & : \text{Tm } (\text{Stream } A) \rightarrow \text{Tm } A \\ \text{head } t & := \text{fst } (\text{des } t) \\ \text{tail} & : \text{Tm } (\text{Stream } A) \rightarrow \text{Tm } (\text{Stream } A) \\ \text{tail } t & := \text{snd } (\text{des } t) \\ \text{gen} & : (\text{Tm } C \rightarrow \text{Tm } A) \rightarrow (\text{Tm } C \rightarrow \text{Tm } C) \rightarrow \text{Tm } (C \Rightarrow \text{Stream } A) \\ \text{gen } h \ t & := \text{fix } \lambda f. \text{lam } \lambda c. \text{con } (h \ c, f \cdot t \ c) \end{aligned}$$

Exercise 188 (long). *Prove the β rules for streams.*

Exercise 189 (long). *Show that for a strictly positive $F : \text{Ty} \rightarrow \text{Ty}$, $\text{Fix } F$ satisfies all the rules of $\text{Ind } F$ and $\text{Coind } F$.*

We have meaningless types such as $D := \text{Fix}(\lambda X.X)$ which gives us a $\text{con} : \text{Tm } D \rightarrow \text{Tm } D$ and $\text{des} : \text{Tm } D \rightarrow \text{Tm } D$, but we cannot create any element of D and cannot obtain any information from assumed elements of D .

3.8.3 Untyped lambda calculus

The simplest programming language with binders is the untyped lambda calculus.

Definition 190 (Untyped lambda calculus).

$$\begin{aligned} \text{Tm} &: \text{Set} \\ - \cdot - &: \text{Tm} \rightarrow \text{Tm} \rightarrow \text{Tm} \\ \text{lam} &: (\text{Tm} \rightarrow \text{Tm}) \rightarrow \text{Tm} \\ \beta &: \text{lam } f \cdot u = f u \end{aligned}$$

A fixpoint combinator:

$$Y : \text{Tm} := \text{lam } \lambda f. (\text{lam } \lambda x. f \cdot (x \cdot x)) \cdot (\text{lam } \lambda x. f \cdot (x \cdot x)).$$

Exercise 191. *Show that $Y \cdot f = f \cdot (Y \cdot f)$.*

Exercise 192. *Show that there are terms $\text{zero}, \text{suc}, \text{ite} : \text{Tm}$ such that $\text{ite} \cdot z \cdot s \cdot \text{zero} = z$ and $\text{ite} \cdot z \cdot s \cdot (\text{suc } t) = s \cdot (\text{ite} \cdot z \cdot s \cdot t)$.*

Exercise 193. *Show that we have similar encoding for all inductive and coinductive types. What about η rules?*

We have another untyped language which has one more operator and one more equation, but is first-order:

Definition 194 (Untyped combinator calculus).

$$\begin{aligned} \text{Tm} &: \text{Set} \\ - \cdot - &: \text{Tm} \rightarrow \text{Tm} \rightarrow \text{Tm} \\ K &: \text{Tm} \\ S &: \text{Tm} \\ K\beta &: K \cdot a \cdot b = a \\ S\beta &: S \cdot f \cdot g \cdot a = f \cdot a \cdot (g \cdot a) \end{aligned}$$

See [AKSV23] for the relationship of untyped lambda and combinator calculi.

It is remarkable that Schönfinkel's untyped combinator calculus is still the simplest Turing complete language, even though it was the first such. Note that in contrast with STCC, untyped combinator calculus does not have normalisation.

TODO: add exercises from Hungarian course notes, including fixpoint operator for SK.

3.9 Martin-Löf type theory

Definition 195 (Minimal Martin-Löf type theory (mini-MLTT)).

$$\begin{aligned} \text{Ty} &: \text{Set} \\ \text{Tm} &: \text{Ty} \rightarrow \text{Set} \\ \iota &: \text{Ty} \\ \Pi &: (A : \text{Ty}) \rightarrow (\text{Tm } A \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ \text{lam} &: ((a : \text{Tm } A) \rightarrow \text{Tm } (B a)) \cong \text{Tm } (\Pi A B) : - \cdot - \end{aligned}$$

Definition 196 (Martin-Löf type theory with Π and universes).

$$\begin{aligned} \text{Ty} &: \mathbb{N} \rightarrow \text{Set} & \text{U} &: (i : \mathbb{N}) \rightarrow \text{Ty } (1 + i) \\ \text{Tm} &: \text{Ty } i \rightarrow \text{Set} & \text{c} &: \text{Ty } i \cong \text{Tm } (\text{U } i) : \text{El} \\ \Pi &: (A : \text{Ty } i) \rightarrow (\text{Tm } A \rightarrow \text{Ty } i) \rightarrow \text{Ty } i & \text{Lift} &: \text{Ty } i \rightarrow \text{Ty } (1 + i) \\ \text{lam} &: ((a : \text{Tm } A) \rightarrow \text{Tm } (B a)) \cong \text{Tm } (\Pi A B) : - \cdot - & \text{mk} &: \text{Tm } A \cong \text{Tm } (\text{Lift } A) : \text{un} \end{aligned}$$

Definition 197 (Martin-Löf type theory with inductive types). *We extend Definition 196 with the following.*

$$\begin{aligned}
\Sigma & : (A : \text{Ty } i) \rightarrow (\text{Tm } A \rightarrow \text{Ty } i) \rightarrow \text{Ty } i \\
(-, -) & : (a : \text{Tm } A) \times \text{Tm } (B a) \cong \text{Tm } (\Sigma A B) : \text{fst}, \text{snd} \\
\perp & : \text{Ty } 0 \\
\text{exfalse} & : \text{Tm } \perp \rightarrow \text{Tm } A \\
\top & : \text{Ty } 0 \\
\text{tt} & : \top \cong \text{Tm } \top \\
\text{Bool} & : \text{Ty } 0 \\
\text{true} & : \text{Tm } \text{Bool} \\
\text{false} & : \text{Tm } \text{Bool} \\
\text{indBool} & : (C : \text{Tm } \text{Bool} \rightarrow \text{Ty } i) \rightarrow \text{Tm } (C \text{ true}) \rightarrow \text{Tm } (C \text{ false}) \rightarrow (b : \text{Tm } \text{Bool}) \rightarrow \text{Tm } (C b) \\
\text{Bool}\beta_1 & : \text{indBool } t \text{ f true} = t \\
\text{Bool}\beta_2 & : \text{indBool } t \text{ f false} = f \\
\text{Id} & : (A : \text{Ty } i) \rightarrow \text{Tm } A \rightarrow \text{Tm } A \rightarrow \text{Ty } i \\
\text{refl} & : (a : \text{Tm } A) \rightarrow \text{Tm } (\text{Id } A a a) \\
J & : (C : (x : \text{Tm } A) \rightarrow \text{Tm } (\text{Id } A a x) \rightarrow \text{Ty } i) \rightarrow \\
& \quad \text{Tm } (C a (\text{refl } a)) \rightarrow (x : \text{Tm } A) (e : \text{Tm } (\text{Id } A a x)) \rightarrow \text{Tm } (C x e) \\
\text{Id}\beta & : J C w a (\text{refl } a) = w \\
W & : (S : \text{Ty } i) \rightarrow (\text{Tm } S \rightarrow \text{Ty } i) \rightarrow \text{Ty } i \\
\text{sup} & : (s : \text{Tm } S) \rightarrow (\text{Tm } (P s) \rightarrow \text{Tm } (W S P)) \rightarrow \text{Tm } (W S P) \\
\text{indW} & : (C : \text{Tm } (W S P) \rightarrow \text{Ty } i) \rightarrow \left(((p : \text{Tm } (P s)) \rightarrow \text{Tm } (C (f p))) \rightarrow \text{Tm } (C (\text{sup } s f)) \right) \rightarrow \\
& \quad (w : \text{Tm } (W S P)) \rightarrow \text{Tm } (C w) \\
W\beta & : \text{indW } C h (\text{sup } s f) = h (\lambda p. \text{indW } C h (f p))
\end{aligned}$$

TODO: extensional t.t. (equality reflection). Quotient types.

3.10 Primitive recursive arithmetic

We separate first-order types from general types so that we can restrict the iterator to not return functions. First-order types are Nat and products, general types include functions.

Definition 198 (Primitive recursive arithmetic (PRA)). *A second-order model comprises the following com-*

ponents:

$$\begin{aligned}
\text{Ty} & : \text{Set} \\
\text{Tm} & : \text{Ty} \rightarrow \text{Set} \\
\text{FTy} & : \text{Set} \\
\ulcorner _ \urcorner & : \text{FTy} \rightarrow \text{Ty} \\
\text{Nat} & : \text{FTy} \\
\text{zero} & : \text{Tm } \ulcorner \text{Nat} \urcorner \\
\text{suc} & : \text{Tm } \ulcorner \text{Nat} \urcorner \rightarrow \text{Tm } \ulcorner \text{Nat} \urcorner \\
\text{ite} & : \text{Tm } \ulcorner A \urcorner \rightarrow (\text{Tm } \ulcorner A \urcorner \rightarrow \text{Tm } \ulcorner A \urcorner) \rightarrow \text{Tm } \ulcorner \text{Nat} \urcorner \rightarrow \text{Tm } \ulcorner A \urcorner \\
\text{Nat}\beta_1 & : \text{ite } z \ s \ \text{zero} = z \\
\text{Nat}\beta_2 & : \text{ite } z \ s \ (\text{suc } t) = s \ (\text{ite } z \ s \ t) \\
- \times - & : \text{FTy} \rightarrow \text{FTy} \rightarrow \text{FTy} \\
-, - & : \text{Tm } \ulcorner A \urcorner \rightarrow \text{Tm } \ulcorner B \urcorner \rightarrow \text{Tm } \ulcorner A \times B \urcorner \\
\text{fst} & : \text{Tm } \ulcorner A \times B \urcorner \rightarrow \text{Tm } \ulcorner A \urcorner \\
\text{snd} & : \text{Tm } \ulcorner A \times B \urcorner \rightarrow \text{Tm } \ulcorner B \urcorner \\
\times\beta_1 & : \text{fst } (a, b) = a \\
\times\beta_2 & : \text{snd } (a, b) = b \\
- \Rightarrow - & : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty} \\
\text{lam} & : (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (A \Rightarrow B) \\
- \cdot - & : \text{Tm } (A \Rightarrow B) \rightarrow \text{Tm } A \rightarrow \text{Tm } B \\
\Rightarrow\beta & : \text{lam } f \cdot a = f a
\end{aligned}$$

The Ackermann function [Ack28] is not definable in this language.

Food for thought 199. Show that the $\ulcorner \text{Nat} \urcorner \Rightarrow \ulcorner \text{Nat} \urcorner$ functions definable in our theory are the same as in the traditional presentation of primitive recursive arithmetic. See [BSvB24] for a PRA similar to ours which also features dependent types.

3.11 First-order logic

Definition 200 (Minimal intuitionistic first-order logic with natural deduction style proof theory).

$$\begin{array}{c}
\frac{}{\text{For} : \text{Set}} \quad \frac{}{\text{Tm} : \text{Set}} \quad \frac{A : \text{For} \quad B : \text{For}}{A \supset B : \text{For}} \quad \frac{x : \text{Tm} \vdash A : \text{For}}{\forall x. A : \text{For}} \quad \frac{t : \text{Tm} \quad t' : \text{Tm}}{\text{Eq } t \ t' : \text{For}} \quad \frac{A : \text{For}}{\text{Pf } A : \text{Set}} \\
\frac{p : \text{Pf } A \quad q : \text{Pf } A}{p = q} \quad \frac{\text{Pf } A \vdash \text{Pf } B}{\text{Pf } (A \supset B)} \quad \frac{\text{Pf } (A \supset B) \quad \text{Pf } A}{\text{Pf } B} \quad \frac{x : \text{Tm} \vdash \text{Pf } A}{\text{Pf } (\forall x. A)} \quad \frac{\text{Pf } (\forall x. A) \quad t : \text{Tm}}{\text{Pf } (A[x \mapsto t])} \\
\frac{t : \text{Tm}}{\text{Pf } (\text{Eq } t \ t)} \quad \frac{x : \text{Tm} \vdash A : \text{For} \quad \text{Pf } (\text{Eq } t \ t') \quad \text{Pf } (A[x \mapsto t])}{\text{Pf } (A[x \mapsto t'])}
\end{array}$$

Exercise 201. Define full intuitionistic first-order logic as a SOGAT. Add something to make it classical.

3.12 Theories of signatures for (SO)(G)ATs

Theories of signatures (ToSs) are languages for describing signatures for algebraic theories. Above we defined (SO)GATs by giving the notion of (second-order) model, signatures are a more precise language which enforces that all operators result in a sort, strict positivity etc.

Definition 202 (AT). A second-order model of the ToS for ATs:

$$\begin{aligned}
\text{Ty} & : \text{Set} \\
\text{Tm} & : \text{Ty} \rightarrow \text{Set} \\
\Sigma & : (A : \text{Ty}) \rightarrow (\text{Tm } A \rightarrow \text{Ty}) \rightarrow \text{Ty} \\
\text{Srt} & : \text{Ty} \\
\Pi\text{Srt} & : (\text{Tm } \text{Srt} \rightarrow \text{Ty}) \rightarrow \text{Ty} \\
- \cdot - & : \text{Tm } (\Pi\text{Srt } B) \rightarrow (x : \text{Tm } \text{Srt}) \rightarrow \text{Tm } (B x) \\
\text{Id} & : \text{Tm } \text{Srt} \rightarrow \text{Tm } \text{Srt} \rightarrow \text{Ty}
\end{aligned}$$

A signature is an element of Ty.

For a $B : \text{Ty}$, we introduce the abbreviation $\text{Srt} \Rightarrow B := \Pi \text{Srt} (\lambda _ . B)$.

Example 203. *The AT of monoids (see Definition 1) is given by the following signature:*

$$\begin{aligned} & \Sigma (\text{Srt} \Rightarrow \text{Srt} \Rightarrow \text{Srt}) \lambda op. \Pi \text{Srt} \lambda x. \Pi \text{Srt} \lambda y. \Pi \text{Srt} \lambda z. \text{Id} (op \cdot (op \cdot x \cdot y) \cdot z) (op \cdot x \cdot (op \cdot y \cdot z)) \times \\ & \Sigma \text{Srt} \lambda u. (\Pi \text{Srt} \lambda x. \text{Id} (op \cdot u \cdot x) x) \times (\Pi \text{Srt} \lambda x. \text{Id} (op \cdot x \cdot u) x) \end{aligned}$$

Exercise 204. *Define the signature for the following ATs: pointed set with an endofunction (see Definition 10), untyped combinator calculus.*

Definition 205 (GAT). *A second-order model of the ToS for GATs:*

$$\begin{aligned} \text{Ty} & : \text{Set} \\ \text{Tm} & : \text{Ty} \rightarrow \text{Set} \\ \Sigma & : (A : \text{Ty}) \rightarrow (\text{Tm } A \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ (-, -) & : (a : \text{Tm } A) \times \text{Tm } (B a) \cong \text{Tm } (\Sigma A B) : \text{fst}, \text{snd} \\ \text{U} & : \text{Ty} \\ \text{El} & : \text{Tm } \text{U} \rightarrow \text{Ty} \\ \Pi & : (a : \text{Tm } \text{U}) \rightarrow (\text{Tm } (\text{El } a) \rightarrow \text{Ty}) \rightarrow \text{Ty} \\ - \cdot - & : \text{Tm } (\Pi a B) \rightarrow (x : \text{Tm } (\text{El } a)) \rightarrow \text{Tm } (B x) \\ \text{Id} & : (a : \text{Tm } \text{U}) \rightarrow \text{Tm } (\text{El } a) \rightarrow \text{Tm } (\text{El } a) \rightarrow \text{Ty} \\ \text{reflect} & : \text{Tm } (\text{Id } a u v) \rightarrow u = v \end{aligned}$$

A signature is an element of Ty.

We will use the abbreviations $A \Rightarrow B := \Pi A (\lambda _ . B)$ and $A \times B := \Sigma A (\lambda _ . B)$.

Example 206. *The GAT of preorders (see Definition 58) is given by the following signature:*

$$\begin{aligned} & \Sigma \text{U} \lambda Ob. \Sigma \\ & (Ob \Rightarrow Ob \Rightarrow \text{U}) \lambda Mor. \\ & (\Pi Ob \lambda I. \Pi Ob \lambda J. \Pi Ob \lambda K. Mor \cdot J \cdot I \Rightarrow Mor \cdot K \cdot J \Rightarrow \text{El } (Mor \cdot K \cdot I)) \times \\ & (\Pi Ob \lambda I. \text{El } (Mor \cdot I \cdot I)) \times \\ & (\Pi Ob \lambda I. \Pi Ob \lambda J. \Pi (Mor \cdot J \cdot I) \lambda f. \Pi (Mor \cdot J \cdot I) \lambda g. \text{Id } (Mor \cdot J \cdot I) f g) \end{aligned}$$

Exercise 207. *Define the signature for the following GATs: category, Razor, STCC.*

Definition 208 (SOGAT). *A second-order model of the ToS for SOGATs is a second-order model of the ToS for GATs (Definition 205) extended with the following components:*

$$\begin{aligned} \text{U}^+ & : \text{Ty} \\ \text{el}^+ & : \text{Tm } \text{U}^+ \rightarrow \text{Tm } \text{U} \\ \pi^+ & : (a^+ : \text{Tm } \text{U}^+) \rightarrow (\text{Tm } (\text{El } (\text{el}^+ a^+)) \rightarrow \text{Tm } \text{U}) \rightarrow \text{Tm } \text{U} \\ \text{lam}^+ & : ((x : \text{El } (\text{el}^+ a^+)) \rightarrow \text{Tm } (\text{El } (b x))) \cong \text{Tm } (\text{El } (\pi^+ a^+ b)) : - \cdot^+ - \end{aligned}$$

U^+ is the universe of those sorts which can appear at the left hand side of an arrow in an argument of an operator. In STLC, Ty is in U, while Tm is in U^+ .

Example 209. *The signature for STLC (see Definition 62):*

$$\begin{aligned}
& \Sigma \cup \lambda Ty. \Sigma \\
& (Ty \Rightarrow U^+) \lambda Tm. \\
& El Ty \times \Sigma \\
& (Ty \Rightarrow Ty \Rightarrow El Ty) \lambda arr. \Sigma \\
& \left(\Pi Ty \lambda A. \Pi Ty \lambda B. (Tm \cdot A \Rightarrow^+ el^+ (Tm \cdot B)) \Rightarrow El (el^+ (Tm \cdot (arr \cdot A \cdot B))) \right) \lambda lam. \Sigma \\
& \left(\Pi Ty \lambda A. \Pi Ty \lambda B. el^+ (Tm \cdot (arr \cdot A \cdot B)) \Rightarrow el^+ (Tm \cdot A) \Rightarrow El (el^+ (Tm \cdot B)) \right) \lambda app. \\
& \left(\Pi Ty \lambda A. \Pi Ty \lambda B. \Pi (Tm \cdot A \Rightarrow^+ el^+ (Tm \cdot B)) \lambda b. \Pi (el^+ (Tm \cdot A)) \lambda a. \right. \\
& \quad \left. Id (el^+ (Tm \cdot B)) (app \cdot A \cdot B \cdot (lam \cdot A \cdot B \cdot b) \cdot a) (b \cdot^+ a) \right) \times \\
& \left(\Pi Ty \lambda A. \Pi Ty \lambda B. \Pi (Tm \cdot (arr \cdot A \cdot B)) \lambda f. \right. \\
& \quad \left. Id (el^+ (Tm \cdot (arr \cdot A \cdot B))) f (lam \cdot A \cdot B \cdot (lam^+ \lambda x. app \cdot A \cdot B \cdot f \cdot x)) \right)
\end{aligned}$$

Exercise 210. *Define the signature for the following SOGATs: untyped lambda calculus, System T, PCF, first-order logic, System F, System F_ω , Martin-Löf type theory, ToS of ATs, ToS of GATs, ToS of SOGATs. Carefully consider which sorts should be in U and which should be in U^+ .*

Remark: we only showed how to define *closed* SOGATs, but e.g. Definition 196 is open as it refers to the external set $\mathbb{N}$. The ToS of (SO)GATs can be extended to allow open signatures, see [KX24].

4 Converting SOGATs into GATs

As mentioned in the beginning of Section 3, second-order algebraic theories are not really algebraic, there is no way to define morphisms of second-order models:

Example 211. *A second-order model of the untyped lambda calculus consists of a $Tm : Set$ together with*

$$lam : (Tm \rightarrow Tm) \cong Tm : - \cdot -.$$

A morphism between second-order models M and N would be a function $Tm : Tm_M \rightarrow Tm_N$ which preserves $- \cdot -$ by $Tm(t_M \cdot_M a_M) = Tm t_M \cdot_N Tm a_M$, but how do we express preservation of lam ? This would be an equation such as $Tm(lam_M f_M) = lam_N (Tm \circ f_M \circ ?)$, but we don't know what to put in place of the $?$ which is in $Tm_N \rightarrow Tm_M$. If we wanted to only define isomorphisms, this would work.

Instead, we translate SOGATs to GATs and work with the resulting GAT. For GATs, we have good notions of morphism, dependent model, syntax, and so on, as we have seen in Section 2. That is, by a category of models of a SOGAT, we mean the category of models of the GAT which is the result of the translation.

Food for thought 212. *Actually, there are multiple different translations which end up in different notions of models, e.g. instead of parallel substitution calculus, one can use single substitutions [KX24], models can be contextual (where contexts are inductively built, which is a special case of models where certain sorts are inductively generated), contexts can come with concatenation, and so on. For each SOGAT, there should also be a combinatory first-order GAT, just like there is the combinatory first-order version of the SOGAT of lambda calculus.*

4.1 STLC

The idea of the translation is that we introduce a new sort of contexts which are a list of the free variables. We index the sorts of our theory with contexts, the context index lists the possible free variables in a term. This way we can get rid of second-order function spaces. For example, the $lam : (Tm A \rightarrow Tm B) \rightarrow Tm (A \Rightarrow B)$ second-order operation becomes $lam : Tm (\Gamma \triangleright A) B \rightarrow Tm \Gamma (A \Rightarrow B)$. That is, the $Tm A$ dependency of the $Tm B$ argument of lam becomes an extra variable of type A in the context (we read $\Gamma \triangleright A$ as the context Γ extended with an extra free variable of type A). We also introduce a sort of substitutions with their action on terms called instantiation. These allow expressing the β law in STLC: $lam b \cdot a = b a$ becomes $lam b \cdot a = b[id, a]$ where $-[-]$ is the instantiation operator for Tm , (id, a) is a substitution from Γ to $\Gamma \triangleright A$ which leaves Γ untouched and substitutes a for the free variable of type A . The GAT that we obtain is called an explicit substitution calculus with parallel substitutions.

We first present and explain the result of the translation for STLC.

Definition 213 (First-order model of STLC (optimised version), see Definition 62 and Example 209).

$\text{Con} : \text{Set}$	$- \triangleright - : \text{Con} \rightarrow \text{Ty} \rightarrow \text{Con}$
$\text{Sub} : \text{Con} \rightarrow \text{Con} \rightarrow \text{Set}$	$-, - : \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta A \rightarrow \text{Sub } \Delta (\Gamma \triangleright A)$
$- \circ - : \text{Sub } \Delta \Gamma \rightarrow \text{Sub } \Theta \Delta \rightarrow \text{Sub } \Theta \Gamma$	$\mathbf{p} : \text{Sub } (\Gamma \triangleright A) \Gamma$
$\text{ass} : (\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta)$	$\mathbf{q} : \text{Tm } (\Gamma \triangleright A) A$
$\text{id} : \text{Sub } \Gamma \Gamma$	$\triangleright \beta_1 : \mathbf{p} \circ (\gamma, a) = \gamma$
$\text{idl} : \text{id} \circ \gamma = \gamma$	$\triangleright \beta_2 : \mathbf{q}[\gamma, a] = a$
$\text{idr} : \gamma \circ \text{id} = \gamma$	$\triangleright \eta : \sigma = (\mathbf{p} \circ \sigma, \mathbf{q}[\sigma])$
$\diamond : \text{Con}$	$\iota : \text{Ty}$
$\epsilon : \text{Sub } \Gamma \diamond$	$- \Rightarrow - : \text{Ty} \rightarrow \text{Ty} \rightarrow \text{Ty}$
$\diamond \eta : (\sigma : \text{Sub } \Gamma \diamond) \rightarrow \sigma = \epsilon$	$\text{lam} : \text{Tm } (\Gamma \triangleright A) B \rightarrow \text{Tm } \Gamma (A \Rightarrow B)$
$\text{Ty} : \text{Set}$	$\text{lam}[] : (\text{lam } b)[\gamma] = \text{lam } (b[\gamma \circ \mathbf{p}, \mathbf{q}])$
$\text{Tm} : \text{Con} \rightarrow \text{Ty} \rightarrow \text{Set}$	$- \cdot - : \text{Tm } \Gamma (A \Rightarrow B) \rightarrow \text{Tm } \Gamma A \rightarrow \text{Tm } \Gamma B$
$-[-] : \text{Tm } \Gamma A \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta A$	$\cdot [] : (f \cdot a)[\gamma] = (f[\gamma]) \cdot (a[\gamma])$
$[\circ] : a[\gamma \circ \delta] = a[\gamma][\delta]$	$\beta : \text{lam } b \cdot a = b[\text{id}, a]$
$[\text{id}] : a[\text{id}] = a$	$\eta : f = \text{lam } (f[\mathbf{p}] \cdot \mathbf{q})$

Contexts (lists of types) and substitutions (context morphisms, lists of terms) form a category $(\text{Con}, \dots, \text{idr})$ with a terminal object ($\diamond$ is the empty context denoting no free variables, ϵ is the empty substitution into $\diamond$, $\diamond \eta$ expresses that it is unique). The sort Ty is unchanged compared to the second-order version. The sort of terms however has an extra context-index. $\text{Tm } \Gamma A$ is a term of type A which might have free variables which are declared in Γ . The $- \triangleright -$ context extension operator allows us to put extra variables in a context. For example, the context $\diamond \triangleright \iota \triangleright \iota \Rightarrow \iota$ has two free variables of types ι and $\iota \Rightarrow \iota$, respectively. The sort $\text{Sub } \Delta \Gamma$ is a substitution from Δ to Γ which means a list of Γ -many terms, all in context Δ . For example, if $\Gamma = \diamond \triangleright \iota \triangleright \iota \Rightarrow \iota$, then $\text{Sub } \Delta \Gamma \cong \text{Tm } \Delta \iota \times \text{Tm } \Delta (\iota \Rightarrow \iota)$. This is ensured by the components $-, -, \dots, \triangleright \eta$ which can be summarised as

$$(\mathbf{p} \circ -, \mathbf{q}[-]) : \text{Sub } \Delta (\Gamma \triangleright A) \cong \text{Sub } \Delta \Gamma \times \text{Tm } \Delta A : (-, -).$$

Instantiation $-[-]$ gives meaning to variables via substitutions. For example, assume a term which depends only on a variable of type $\iota \Rightarrow \iota$, that is, $t : \text{Tm } (\diamond \triangleright \iota \Rightarrow \iota) A$, and assume a closed term of type $\iota \Rightarrow \iota$, that is $u : \text{Tm } \diamond (\iota \Rightarrow \iota)$. Now we can substitute u into t making it closed: $t[\epsilon, u] : \text{Tm } \diamond A$. The rules $\text{lam}[], \cdot []$ explain how to commute instantiation and term formers, the rule $\triangleright \beta_2$ explains how to instantiate the last variable (De Bruijn index 0) in the context. Further De Bruijn indices are given by weakening: $1 = \mathbf{q}[\mathbf{p}]$, $2 = \mathbf{q}[\mathbf{p}][\mathbf{p}]$, $3 = \mathbf{q}[\mathbf{p}][\mathbf{p}][\mathbf{p}]$, and so on. The rule $\text{lam}[]$ is interesting: here $\gamma : \text{Sub } \Delta \Gamma$ and $b : \text{Tm } (\Gamma \triangleright A) B$, so $b[\gamma]$ does not make sense. We have to *lift* γ so that we obtain a substitution which does not touch the last variable, $(\gamma \circ \mathbf{p}, \mathbf{q}) : \text{Sub } (\Delta \triangleright A) (\Gamma \triangleright A)$, and apply this under the lam .

Example 214 (Naturality of $-, -$). *We prove that in a first-order model of STLC, $(\gamma, a) \circ \delta = (\gamma \circ \delta, a[\delta])$ for any γ, a and δ .*

$$\begin{aligned}
(\gamma, a) \circ \delta &= (\triangleright \eta) \\
(\mathbf{p} \circ ((\gamma, a) \circ \delta), \mathbf{q}[(\gamma, a) \circ \delta]) &= (\text{ass}) \\
((\mathbf{p} \circ (\gamma, a)) \circ \delta, \mathbf{q}[(\gamma, a) \circ \delta]) &= (\triangleright \beta_1) \\
(\gamma \circ \delta, \mathbf{q}[(\gamma, a) \circ \delta]) &= ([\circ]) \\
(\gamma \circ \delta, \mathbf{q}[\gamma, a][\delta]) &= (\triangleright \beta_2) \\
(\gamma \circ \delta, a[\delta]) &= (\triangleright \beta_2)
\end{aligned}$$

Exercise 215. *Prove that assuming naturality of $-, -$ (i.e. $(\gamma, a) \circ \delta = (\gamma \circ \delta, a[\delta])$), the functor laws for $\text{Tm}([\circ], [\text{id}])$ can be derived.*

Exercise 216. *Define substitutions which swap variables, duplicate variables, forget variables somewhere in the middle of the context, i.e. elements of the following sets:*

$$\begin{aligned}
&\text{Sub } (\Gamma \triangleright A \triangleright B_1 \triangleright \dots \triangleright B_n \triangleright C \triangleright D_1 \triangleright \dots \triangleright D_m) (\Gamma \triangleright C \triangleright B_1 \triangleright \dots \triangleright B_n \triangleright A \triangleright D_1 \triangleright \dots \triangleright D_m) \\
&\text{Sub } (\Gamma \triangleright A \triangleright B_1 \triangleright \dots \triangleright B_n \triangleright C_1 \triangleright \dots \triangleright C_m) (\Gamma \triangleright A \triangleright B_1 \triangleright \dots \triangleright B_n \triangleright A \triangleright C_1 \triangleright \dots \triangleright C_m) \\
&\text{Sub } (\Gamma \triangleright A \triangleright B_1 \triangleright \dots \triangleright B_n) (\Gamma \triangleright B_1 \triangleright \dots \triangleright B_n)
\end{aligned}$$

Exercise 217. *Lifting of $\gamma : \text{Sub } \Delta \Gamma$ is $\gamma^\uparrow := (\gamma \circ \mathbf{p}, \mathbf{q})$. Show $(\gamma \circ \delta)^\uparrow = (\gamma^\uparrow) \circ (\delta^\uparrow)$, $\text{id}^\uparrow = \text{id}$, $\mathbf{p}^\uparrow \circ (\text{id}, \mathbf{q}) = \text{id}$.*

Example 218 (C.f. Example 64). *In any first-order model of STLC, we have $\text{lam } q : \text{Tm} \diamond (\iota \Rightarrow \iota)$. If we write the implicit arguments, this is $\text{lam } \{\diamond\} \{\iota\} \{\iota\} (q \{\diamond\} \{\iota\})$. We can move this program into any context Γ by instantiating it by $\epsilon \{\Gamma\} : \text{Sub } \Gamma \diamond$, and obtain $(\text{lam } q)[\epsilon] \stackrel{\text{lam}[1]}{=} \text{lam } (q[\epsilon \circ p, q]) \stackrel{\triangleright \beta_2}{=} \text{lam } q$. The original and the instantiated terms have different implicit arguments: the right hand side $\text{lam } q$ is actually $\text{lam } \{\Gamma\} \{\iota\} \{\iota\} (q \{\Gamma\} \{\iota\})$.*

Applying the identity function to some argument gives $\text{lam } q \cdot u \stackrel{\beta}{=} q[\text{id}, u] \stackrel{\triangleright \beta_2}{=} u$.

Example 219 (C.f. Example 69). *In any first-order model of STLC, the double function is defined as*

$$\text{lam} \left(\text{lam} \left((q[p]) \cdot ((q[p]) \cdot q) \right) \right) : \text{Tm} \diamond ((A \Rightarrow A) \Rightarrow A \Rightarrow A)$$

Exercise 220. *Derive $\left(\text{lam} \left(\text{lam} \left((q[p]) \cdot ((q[p]) \cdot q) \right) \right) \right) [\epsilon] = \text{lam} \left(\text{lam} \left((q[p]) \cdot ((q[p]) \cdot q) \right) \right)$.*

Exercise 221. *Derive $\left(\text{lam} \left(\text{lam} \left((q[p]) \cdot ((q[p]) \cdot q) \right) \right) \right) \cdot t \cdot u = t \cdot (t \cdot u)$.*

4.2 The general translation

The recipe for translating a SOGAT into a GAT is the following:

- the GAT starts with a category with a terminal object,
- closed sorts become presheaves on this category,
- open sorts become dependent presheaves,
- operations become natural transformations (that is, operations indexed by contexts together with substitution laws),
- equations become equations indexed by contexts,
- sorts which are in $\mathbf{U}^+$ (which appear on the left hand side of an arrow in an operator argument) moreover have local representability structure (that is, come with a context extension operation)
- second-order arguments become context extensions,
- Π^+ -applications become explicit instantiations,
- Π^+ -abstractions become weakenings,
- variables bound by Π^+ -functions become De Bruijn indices.

Here we show more examples. For the precise algorithm, see [KX24].

In Definition 213, we used some cleverness: we knew that there are no second-order Ty-operators (no binders in Ty, no types refer to variables), so we made Ty a closed sort. The generic algorithm does not know about this (we might have dependent types), so the first part of the GAT will be simply the result of translating the SOGAT signature $\Sigma \mathbf{U} \lambda \text{Ty}. \text{Ty} \Rightarrow \mathbf{U}^+$ (types and terms indexed by types where we only have term-variables):

Definition 222 (Category with family, CwF).

Con	: Set	[id]	: A[id] = A
Sub	: Con $\rightarrow$ Con $\rightarrow$ Set	Tm	: (Γ : Con) $\rightarrow$ Ty $\Gamma \rightarrow$ Set
$- \circ -$	: Sub $\Delta \Gamma \rightarrow$ Sub $\Theta \Delta \rightarrow$ Sub $\Theta \Gamma$	$-[-]$	: Tm $\Gamma A \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Tm } \Delta (A[\gamma])$
ass	: $(\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta)$	[$\circ$]	: $a[\gamma \circ \delta] = a[\gamma][\delta]$
id	: Sub $\Gamma \Gamma$	[id]	: $a[\text{id}] = a$
idl	: $\text{id} \circ \gamma = \gamma$	$- \triangleright -$	: (Γ : Con) $\rightarrow$ Ty $\Gamma \rightarrow$ Con
idr	: $\gamma \circ \text{id} = \gamma$	$- , -$	: $(\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Tm } \Delta (A[\gamma]) \rightarrow \text{Sub } \Delta (\Gamma \triangleright A)$
$\diamond$	: Con	p	: Sub $(\Gamma \triangleright A) \Gamma$
ϵ	: Sub $\Gamma \diamond$	q	: Tm $(\Gamma \triangleright A) (A[p])$
$\diamond \eta$	: $(\sigma : \text{Sub } \Gamma \diamond) \rightarrow \sigma = \epsilon$	$\triangleright \beta_1$	: $p \circ (\gamma, a) = \gamma$
Ty	: Con $\rightarrow$ Set	$\triangleright \beta_2$	: $q[\gamma, a] = a$
$-[-]$	: Ty $\Gamma \rightarrow$ Sub $\Delta \Gamma \rightarrow$ Ty Δ	$\triangleright \eta$	: $\sigma = (p \circ \sigma, q[\sigma])$
[$\circ$]	: $A[\gamma \circ \delta] = A[\gamma][\delta]$		

Exercise 223. Define the syntax of *CwF* without using quotients or *QIITs*. This involves proving its induction principle.

The unoptimised first-order version of STLC is the following.

Definition 224 (First-order model of STLC (unoptimised version), see Definition 62 and Example 209). We extend *CwF* (Definition 222) with the following components.

$$\begin{array}{ll}
\iota & : \text{Ty } \Gamma \\
\iota[] & : \iota[\gamma] = \iota \\
- \Rightarrow - & : \text{Ty } \Gamma \rightarrow \text{Ty } \Gamma \rightarrow \text{Ty } \Gamma \\
\Rightarrow[] & : (A \Rightarrow B)[\gamma] = (A[\gamma]) \Rightarrow (B[\gamma]) \\
\text{lam} & : \text{Tm } (\Gamma \triangleright A) (B[p]) \rightarrow \text{Tm } \Gamma (A \Rightarrow B) \\
\text{lam}[] & : (\text{lam } b)[\gamma] = \text{lam } (b[\gamma \circ p, q]) \\
- \cdot - & : \text{Tm } \Gamma (A \Rightarrow B) \rightarrow \text{Tm } \Gamma A \rightarrow \text{Tm } \Gamma B \\
\cdot[] & : (f \cdot a)[\gamma] = (f[\gamma]) \cdot (a[\gamma]) \\
\beta & : \text{lam } b \cdot a = b[\text{id}, a] \\
\eta & : f = \text{lam } (f[p] \cdot q) \\
[\text{id}] & : a[\text{id}] = a
\end{array}$$

The unoptimised version of STLC has more models than the optimised version, but the syntaxes are equivalent.

Actually, Definition 62 does not completely determine the shape of the application operator in the first-order version. However, the formal definition by a SOGAT signature completely determines the first-order version. Here are the three different versions, we used the first one in Example 209.

second-order signature version

$$\text{el}^+ (Tm \cdot (\text{arr} \cdot A \cdot B)) \Rightarrow \text{el}^+ (Tm \cdot A) \Rightarrow \text{El} (\text{el}^+ (Tm \cdot B))$$

$$\text{el}^+ (Tm \cdot (\text{arr} \cdot A \cdot B)) \Rightarrow \text{El} (Tm \cdot A \Rightarrow^+ \text{el}^+ (Tm \cdot B))$$

$$\text{El} (Tm \cdot (\text{arr} \cdot A \cdot B) \Rightarrow^+ Tm \cdot A \Rightarrow^+ \text{el}^+ (Tm \cdot B))$$

first-order version

$$- \cdot - : \text{Tm } \Gamma (A \Rightarrow B) \rightarrow \text{Tm } \Gamma A \rightarrow \text{Tm } \Gamma B$$

$$\cdot[] : (f \cdot a)[\gamma] = (f[\gamma]) \cdot (a[\gamma])$$

$$\text{app} : \text{Tm } \Gamma (A \Rightarrow B) \rightarrow \text{Tm } (\Gamma \triangleright A) (B[p])$$

$$\text{app}[] : (\text{app } t)[\gamma \circ p, q] = \text{app } (t[\gamma])$$

$$\text{APP} : \text{Tm } (\Gamma \triangleright A \Rightarrow B \triangleright A[p]) (B[p][p])$$

$$\text{APP}[] : \text{APP}[(\gamma \circ p, q) \circ p, q] = \text{APP}$$

TODO: show that substitution for app is equivalent...

Definition 225 (First-order model of mini-MLTT, see Definition 195). We extend *CwF* (Definition 222) with the following components.

$$\begin{array}{ll}
\iota & : \text{Ty } \Gamma \\
\iota[] & : \iota[\gamma] = \iota \\
\Pi & : (A : \text{Ty } \Gamma) \rightarrow \text{Ty } (\Gamma \triangleright A) \rightarrow \text{Ty } \Gamma \\
\Pi[] & : (\Pi A B)[\gamma] = \Pi (A[\gamma]) (B[\gamma \circ p, q]) \\
\text{lam} & : \text{Tm } (\Gamma \triangleright A) B \rightarrow \text{Tm } \Gamma (\Pi A B) \\
\text{lam}[] & : (\text{lam } b)[\gamma] = \text{lam } (b[\gamma \circ p, q]) \\
- \cdot - & : \text{Tm } \Gamma (\Pi A B) \rightarrow (a : \text{Tm } \Gamma A) \rightarrow \text{Tm } \Gamma (B[\text{id}, a]) \\
\cdot[] & : (f \cdot a)[\gamma] = (f[\gamma]) \cdot (a[\gamma]) \\
\beta & : \text{lam } b \cdot a = b[\text{id}, a] \\
\eta & : f = \text{lam } (f[p] \cdot q) \\
[\text{id}] & : a[\text{id}] = a
\end{array}$$

Exercise 226. Show that the equations $\cdot[], \beta, \eta$ make sense (the two sides are in the same set (meta type)).

Exercise 227 (long). What are the extra equations that we need to add to a *CwF* with $\Pi, \Sigma, \top$ to obtain something equivalent to cartesian closed categories? (CCC)

TODO: constant types.

Example 228 (Models of any GAT form a (co)complete *CwF*). For any GAT, we obtain a category of models where objects are models and morphisms are homomorphisms. This category has the structure of a *CwF* which supports constant types, $\top, \Sigma$, extensional equality, booleans, quotient types, and an initial object. We list what

some of the components are in this CwF .

Con	<i>model</i>
$\text{Sub } \Delta \Gamma$	<i>morphism from Δ to Γ</i>
$\text{Ty } \Gamma$	<i>dependent model over Γ</i>
$\text{Tm } \Gamma A$	<i>dependent morphism from Γ to A</i>
$-[-] : \text{Ty } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Ty } \Delta$	<i>reindexing a dependent model over Γ along a morphism from Δ to Γ</i>
$A[\gamma \circ \delta] = A[\gamma][\delta]$	<i>reindexing is functorial</i>
$\diamond$	<i>trivial model where all sorts are $\mathbb{1}$</i>
$- \triangleright - : (\Gamma : \text{Con}) \rightarrow \text{Ty } \Gamma \rightarrow \text{Con}$	<i>total model from a model and a dep. model over it, sorts are meta Σs</i>
$\text{p} : \text{Sub } (\Gamma \triangleright A) \Gamma$	<i>first projection morphism from a total model</i>
$\Sigma : (A : \text{Ty } \Gamma) \rightarrow \text{Ty } (\Gamma \triangleright A) \rightarrow \text{Ty } \Gamma$	<i>the meta Σ of a dependent model and another dependent over it</i>
$(\Sigma A B)[\gamma] = \Sigma (A[\gamma]) (B[\gamma \circ \text{p}, \text{q}])$	<i>total dependent models are stable under reindexing</i>
$\text{K} : \text{Con} \rightarrow \text{Ty } \Gamma$	<i>a dependent model which does not actually depend on the base model</i>
$\text{Eq} : \text{Tm } \Gamma A \rightarrow \text{Tm } \Gamma A \rightarrow \text{Ty } \Gamma$	<i>a dependent model expressing equality of two dependent morphisms</i>
$\text{I} : \text{Con}$	<i>the syntax (initial model)</i>
<i>iterator</i> : $(\Gamma : \text{Con}) \rightarrow \text{Sub I } \Gamma$	<i>iterator from the syntax to any model Γ</i>
<i>induction</i> : $(A : \text{Ty I}) \rightarrow \text{Tm I } A$	<i>induction principle of the syntax: a dependent morphism from I to any A</i>

Exercise 229 (very very long). Show that any GAT (element of Ty in the syntax of Definition 205) gives rise to a (co)complete CwF as given in Example 228. Hint: define a first-order model of Definition 205 where Con is CwF . For checking your solution, see [KKA19, Kov22].

First-order logic is interesting because it has two different kinds of variables: term and proof variables, so there are two context extensions corresponding to these (there are no formula-variables). The only optimisation that we do is to omit equations for Pf which all hold by irr.

Definition 230 (First-order model of minimal intuitionistic first-order logic, see Definition 200).

Con	: Set	$\triangleright_{\text{Tm}} \beta_2$	: $\text{q}_{\text{Tm}}[\gamma, \text{Tm } t] = t$
Sub	: $\text{Con} \rightarrow \text{Con} \rightarrow \text{Set}$	$\triangleright_{\text{Tm}} \eta$	: $\sigma = (\text{p} \circ \sigma, \text{q}[\sigma])$
$- \circ -$	: $\text{Sub } \Delta \Gamma \rightarrow \text{Sub } \theta \Delta \rightarrow \text{Sub } \theta \Gamma$	$- \supset -$	: $\text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$
ass	: $(\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta)$	$\supset []$	: $(A \supset B)[\gamma] = (A[\gamma]) \supset (B[\gamma])$
id	: $\text{Sub } \Gamma \Gamma$	$\forall$	: $\text{For } (\Gamma \triangleright_{\text{Tm}}) \rightarrow \text{For } \Gamma$
idl	: $\text{id} \circ \gamma = \gamma$	$\forall []$	: $(\forall A)[\gamma] = \forall (A[\gamma \circ \text{p}_{\text{Tm}}, \text{Tm } \text{q}_{\text{Tm}}])$
idr	: $\gamma \circ \text{id} = \gamma$	Eq	: $\text{Tm } \Gamma \rightarrow \text{Tm } \Gamma \rightarrow \text{For } \Gamma$
$\diamond$	: Con	$\text{Eq}[]$	: $(\text{Eq } t t')[\gamma] = \text{Eq } (t[\gamma]) (t'[\gamma])$
ϵ	: $\text{Sub } \Gamma \diamond$	Pf	: $(\Gamma : \text{Con}) \rightarrow \text{For } \Gamma \rightarrow \text{Set}$
$\diamond \eta$	: $(\sigma : \text{Sub } \Gamma \diamond) \rightarrow \sigma = \epsilon$	$-[-]$	: $\text{Pf } \Gamma A \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Pf } \Delta (A[\gamma])$
For	: $\text{Con} \rightarrow \text{Set}$	irr	: $(p q : \text{Pf } \Gamma A) \rightarrow p = q$
$-[-]$	: $\text{For } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{For } \Delta$	$- \triangleright_{\text{Pf}} -$	: $(\Gamma : \text{Con}) \rightarrow \text{For } \Gamma \rightarrow \text{Con}$
$[o]$	: $A[\gamma \circ \delta] = A[\gamma][\delta]$	$- ,_{\text{Pf}} -$	: $(\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Pf } \Delta (A[\gamma]) \rightarrow \text{Sub } \Delta (\Gamma \triangleright_{\text{Pf}} A)$
$[\text{id}]$	: $A[\text{id}] = A$	p_{Pf}	: $\text{Sub } (\Gamma \triangleright_{\text{Pf}} A) \Gamma$
Tm	: $\text{Con} \rightarrow \text{Set}$	q_{Pf}	: $\text{Pf } (\Gamma \triangleright_{\text{Pf}} A) (A[\text{p}_{\text{Pf}}])$
$-[-]$	: $\text{Tm } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta$	$\triangleright_{\text{Pf}} \beta_1$	: $\text{p}_{\text{Pf}} \circ (\gamma, \text{p}_{\text{Pf}} p) = \gamma$
$[o]$	: $t[\gamma \circ \delta] = t[\gamma][\delta]$	$\triangleright_{\text{Pf}} \eta$	: $\sigma = (\text{p}_{\text{Pf}} \circ \sigma, \text{p}_{\text{Pf}} \text{q}_{\text{Pf}}[\sigma])$
$[\text{id}]$	: $t[\text{id}] = t$	$\text{intro}_{\supset}$	: $\text{Pf } (\Gamma \triangleright_{\text{Pf}} A) (B[\text{p}_{\text{Pf}}]) \rightarrow \text{Pf } \Gamma (A \supset B)$
$- \triangleright_{\text{Tm}}$	: $\text{Con} \rightarrow \text{Con}$	$\text{elim}_{\supset}$	: $\text{Pf } \Gamma (A \supset B) \rightarrow \text{Pf } \Gamma A \rightarrow \text{Pf } \Gamma B$
$- ,_{\text{Tm}} -$	: $\text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub } \Delta (\Gamma \triangleright_{\text{Tm}})$	$\text{intro}_{\forall}$	: $\text{Pf } (\Gamma \triangleright_{\text{Tm}}) A \rightarrow \text{Pf } \Gamma (\forall A)$
p_{Tm}	: $\text{Sub } (\Gamma \triangleright_{\text{Tm}}) \Gamma$	$\text{elim}_{\forall}$	: $\text{Pf } \Gamma (\forall A) \rightarrow (t : \text{Tm } \Gamma) \rightarrow \text{Pf } \Gamma (A[\text{id}, \text{Tm } t])$
q_{Tm}	: $\text{Tm } (\Gamma \triangleright_{\text{Tm}})$	intro_{Eq}	: $(t : \text{Tm } \Gamma) \rightarrow \text{Pf } \Gamma (\text{Eq } t t)$
$\triangleright_{\text{Tm}} \beta_1$	: $\text{p}_{\text{Tm}} \circ (\gamma, \text{Tm } t) = \gamma$	elim_{Eq}	: $\text{Pf } \Gamma (A[\text{id}, t]) \rightarrow \text{Pf } (\text{Eq } t t') \rightarrow \text{Pf } \Gamma (A[\text{id}, t'])$

The syntax of monoids over A is definable without quotients, see Exercise 51. The syntax of STLC is definable without quotients using normal forms and proving normalisation via hereditary substitution [KA10]. Definable quotients were studied by Nuo Li [Li15].

Exercise 231 (long). *Show that the syntax of Definition 230 can be defined in Agda without quotients (see [Avr23]); we need Pf to be in the universe of propositions to justify the only equation in (the second-order version of) this theory.*

Quotients not definable via normal forms are e.g. the syntax of untyped combinator calculus or lambda calculus. The simplest example is unordered pairs of a given type A . There are theories where it is open whether they are definable without quotients:

Food for thought 232. *First-order logic is a theory without equations, and its first-order syntax is definable without quotients. In objective type theory [vdBdB21] (also called weak type theory [BW19], axiomatic type theory) there are also no equations, the equations are expressed as elements of the Id type. Although the second-order version of this theory does not have any equations, it is open whether its first-order syntax is definable without quotients.*

System F is interesting because there are two different kind of variables: type and term variables.

Definition 233 (First-order model of System F, see Definition 160). *We extend CwF (Definition 222) with the following components.*

$$\begin{array}{ll}
\multimap_{\text{Ty}} & : \text{Con} \rightarrow \text{Con} \\
-,_{\text{Ty}} - & : \text{Sub } \Delta \Gamma \rightarrow \text{Ty } \Delta \rightarrow \text{Sub } \Delta (\Gamma \multimap_{\text{Ty}}) \\
p_{\text{Ty}} & : \text{Sub } (\Gamma \multimap_{\text{Ty}}) \Gamma \\
q_{\text{Ty}} & : \text{Tm } (\Gamma \multimap_{\text{Ty}}) (A[p_{\text{Ty}}]) \\
\multimap_{\text{Ty}} \beta_1 & : p_{\text{Ty}} \circ (\gamma,_{\text{Ty}} A) = \gamma \\
\multimap_{\text{Ty}} \beta_2 & : q_{\text{Ty}}[\gamma,_{\text{Ty}} A] = A \\
\multimap_{\text{Ty}} \eta & : \sigma = (p_{\text{Ty}} \circ \sigma,_{\text{Ty}} q_{\text{Ty}}[\sigma]) \\
\forall & : \text{Ty } (\Gamma \multimap_{\text{Ty}}) \rightarrow \text{Ty } \Gamma \\
\forall [] & : (\forall A)[\gamma] = \forall (A[\gamma \circ p_{\text{Ty}},_{\text{Ty}} q_{\text{Ty}}]) \\
\text{Lam} & : \text{Tm } (\Gamma \multimap_{\text{Ty}}) A \rightarrow \text{Tm } \Gamma (\forall A) \\
\text{Lam} [] & : (\text{Lam } a)[\gamma] = \text{Lam } (a[\gamma \circ p_{\text{Ty}},_{\text{Ty}} q_{\text{Ty}}]) \\
- \bullet - & : \text{Tm } \Gamma (\forall A) \rightarrow (B : \text{Ty } \Gamma) \rightarrow \text{Tm } \Gamma (A[\text{id},_{\text{Ty}} B]) \\
\bullet [] & : (t \bullet B)[\gamma] = (t[\gamma]) \bullet (B[\gamma]) \\
\forall \beta & : (\text{Lam } a) \bullet B = a[\text{id},_{\text{Ty}} B] \\
\forall \eta & : t = \text{Lam } (t[p_{\text{Ty}}] \bullet q_{\text{Ty}})
\end{array}$$

In System F_ω , we still have two extensions, but type variables are now indexed with their kinds, so their context extension operation is more interesting. We could optimise the following definition such that Kind does not depend on Con . We write the universal properties in concise form.

Definition 234 (First-order model of System F_ω , see Definition 173).

$\text{Con} : \text{Set}$	$\Rightarrow\beta : \text{LAM } A \bullet B = A[\text{id}, \text{Ty } B]$
$\text{Sub} : \text{Con} \rightarrow \text{Con} \rightarrow \text{Set}$	$\Rightarrow\eta : F = \text{LAM } (F[\text{pTy}] \bullet \text{qTy})$
$- \circ - : \text{Sub } \Delta \Gamma \rightarrow \text{Sub } \theta \Delta \rightarrow \text{Sub } \theta \Gamma$	$* : \text{Kind } \Gamma$
$\text{ass} : (\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta)$	$*[] : *[\gamma] = *$
$\text{id} : \text{Sub } \Gamma \Gamma$	$\text{Tm} : (\Gamma : \text{Con}) \rightarrow \text{Ty } \Gamma * \rightarrow \text{Set}$
$\text{idl} : \text{id} \circ \gamma = \gamma$	$-[-] : \text{Tm } \Gamma A \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Tm } \Delta (A[\gamma])$
$\text{idr} : \gamma \circ \text{id} = \gamma$	$[\circ] : a[\gamma \circ \delta] = a[\gamma][\delta]$
$\diamond : \text{Con}$	$[\text{id}] : a[\text{id}] = a$
$\epsilon : \text{Sub } \Gamma \diamond$	$\neg, \text{Tm} - : (\gamma : \text{Sub } \Delta \Gamma) \times \text{Tm } \Delta (A[\gamma]) \rightarrow$
$\diamond\eta : (\sigma : \text{Sub } \Gamma \diamond) \rightarrow \sigma = \epsilon$	$\text{Sub } \Delta (\Gamma \triangleright_{\text{Tm}} A)$
$\text{Kind} : \text{Con} \rightarrow \text{Set}$	$\text{pTm} : \text{Sub } (\Gamma \triangleright_{\text{Tm}} A) \Gamma$
$-[-] : \text{Kind } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Kind } \Delta$	$\text{qTm} : \text{Tm } (\Gamma \triangleright_{\text{Tm}} A) (A[\text{pTm}])$
$[\circ] : K[\gamma \circ \delta] = K[\gamma][\delta]$	$\triangleright_{\text{Tm}}\beta_1 : \text{pTm} \circ (\gamma, \text{Tm } a) = \gamma$
$[\text{id}] : K[\text{id}] = K$	$\triangleright_{\text{Tm}}\beta_2 : \text{qTm}[\gamma, \text{Tm } a] = a$
$\text{Ty} : (\Gamma : \text{Con}) \rightarrow \text{Ty } \Gamma \rightarrow \text{Set}$	$\triangleright_{\text{Tm}}\eta : \sigma = (\text{p} \circ \sigma, \text{Tm } \text{qTm}[\sigma])$
$-[-] : \text{Ty } \Gamma K \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Ty } \Delta (K[\gamma])$	$\forall : \text{Ty } (\Gamma \triangleright_{\text{Ty}} K) * \rightarrow \text{Ty } \Gamma *$
$[\circ] : A[\gamma \circ \delta] = A[\gamma][\delta]$	$\forall[] : (\forall A)[\gamma] = \forall (A[\gamma \circ \text{pTy} \circ \text{Ty } \text{qTy}])$
$[\text{id}] : A[\text{id}] = A$	$\text{Lam} : \text{Tm } (\Gamma \triangleright_{\text{Ty}} K) A \rightarrow \text{Tm } \Gamma (\forall A)$
$\neg \triangleright_{\text{Ty}} - : (\Gamma : \text{Con}) \rightarrow \text{Kind } \Gamma \rightarrow \text{Con}$	$\text{Lam}[] : (\text{Lam } a)[\gamma] = \text{Lam } (a[\gamma \circ \text{pTy}, \text{Ty } \text{qTy}])$
$\neg, \text{Ty} - : (\gamma : \text{Sub } \Delta \Gamma) \times \text{Ty } \Delta (K[\gamma]) \rightarrow$	$- \bullet - : \text{Tm } \Gamma (\forall A) \rightarrow (B : \text{Ty } \Gamma K) \rightarrow$
$\text{Sub } \Delta (\Gamma \triangleright_{\text{Ty}} K)$	$\text{Tm } \Gamma (A[\text{id}, \text{Ty } B])$
$\text{pTy} : \text{Sub } (\Gamma \triangleright_{\text{Ty}} K) \Gamma$	$\bullet[] : (t \bullet A)[\gamma] = (t[\gamma]) \bullet (A[\gamma])$
$\text{qTy} : \text{Ty } (\Gamma \triangleright_{\text{Ty}} K) (K[\text{pTy}])$	$\forall\beta : (\text{Lam } a) \bullet B = a[\text{id}, \text{Ty } B]$
$\triangleright_{\text{Ty}}\beta_1 : \text{pTy} \circ (\gamma, \text{Ty } A) = \gamma$	$\forall\eta : t = \text{Lam } (t[\text{pTy}] \bullet \text{qTy})$
$\triangleright_{\text{Ty}}\beta_2 : \text{qTy}[\gamma, \text{Ty } A] = A$	$\Rightarrow - : \text{Ty } \Gamma * \rightarrow \text{Ty } \Gamma * \rightarrow \text{Ty } \Gamma *$
$\triangleright_{\text{Ty}}\eta : \sigma = (\text{p} \circ \sigma, \text{Ty } \text{qTy}[\sigma])$	$\Rightarrow[] : (A \Rightarrow B)[\gamma] = (A[\gamma]) \Rightarrow (B[\gamma])$
$\Rightarrow - : \text{Kind } \Gamma \rightarrow \text{Kind } \Gamma \rightarrow \text{Kind } \Gamma$	$\text{lam} : \text{Tm } (\Gamma \triangleright_{\text{Tm}} A) B \rightarrow \text{Tm } \Gamma (A \Rightarrow B)$
$\Rightarrow[] : (K \Rightarrow L)[\gamma] = (K[\gamma]) \Rightarrow (L[\gamma])$	$\text{lam}[] : (\text{lam } b)[\gamma] = \text{lam } (b[\gamma \circ \text{pTm}, \text{Tm } \text{qTm}])$
$\text{LAM} : \text{Ty } (\Gamma \triangleright_{\text{Ty}} K) L \rightarrow \text{Ty } \Gamma (K \Rightarrow L)$	$- \cdot - : \text{Tm } \Gamma (A \Rightarrow B) \rightarrow \text{Tm } \Gamma A \rightarrow \text{Tm } \Gamma B$
$\text{LAM}[] : (\text{LAM } A)[\gamma] = \text{LAM } (A[\gamma \circ \text{pTy}, \text{Ty } \text{qTy}])$	$\cdot[] : (t \cdot a)[\gamma] = (t[\gamma]) \cdot (a[\gamma])$
$- \bullet - : \text{Ty } \Gamma (K \Rightarrow L) \rightarrow \text{Ty } \Gamma K \rightarrow \text{Ty } \Gamma L$	$\Rightarrow\beta : (\text{lam } b) \cdot a = b[\text{id}, a]$
$\bullet[] : (F \bullet B)[\gamma] = (F[\gamma]) \bullet (B[\gamma])$	$\Rightarrow\eta : t = \text{lam } (t[\text{pTm}] \cdot \text{qTm})$

Exercise 235. Check that instantiation for terms make sense (is well-typed in the metatheory).

TODO: add MLTT with equality etc.

5 Admissibility in SOGATs

It is not a bad position to define languages abstractly as SOGATs, and then when proving properties of the language, we have to work with its more verbose translated GAT-version. It is nice that all the substitution rules are generated automatically, and we don't have to worry that we missed one or wrote down one incorrectly.

However, we can do better. Some proofs about first-order models can be factored through a generic scoping construction.

In this section, we fix the SOGAT to mini-MLTT: its second-order model is Definition 195, its first-order model is Definition 225.

5.1 Proofs about closed syntactic terms

$\mathbf{l}$ denotes the first-order model of mini-MLTT.

Definition 236 (Second-order dependent model).

$$\begin{aligned}
\text{Ty} & : \text{Ty}_I \diamond_I \rightarrow \text{Set} \\
\text{Tm} & : \{A_I : \text{Ty}_I \diamond_I\} \rightarrow \text{Ty } A_I \rightarrow \text{Tm}_I \diamond_I A_I \rightarrow \text{Set} \\
\iota & : \text{Ty } (\iota \{ \diamond_I \}) \\
\Pi & : \{A_I : \text{Ty}_I \diamond_I\} \{A : \text{Ty } A_I\} \{B_I : \text{Ty}_I (\diamond_I \triangleright_I A_I)\} \rightarrow (\{a_I : \text{Tm}_I \diamond_I A_I\} \rightarrow \text{Tm } A_I a_I \rightarrow \text{Ty } (B_I[\epsilon_I, \iota a_I]_I)) \rightarrow \text{Ty } (\Pi_I A_I B_I) \\
\text{lam} & : ((a : \text{Tm } A a_I) \rightarrow \text{Tm } (B a) (b_I[\epsilon_I, \iota a_I]_I)) \rightarrow \text{Tm } (\Pi A B) (\text{lam}_I b_I) \\
- \cdot - & : \text{Tm } (\Pi A B) f_I \rightarrow (a : \text{Tm } A a_I) \rightarrow \text{Tm } (B a) (f_I \cdot_I a_I) \\
\beta & : \text{lam } b \cdot a = b a \\
\eta & : f = \text{lam } \lambda a. f \cdot a
\end{aligned}$$

The left hand side of β has type $\text{Tm } (B a) (\text{lam}_I b_I \cdot_I a_I)$, the right hand side has type $\text{Tm } (B a) (b_I[\epsilon_I, \iota a_I]_I)$, which are equal by β_I .

Exercise 237. Similarly check that the two sides of η are in the same set.

The above dependent model only says something about closed types and closed terms. We have a general construction to make it work for any type and term:

Definition 238 (Scone-contextualisation). Assuming a second-order dependent model, we obtain the following first-order dependent model.

$$\begin{aligned}
\text{Con } \Gamma_I & := \text{Sub}_I \diamond_I \Gamma_I \rightarrow \text{Set} \\
\text{Sub } \Delta \Gamma \gamma_I & := \Delta \delta_I \rightarrow \Gamma (\gamma_I \diamond_I \delta_I) \\
\delta \circ \gamma & := \lambda \theta_*. \delta (\gamma \theta_*) \\
\text{id} & := \lambda \gamma_*. \gamma_* \\
\diamond & := \lambda _ . \mathbb{1} \\
\text{Ty } \Gamma A_I & := (\gamma_* : \Gamma \gamma_I) \rightarrow \text{Ty } (A_I[\gamma_I]_I) \\
A[\gamma] & := \lambda \delta_*. A (\gamma \delta_*) \\
\text{Tm } \Gamma A a_I & := (\gamma_* : \Gamma \gamma_I) \rightarrow \text{Tm } (A \gamma_*) (a_I[\gamma_I]_I) \\
a[\gamma] & := \lambda \delta_*. a (\gamma \delta_*) \\
\Gamma \triangleright A & := \lambda (\gamma_I, \iota a_I). (\gamma_* : \Gamma \gamma_I) \times \text{Tm } (A \gamma_I) a_I \\
\gamma, a & := \lambda \delta_*. (\gamma \delta_*, a \delta_*) \\
p & := \lambda (\gamma_*, a_*) . \gamma_* \\
q & := \lambda (\gamma_*, a_*) . a_* \\
\Pi A B & := \lambda \gamma_*. \Pi (A \gamma_*) (\lambda a_*. B (\gamma_*, a_*)) \\
\text{lam } b & := \lambda \gamma_*. \text{lam } (\lambda a_*. b (\gamma_*, a_*)) \\
f \cdot a & := \lambda \gamma_*. f \gamma_* \cdot a \gamma_*
\end{aligned}$$

Example 239 (Canonicity for mini-MLTT). We define the following second-order dependent model:

$$\begin{aligned}
\text{Ty } A_I & := \text{Tm}_I \diamond_I A_I \rightarrow \text{Set} \\
\text{Tm } A a_I & := A a_I \\
\iota & := \lambda t_I . \mathbb{0} \\
\Pi A B & := \lambda f_I . \{a_I : \text{Tm}_I \diamond_I A_I\} (a_* : A a_I) \rightarrow B a_* (f_I \cdot_I a_I) \\
\text{lam } b & := b \\
f \cdot a & := f a
\end{aligned}$$

This is the canonicity proof without boilerplate. From this, we get e.g. that the set $\text{Tm}_I \diamond_I \iota_I$ is empty: applying (induction into the dependent scone-contextualisation of the canonicity second-order dependent model) on such a t_I we obtain an element of $(\sigma_* : \diamond \sigma_I) \rightarrow \text{Tm } (\iota \sigma_*) (t_I[\sigma_I]_I) = \mathbb{1} \rightarrow \mathbb{0}$.

5.2 Internal languages of presheaf models of MLTT

If we say that a model of a SOGAT is a model of its first-orderification, then did the derived things in Section 3 made sense?

How do we know that the $\text{SOGAT} \rightarrow \text{GAT}$ translation is the correct one? One requirement is that they can be used to build the same things: if we build something assuming a second-order model, then the corresponding first-order thing should be also buildable using De Bruijn combinators and explicit weakenings etc. How do we make this sure?

The derivable operations and equations of Section 3 are also derivable in the result of the translations.

5.3 Proofs about open syntactic terms

Renamings.

Internal to presheaves over renamings, we have a first-order model which we also denote $\mathbf{I}$.

References

- [Ack28] Wilhelm Ackermann. Zum hilbertschen aufbau der reellen zahlen. *Mathematische Annalen*, 99:118–133, 1928. URL: <http://eudml.org/doc/159248>.
- [ACKS23] Danil Annenkov, Paolo Capriotti, Nicolai Kraus, and Christian Sattler. Two-level type theory and applications. *Math. Struct. Comput. Sci.*, 33(8):688–743, 2023. URL: <https://doi.org/10.1017/S0960129523000130>, doi:10.1017/S0960129523000130.
- [ACKS24] Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi, and Michael Shulman. Internal parametricity, without an interval. *Proc. ACM Program. Lang.*, 8(POPL):2340–2369, 2024. doi:10.1145/3632920.
- [AKSV23] Thorsten Altenkirch, Ambrus Kaposi, Artjoms Sinkarovs, and Tamás Vég. Combinatory logic and lambda calculus are equal, algebraically. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPICs*, pages 24:1–24:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. URL: <https://doi.org/10.4230/LIPICs.FSCD.2023.24>, doi:10.4230/LIPICs.FSCD.2023.24.
- [Alt08] Thorsten Altenkirch. From high school to university algebra. <https://people.cs.nott.ac.uk/psztxa/publ/unialg.pdf>, 2008.
- [Avr23] Samy Avrillon. Logic as a second-order generalized algebraic theory, 2023. Report on the 3-month research internship at the Faculty of Informatics of ELTE. URL: <https://github.com/MysaaJava/m1-internship/releases/download/project-report/Avrillon-02.pdf>.
- [Awo10] S. Awodey. *Category Theory*. Oxford Logic Guides. OUP Oxford, 2010. URL: <http://books.google.co.uk/books?id=-MCJ6x21C7oC>.
- [Bar91] Henk Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991. doi:10.1017/S0956796800020025.
- [BKS23] Rafaël Bocquet, Ambrus Kaposi, and Christian Sattler. For the metatheory of type theory, internal scoping is enough. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPICs*, pages 18:1–18:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPICs.FSCD.2023.18.
- [BL93] Stanley Burris and Simon Lee. Tarski’s high school identities. *The American Mathematical Monthly*, 100(3):231–236, 1993. URL: <http://www.jstor.org/stable/2324454>.
- [Boc25] Rafaël Bocquet. *Relative induction principles for second-order generalized algebraic theories*. PhD thesis, Eötvös Loránd University, Hungary, 2025. URL: <https://rafaelbocquet.gitlab.io/pdfs/thesis.pdf>.
- [BSvB24] Ulrik Torben Buchholtz and Johannes Schipp von Branitz. Primitive recursive dependent type theory. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’24*, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3661814.3662136.
- [BW19] Simon Boulrier and Théo Winterhalter. Weak type theory is rather strong. In Marc Bezem, editor, *25th International Conference on Types for Proofs and Programs, TYPES 2019*. Centre for Advanced Study at the Norwegian Academy of Science and Letters, 2019. URL: https://www.ii.uib.no/~bezem/abstracts/TYPES_2019_paper_18.

- [CD07] Denis Cousineau and Gilles Dowek. Embedding pure type systems in the lambda-pi-calculus modulo. In Simona Ronchi Della Rocca, editor, *Typed Lambda Calculi and Applications, 8th International Conference, TLCA 2007, Paris, France, June 26-28, 2007, Proceedings*, volume 4583 of *Lecture Notes in Computer Science*, pages 102–117. Springer, 2007. doi:10.1007/978-3-540-73228-0_9.
- [Csö12] Zoltán Csőrnyci. *Bevezetés a típusrendszerek elméletébe*. ELTE Eötvös Kiadó, 2012.
- [DL11] Gilles Dowek and Jean-Jacques Lévy. *Introduction to the Theory of Programming Languages*. Undergraduate Topics in Computer Science. Springer, 2011. doi:10.1007/978-0-85729-076-2.
- [Gir72] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures dans l’arithmétique d’ordre supérieure*. PhD thesis, Université Paris VII, 1972.
- [Har16] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, New York, NY, USA, 2nd edition, 2016.
- [HHP93] Robert Harper, Furio Honsell, and Gordon D. Plotkin. A framework for defining logics. *J. ACM*, 40(1):143–184, 1993. doi:10.1145/138027.138060.
- [Hin69] R. Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60, 1969. URL: <http://www.jstor.org/stable/1995158>.
- [Hof99] Martin Hofmann. Semantical analysis of higher-order abstract syntax. In *14th Annual IEEE Symposium on Logic in Computer Science, Trento, Italy, July 2-5, 1999*, pages 204–213. IEEE Computer Society, 1999. doi:10.1109/LICS.1999.782616.
- [Hut23] Graham Hutton. Programming Language Semantics: It’s Easy As 1,2,3. *Journal of Functional Programming*, 33, October 2023.
- [KA10] Chantal Keller and Thorsten Altenkirch. Hereditary substitutions for simple types, formalized. In Venanzio Capretta and James Chapman, editors, *Proceedings of the 3rd ACM SIGPLAN Workshop on Mathematically Structured Functional Programming, MSFP@ICFP 2010, Baltimore, MD, USA, September 25, 2010*, pages 3–10. ACM, 2010. doi:10.1145/1863597.1863601.
- [Kap19] Ambrus Kaposi. Reduction of indexed w-types to w-types, an agda formalisation. <https://akaposi.github.io/IW.agda>, 2019.
- [KKA19] Ambrus Kaposi, András Kovács, and Thorsten Altenkirch. Constructing quotient inductive-inductive types. *Proc. ACM Program. Lang.*, 3(POPL):2:1–2:24, 2019. doi:10.1145/3290315.
- [Kov22] András Kovács. *Type-Theoretic Signatures for Algebraic Theories and Inductive Types*. PhD thesis, Eötvös Loránd University, Hungary, 2022. URL: <https://arxiv.org/pdf/2302.08837.pdf>.
- [KvR20] Ambrus Kaposi and Jakob von Raumer. A syntax for mutual inductive families. In Zena M. Ariola, editor, *5th International Conference on Formal Structures for Computation and Deduction, FSCD 2020, June 29-July 6, 2020, Paris, France (Virtual Conference)*, volume 167 of *LIPIcs*, pages 23:1–23:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. URL: <https://doi.org/10.4230/LIPIcs.FSCD.2020.23>, doi:10.4230/LIPIcs.FSCD.2020.23.
- [KX24] Ambrus Kaposi and Szumi Xie. Second-order generalised algebraic theories: Signatures and first-order semantics. In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction, FSCD 2024, July 10-13, 2024, Tallinn, Estonia*, volume 299 of *LIPIcs*, pages 10:1–10:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. URL: <https://doi.org/10.4230/LIPIcs.FSCD.2024.10>, doi:10.4230/LIPIcs.FSCD.2024.10.
- [Li15] Nuo Li. *Quotient types in type theory*. Thesis. University of Nottingham, Department of Computer Science, 2015. URL: <http://eprints.nottingham.ac.uk/28941>.
- [Lor25] Jem Lord. Easy parametricity. Workshop on Homotopy Type Theory / Univalent Foundations 2025, 2025. URL: https://hott-uf.github.io/2025/abstracts/HotTUF_2025_paper_21.pdf.
- [Mil78] Robin Milner. A theory of type polymorphism in programming. *J. Comput. Syst. Sci.*, 17(3):348–375, 1978. doi:10.1016/0022-0000(78)90014-4.
- [Moe22] Hugo Moeneclaey. *Cubical models are cofreely parametric. (Les modèles cubiques sont colibrement paramétriques)*. PhD thesis, Paris Cité University, France, 2022. URL: <https://tel.archives-ouvertes.fr/tel-04435596>.

- [MP88] John C. Mitchell and Gordon D. Plotkin. Abstract types have existential type. *ACM Trans. Program. Lang. Syst.*, 10(3):470–502, July 1988. doi:10.1145/44501.45065.
- [Par89] Michel Parigot. On the representation of data in lambda-calculus. In Egon Börger, Hans Kleine Büning, and Michael M. Richter, editors, *CSL '89, 3rd Workshop on Computer Science Logic, Kaiserslautern, Germany, October 2-6, 1989, Proceedings*, volume 440 of *Lecture Notes in Computer Science*, pages 309–321. Springer, 1989. doi:10.1007/3-540-52753-2_47.
- [PdAC⁺23] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, and Brent Yorgey. *Logical Foundations*, volume 1 of *Software Foundations*. Electronic textbook, 2023. Version 6.5, <http://softwarefoundations.cis.upenn.edu>.
- [Pie02] Benjamin C. Pierce. *Types and programming languages*. MIT Press, 2002.
- [Rey74] John C. Reynolds. Towards a theory of type structure. In Bernard J. Robinet, editor, *Programming Symposium, Proceedings Colloque sur la Programmation, Paris, France, April 9-11, 1974*, volume 19 of *Lecture Notes in Computer Science*, pages 408–423. Springer, 1974. doi:10.1007/3-540-06859-7_148.
- [Rey83] John C. Reynolds. Types, abstraction and parametric polymorphism. In R. E. A. Mason, editor, *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress, Paris, France, September 19-23, 1983*, pages 513–523. North-Holland/IFIP, 1983.
- [Sch24] Moses Schönfinkel. Über die bausteine der mathematischen logik. *Mathematische Annalen*, 92(3):305–316, Sep 1924. doi:10.1007/BF01448013.
- [Sch17] Gabriel Scherer. Deciding equivalence with sums and the empty type. In Giuseppe Castagna and Andrew D. Gordon, editors, *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, pages 374–386. ACM, 2017. doi:10.1145/3009837.3009901.
- [Ste22] Jonathan Sterling. *First Steps in Synthetic Tait Computability: The Objective Metatheory of Cubical Type Theory*. PhD thesis, Carnegie Mellon University, USA, 2022. URL: <https://doi.org/10.1184/r1/19632681.v1>, doi:10.1184/R1/19632681.V1.
- [Tai67] William W. Tait. Intensional interpretations of functionals of finite type I. *J. Symb. Log.*, 32(2):198–212, 1967. doi:10.2307/2271658.
- [vdBdB21] Benno van den Berg and Martijn den Besten. Quadratic type checking for objective type theory. *CoRR*, abs/2102.00905, 2021. URL: <https://arxiv.org/abs/2102.00905>, arXiv:2102.00905.
- [Wad89] Philip Wadler. Theorems for free! In Joseph E. Stoy, editor, *Proceedings of the fourth international conference on Functional programming languages and computer architecture, FPCA 1989, London, UK, September 11-13, 1989*, pages 347–359. ACM, 1989. doi:10.1145/99370.99404.
- [Wad90] Philip Wadler. Recursive types for free! <https://homepages.inf.ed.ac.uk/wadler/papers/free-rectypes/free-rectypes.txt>, 1990.
- [Wil01] Alex Wilkie. On exponentiation - a solution to tarski’s high school algebra problem. *Quaderni di Matematica*, 6, 02 2001.
- [WKS22] Philip Wadler, Wen Kokke, and Jeremy G. Siek. *Programming Language Foundations in Agda*. August 2022. URL: <https://plfa.inf.ed.ac.uk/22.08/>.